

Machine Learning

Sara Cristina Basco

January 26, 2026

Contents

1	Introduzione al Machine Learning	7
1.1	Definizione di Machine Learning	7
1.2	Fattori abilitanti del Machine Learning	7
1.3	Applicazioni del Machine Learning	7
1.4	Cos'è un problema di apprendimento	8
1.5	Funzione ideale	8
1.6	Funzione valore	8
1.6.1	Rappresentazione approssimata della funzione valore	9
1.7	Algoritmo di apprendimento	9
1.8	Funzione obiettivo e rappresentazione	10
1.9	Dataset e generalizzazione	10
1.10	Tipologie di problemi di Machine Learning	10
1.10.1	Supervised Learning	10
1.10.2	Unsupervised Learning	10
1.10.3	Reinforcement Learning	11
1.11	Ipotesi e spazio delle ipotesi	11
1.12	Consistenza e Version Space	11
1.13	Algoritmo List-Then-Eliminate	12
1.13.1	Osservazioni	12
1.14	Rappresentazione vs Generalizzazione	13
1.15	Dati rumorosi	13
2	Performance Evaluation	15
2.1	Introduzione alla valutazione delle prestazioni	15
2.2	Errore vero ed errore campionario	15
2.2.1	Errore vero	16
2.2.2	Errore campionario	16
2.3	Intervalli di confidenza	17
2.4	Stima non distorta e trade-off training/testing	17
2.5	k-Fold Cross Validation	17
2.5.1	Descrizione dell'algoritmo	17
2.6	Overfitting	18
2.7	Metriche di valutazione	19
2.7.1	Limiti dell'accuratezza	19
2.7.2	Precision, Recall e F1-score	19
2.7.3	Metriche derivate e curva ROC	20
2.7.4	Dipendenza dal dominio applicativo	20
2.7.5	Matrice di confusione multi-classe	20

2.8	Metriche di valutazione per la regressione	21
2.8.1	Metriche percentuali e coefficiente di determinazione	23
2.8.2	Cross-validation per la regressione	23
3	Decision Tree	25
3.1	Definizione	25
3.1.1	Struttura del Decision Tree	25
3.1.2	Caratteristiche principali	26
3.2	Regole di decisione	27
3.2.1	Decision Tree come insieme di regole	28
3.2.2	Conversione di un decision tree in regole	28
3.3	Algoritmo ID3	29
3.3.1	Costruzione del decision tree	29
3.3.2	Proprietà dell'algoritmo ID3	30
3.3.3	ID3 e regole di decisione	30
3.4	Entropia e Information Gain	30
3.4.1	Entropia	31
3.4.2	Information Gain	31
3.4.3	Schema operativo	32
3.5	Overfitting nei Decision Tree	32
3.6	Pruning	33
3.6.1	Pre-pruning	33
3.6.2	Post-pruning	34
3.7	Random Forest	34
3.7.1	Aggregazione delle predizioni	35
3.7.2	Riduzione dell'overfitting	35
4	Bayesian Learning	39
4.1	Motivazione	39
4.2	Classificazione come stima probabilistica	39
4.3	ML e MAP	40
4.3.1	Maximum Likelihood Classification	40
4.3.2	Maximum A Posteriori Classification	40
4.4	Bayes Optimal Classifier	41
4.5	Naive Bayes Classifier	42
4.5.1	Definizione	42
4.5.2	Formulazione formale	43
4.5.3	Stima delle probabilità	44
5	Probabilistic Models for Classification	48
5.1	Modelli probabilistici generativi	48
5.1.1	Caso gaussiano con covarianza condivisa	48
5.1.2	Stima dei parametri tramite Maximum Likelihood	49
5.1.3	Estensione a K classi	49
5.1.4	Gaussian Naive Bayes	49
5.2	Modelli probabilistici discriminativi	49
5.2.1	Logistic Regression	50
5.2.2	Regressione logistica multi-classe	50
5.2.3	Generalizzazione e spazio delle feature	50

5.3	RIEPILOGO	51
6	Modelli lineari per la classificazione	53
6.1	Dati linearmente separabili	53
6.1.1	Dati non linearmente separabili	54
6.2	Linear Discriminant Function	54
6.2.1	Regola di decisione	54
6.3	Learning di funzioni discriminanti lineari	56
6.4	Least Squares per la classificazione	56
6.4.1	Definizione del problema	56
6.4.2	Funzione di costo	57
6.4.3	Soluzione ottima	57
6.4.4	Regola di classificazione	58
6.4.5	Sensibilità agli outlier	58
6.5	Perceptron	59
6.5.1	Definizione del problema	59
6.5.2	Modello del Perceptron	59
6.5.3	Perceptron Learning Rule	60
6.5.4	Algoritmo del Perceptron	61
6.5.5	Interpretazione neurale	63
6.6	Fisher's Linear Discriminant	64
6.6.1	Medie delle classi	64
6.6.2	Varianza intra-classe	65
6.6.3	Criterio di Fisher	65
6.7	Support Vector Machine	68
6.7.1	Formulazione formale	68
6.7.2	Soft-Margin Support Vector Machine	70
6.7.3	Interpretazione geometrica	71
6.7.4	Basis Functions e separazione non lineare	72
6.8	RIEPILOGO	73
7	Linear Models for Regression	74
7.1	Modelli lineari per funzioni lineari	74
7.2	Modelli lineari con basis functions	75
7.3	Stima dei parametri: massima verosimiglianza e minimi quadrati	77
7.4	Apprendimento sequenziale	77
7.4.1	Regolarizzazione	78
7.4.2	Regressione con output multipli	79
8	Kernel Methods	81
8.1	Dai modelli lineari alla formulazione duale	81
8.2	Gram matrix e formulazione kernelizzata	81
8.3	Kernel functions	82
8.3.1	Kernel trick	83
8.3.2	Esempio: applicazione concreta del kernel trick	83
8.4	Famiglie di kernel	84
8.5	Kernelized SVM	84
8.5.1	Kernelized SVM per la regressione	85

9	Instance Based	86
9.1	K-Nearest Neighbors: interpretazione geometrica	87
9.1.1	Effetto del parametro K sulle regioni di decisione	88
9.2	Kernelized Nearest Neighbors	88
9.3	Locally Weighted Regression	89
10	Multiple Learners	92
10.1	Voting	93
10.2	Mixture of Experts	94
10.3	Stacking	95
10.4	Cascading	96
10.5	Bagging	97
10.6	Boosting	98
10.6.1	AdaBoost	99
11	Artificial Neural Networks	104
11.1	Feedforward Neural Networks	104
11.1.1	Perché le FNN	105
11.1.2	Architettura delle Feedforward Neural Networks	106
11.2	Profondità e Teorema di Approssimazione Universale	106
11.3	Funzioni di attivazione e funzione di costo	108
11.3.1	Funzioni di attivazione dell'output layer	108
11.3.2	Funzioni di attivazione nell'hidden layer	109
11.3.3	Loss function e principio di massima verosimiglianza	111
11.4	Calcolo del gradiente	112
11.5	Back-propagation	113
11.5.1	Come si applica operativamente	114
11.5.2	Perché è efficiente	115
11.6	Stochastic Gradient Descent	115
11.6.1	Idea di base	115
11.6.2	SGD con momentum	116
11.7	Metodi di regolarizzazione	117
11.7.1	Regolarizzazione dei parametri	117
11.7.2	Dataset augmentation	117
11.7.3	Parameter sharing	117
11.7.4	Early stopping	118
11.7.5	Dropout	118
12	Convolutional Neural Networks	121
12.1	Motivazioni	121
12.2	Richiamo alla convoluzione	122
12.2.1	Cross-correlazione	124
12.2.2	Il kernel	125
12.2.3	La feature map	125
12.3	Convolutional Neural Networks	126
12.4	Convolutional layer	126
12.4.1	Convolution stage	127
12.4.2	Detector stage	130
12.4.3	Pooling stage	130

12.4.4	Dimensione delle feature map e numero di parametri	131
12.5	Dimensione della feature map di output	131
13	Unsupervised Learning	134
13.0.1	Cluster	134
13.0.2	Variabili latenti	134
13.1	Gaussian Mixture Models	135
13.1.1	Generazione dei dati con un Gaussian Mixture Model	135
13.1.2	Introduzione formale delle variabili latenti nel GMM	137
13.1.3	Distribuzione a priori delle variabili latenti	137
13.1.4	Distribuzione condizionata dei dati	137
13.1.5	Distribuzione congiunta e marginale	137
13.1.6	Distribuzione a posteriori delle variabili latenti	138
13.2	K-means Clustering	139
13.2.1	Obiettivo dell'algoritmo	139
13.2.2	Algoritmo K-means	140
13.2.3	Interpretazione geometrica	140
13.3	Expectation Maximization (EM)	142
13.3.1	Massima verosimiglianza con variabili latenti	142
13.3.2	Idea dell'algoritmo EM	142
13.3.3	E-step (Expectation step)	142
13.3.4	M-step (Maximization step)	143
13.3.5	Algoritmo EM	143
13.3.6	Relazione con K-means	143
13.3.7	Interpretazione dell'algoritmo EM	143
13.4	General Expectation Maximization Problem	145
13.4.1	Definizione del problema	145
13.4.2	Funzione ausiliaria Q	145
13.4.3	Algoritmo EM generale	145
13.4.4	Proprietà dell'algoritmo EM	146
13.4.5	Applicazioni	146
14	Dimensionally reduction	148
14.1	Variabili latenti	148
14.2	Manifold dei dati	149
14.3	Principal Component Analysis	149
14.3.1	Centratura dei dati e matrice di covarianza	150
14.3.2	Massimizzazione della varianza	150
14.4	PCA come minimizzazione dell'errore di ricostruzione	151
14.5	Algoritmi per la PCA	154
14.5.1	PCA per dati ad alta dimensionalità	154
14.6	Probabilistic Principal Component Analysis	155
14.6.1	Maximum Likelihood PCA	156
14.6.2	Stima dei parametri tramite algoritmo EM	157
14.7	Autoencoder	157
14.7.1	Bottleneck e rappresentazione latente	158
14.7.2	Convolutional Autoencoders	159
14.7.3	Autoencoder per Anomaly Detection	159
14.8	Variational Autoencoders	160

14.8.1	Problemi dei VAE e funzione obiettivo	161
14.8.2	Problemi dei VAE e funzione obiettivo	162
14.8.3	Reparameterization Trick	162
14.9	Generative Adversarial Networks	163
14.9.1	Addestramento avversario	164
14.9.2	Spazio latente nei GAN	165
14.9.3	Vantaggi e limiti	165
15	Reinforcement learning	167
15.1	Dynamic Systems e Proprietà di Markov	167
15.1.1	Proprietà di Markov	167
15.2	Markov Decision Processes	167
15.2.1	Modello grafico	168
15.2.2	Definizione formale	168
15.3	MDP	168
15.3.1	MDP non deterministico	169
15.3.2	MDP stocastico	169
15.3.3	Esempio di MDP non deterministico: Grid World	169
15.4	Policy	170
15.4.1	Funzione valore	170
15.4.2	Policy ottima	171
15.5	Equazione di Bellman	171
15.5.1	Equazione di Bellman per una policy	172
15.5.2	Equazione di Bellman per il valore ottimo	172
15.6	One-State Markov Decision Process	172
15.6.1	Policy ottima	172
15.7	Q-function	173
15.8	Q-learning e apprendimento senza modello	173
15.8.1	Regola di aggiornamento	174
15.8.2	Action selection: exploration vs exploitation	174
15.8.3	Esempio: Slot Machine e k-Armed Bandit	175
15.8.4	Problema e concetto di soluzione	176
15.9	Altri algoritmi di reinforcement learning	176

Chapter 1

Introduzione al Machine Learning

1.1 Definizione di Machine Learning

Il *Machine Learning* (ML) può essere definito come il campo di studio che si occupa di programmare calcolatori affinché migliorino la loro performance su un determinato compito attraverso l'esperienza. Formalmente, un sistema di apprendimento migliora la propria prestazione rispetto a un compito T , misurata da una metrica di performance P , sulla base dell'esperienza E .

$$\text{Learning} \equiv \text{Improve}(T, P, E)$$

Il Machine Learning è particolarmente utile quando:

- non esiste una competenza umana esplicita;
- gli esseri umani non sono in grado di formalizzare la propria esperienza;
- la soluzione deve adattarsi a casi specifici o a dati in continua evoluzione.

1.2 Fattori abilitanti del Machine Learning

Lo sviluppo del Machine Learning è stato reso possibile dalla convergenza di tre fattori principali:

- progressi teorici e algoritmici;
- disponibilità massiva di dati (Big Data);
- aumento della potenza computazionale, in particolare grazie alle GPU.

1.3 Applicazioni del Machine Learning

Il Machine Learning trova applicazione in numerosi domini, tra cui:

- computer vision;
- riconoscimento vocale e interazione parlata;
- analisi di testi e documenti;

- data analytics;
- controllo robotico;
- giochi e sistemi decisionali.

1.4 Cos'è un problema di apprendimento

Un problema di apprendimento consiste nel migliorare una prestazione nel tempo grazie all'esperienza. Un classico esempio è l'apprendimento del gioco della dama:

- Compito (T): giocare a dama;
- Performance (P): percentuale di partite vinte;
- Esperienza (E): partite giocate contro sé stessi.

La progettazione di un sistema di apprendimento richiede di rispondere alle seguenti domande:

- quale esperienza utilizzare;
- cosa deve essere appreso;
- come rappresentare la conoscenza;
- quale algoritmo usare per apprendere.

1.5 Funzione ideale

In un problema di apprendimento si assume l'esistenza di una funzione ideale che descrive perfettamente il comportamento desiderato. Tale funzione è concettualmente corretta, ma non è disponibile in forma esplicita e non è computabile nella pratica.

Nel caso generale, si tratta di una funzione:

$$f : X \rightarrow Y$$

che associa a ogni possibile istanza di input $x \in X$ l'output corretto $y \in Y$. L'obiettivo del Machine Learning è apprendere una funzione che approssimi il più possibile questa funzione ideale, utilizzando un insieme finito di esempi.

1.6 Funzione valore

Nel problema dell'apprendimento del gioco della dama, la funzione ideale può essere espressa tramite una funzione valore:

$$V(b)$$

che associa a ogni configurazione della scacchiera b un valore reale che rappresenta la qualità della posizione.

Una possibile definizione concettuale della funzione valore è la seguente:

- se b è uno stato finale vinto, allora $V(b) = 100$;
- se b è uno stato finale perso, allora $V(b) = -100$;
- se b è uno stato finale di pareggio, allora $V(b) = 0$;
- se b non è uno stato finale, allora $V(b)$ è il valore del miglior stato finale raggiungibile giocando in modo ottimale.

Questa funzione è corretta dal punto di vista teorico, ma non è computabile, poiché richiederebbe di esplorare completamente lo spazio degli stati e delle possibili partite.

1.6.1 Rappresentazione approssimata della funzione valore

Poiché la funzione valore ideale $V(b)$ non è computabile, si introduce una funzione approssimata $\hat{V}(b)$, appartenente a una classe di funzioni più semplici e computabili.

Nel caso considerato, $\hat{V}(b)$ è rappresentata come una combinazione lineare di feature estratte dalla configurazione della scacchiera:

$$\hat{V}(b) = w_0 + \sum_{i=1}^n w_i f_i(b)$$

dove:

- $f_i(b)$ sono feature note e computabili;
- w_i sono coefficienti reali ignoti;
- $\hat{V}(b)$ approssima il valore reale della configurazione.

Nel caso specifico della dama, le feature includono il numero di pedine, dame e pezzi minacciati per ciascun giocatore. L'apprendimento si riduce quindi alla stima dei pesi w_i .

1.7 Algoritmo di apprendimento

L'apprendimento dei pesi w_i avviene tramite un algoritmo iterativo basato sulla riduzione dell'errore tra il valore stimato e il valore di training.

Nel caso considerato viene utilizzato l'algoritmo *LMS* (*Least Mean Squares*).

Dato un esempio di training b , l'errore è definito come:

$$\text{error}(b) = V_{\text{train}}(b) - \hat{V}(b)$$

I pesi vengono aggiornati secondo la regola:

$$w_i \leftarrow w_i + c \cdot f_i(b) \cdot \text{error}(b)$$

dove c è il learning rate, che controlla la velocità di apprendimento.

L'algoritmo LMS è un metodo di apprendimento incrementale basato sulla discesa del gradiente stocastica.

1.8 Funzione obiettivo e rappresentazione

Nel Machine Learning si assume l'esistenza di una funzione obiettivo (o funzione target) sconosciuta:

$$f : X \rightarrow Y$$

L'obiettivo dell'apprendimento è calcolare una funzione approssimata:

$$\hat{f}(x) \approx f(x)$$

che generalizzi correttamente anche su esempi non presenti nel dataset di training.

1.9 Dataset e generalizzazione

Dato un dataset di addestramento:

$$D = \{(x_i, y_i)\}_{i=1}^n$$

il sottoinsieme degli input osservati è:

$$X_D \subset X \quad \text{con} \quad |X_D| \ll |X|$$

Il vero obiettivo dell'apprendimento non è la correttezza su D , ma la capacità di generalizzare a nuovi campioni $x \notin X_D$.

1.10 Tipologie di problemi di Machine Learning

In base al tipo di dataset e di output, si distinguono tre principali categorie:

1.10.1 Supervised Learning

Il dataset contiene coppie input-output:

$$D = \{(x_i, y_i)\}_{i=1}^n$$

Include:

- **Classificazione:** $Y = \{C_1, \dots, C_k\}$;
- **Regressione:** $Y \subset \mathbb{R}^k$.

Caso particolare: **Concept Learning**, con output binario $Y = \{0, 1\}$.

1.10.2 Unsupervised Learning

Il dataset contiene solo input:

$$D = \{x_i\}_{i=1}^n$$

L'obiettivo è individuare strutture latenti nei dati, come nel clustering o nella riduzione dimensionale.

1.10.3 Reinforcement Learning

Il sistema apprende una politica:

$$\pi : S \rightarrow A$$

sulla base di sequenze di stati, azioni e ricompense:

$$D = \{(s_0, a_1, r_1, s_1, \dots)\}$$

Non è disponibile un output supervisionato diretto, ma solo ricompense sparse e ritardate.

1.11 Ipotesi e spazio delle ipotesi

Un'ipotesi h è una possibile approssimazione della funzione target ideale. Ogni ipotesi rappresenta una funzione che associa a ciascuna istanza di input un valore di output.

Lo **spazio delle ipotesi** H è l'insieme di tutte le ipotesi che il sistema di apprendimento è in grado di rappresentare, dato un certo formalismo o modello scelto.

L'apprendimento può essere visto come un problema di ricerca nello spazio delle ipotesi, in cui si vuole individuare l'ipotesi che massimizza una misura di performance sul dataset di training:

$$h^* = \arg \max_{h \in H} \text{Performance}(h, D)$$

La scelta dello spazio delle ipotesi è cruciale, poiché determina:

- il potere rappresentativo del modello;
- la capacità di generalizzazione;
- la complessità computazionale del processo di apprendimento.

Uno spazio delle ipotesi troppo limitato può non contenere una buona approssimazione della funzione target, mentre uno spazio troppo ampio può portare a scarsa generalizzazione.

1.12 Consistenza e Version Space

Un'ipotesi h si dice **consistente** con un dataset di training D se fornisce la risposta corretta per tutti gli esempi presenti nel dataset:

$$\forall (x, c(x)) \in D \quad h(x) = c(x)$$

Il **Version Space** è definito come il sottoinsieme dello spazio delle ipotesi contenente tutte le ipotesi consistenti con i dati osservati:

$$VS_{H,D} = \{h \in H \mid h \text{ è consistente con } D\}$$

Il Version Space rappresenta l'insieme di tutte le spiegazioni compatibili con i dati disponibili. L'obiettivo di un sistema di apprendimento non è semplicemente trovare una qualunque ipotesi consistente, ma selezionare un'ipotesi che generalizzi correttamente anche su istanze non presenti nel dataset.

Schema concettuale



1.13 Algoritmo List-Then-Eliminate

Un modo concettualmente semplice per calcolare il Version Space è l'algoritmo *List-Then-Eliminate*. Questo algoritmo assume che lo spazio delle ipotesi H sia finito ed enumerabile.

L'algoritmo opera come segue:

1. inizializza il Version Space contenendo tutte le ipotesi in H ;
2. per ogni esempio di training $(x, c(x))$, elimina dal Version Space tutte le ipotesi h tali che $h(x) \neq c(x)$;
3. restituisce l'insieme delle ipotesi rimanenti nel Version Space.

Formalmente:

$$VS_{H,D} = \{h \in H \mid \forall (x, c(x)) \in D, h(x) = c(x)\}$$

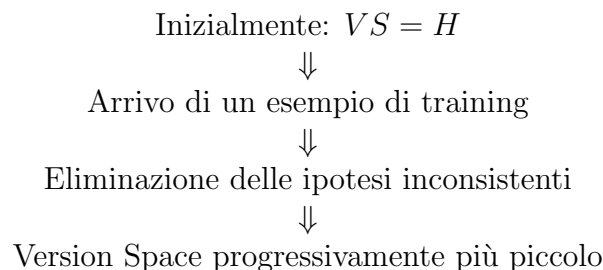
1.13.1 Osservazioni

L'algoritmo List-Then-Eliminate è corretto dal punto di vista teorico, ma non è utilizzabile nella pratica, poiché richiede l'enumerazione esplicita di tutte le ipotesi in H .

Il suo ruolo principale è quello di chiarire:

- il concetto di consistenza;
- il significato operativo di Version Space;
- il legame tra dati di training e riduzione dello spazio delle ipotesi.

Schema concettuale



1.14 Rappresentazione vs Generalizzazione

Esiste un compromesso fondamentale tra:

- **potere rappresentativo** dello spazio delle ipotesi;
- **potere di generalizzazione**.

Se lo spazio delle ipotesi è troppo potente (può rappresentare ogni sottoinsieme di X), il sistema perde capacità di generalizzare.

1.15 Dati rumorosi

Nei problemi reali, il dataset può contenere rumore:

$$y_i \neq c(x_i)$$

In questi casi:

- il Version Space può essere vuoto;
- non esistono ipotesi perfettamente consistenti;
- è necessario adottare metodi statistici robusti.

RIEPILOGO

Machine Learning è un approccio *data-driven* che si basa sull'uso dei dati per costruire modelli quando una descrizione completa dell'ambiente non è disponibile. L'apprendimento è definito come il miglioramento delle prestazioni su un compito, sulla base dell'esperienza e misurato tramite una metrica di performance.

Nel Machine Learning si assume l'esistenza di una funzione ideale (target) sconosciuta, che viene approssimata mediante una funzione computabile. Nel caso dell'apprendimento del gioco della dama, tale funzione è una funzione valore $V(b)$, che assegna a ogni configurazione un valore numerico. Poiché $V(b)$ non è computabile, si introduce una funzione approssimata $\hat{V}(b)$, rappresentata come combinazione lineare di feature, i cui parametri vengono appresi dai dati.

L'apprendimento dei parametri avviene tramite un algoritmo iterativo, come l'algoritmo LMS, che aggiorna i pesi riducendo l'errore tra il valore stimato e il valore di training.

Tipologie di problemi di Machine Learning:

- **Supervised Learning:** il dataset contiene coppie input-output; include problemi di regressione e classificazione;
- **Unsupervised Learning:** il dataset contiene solo input, senza etichette; l'obiettivo è scoprire strutture latenti nei dati;
- **Reinforcement Learning:** l'agente apprende una politica sulla base di ricompense ritardate e interazioni con l'ambiente.

Notazione ricorrente:

- X : spazio delle istanze;
- $x \in X$: una singola istanza;
- $D = \{(x_i, c(x_i))\}_{i=1}^n$: dataset di training;
- c : funzione target ideale;
- H : spazio delle ipotesi;
- $h \in H$: ipotesi;
- $h(x)$: valore stimato dall'ipotesi su x .

Un'ipotesi è detta **consistente** con il dataset se fornisce la risposta corretta per tutti gli esempi di training. Il **Version Space** è l'insieme di tutte le ipotesi consistenti con i dati osservati e rappresenta lo spazio delle soluzioni compatibili con il dataset. La scelta dello spazio delle ipotesi è cruciale: uno spazio troppo semplice limita il potere rappresentativo, mentre uno spazio troppo complesso riduce la capacità di generalizzazione. Trovare il giusto equilibrio è fondamentale per evitare fenomeni di *underfitting* e *overfitting*.

Chapter 2

Performance Evaluation

2.1 Introduzione alla valutazione delle prestazioni

La valutazione delle prestazioni è il processo mediante il quale si stima la qualità di una ipotesi o di un algoritmo di apprendimento nel generalizzare a nuovi dati non osservati.

Dato che un modello di Machine Learning viene addestrato su un insieme finito di esempi, non è possibile osservare direttamente il suo comportamento sull'intera distribuzione dei dati. La valutazione delle prestazioni fornisce quindi una **stima statistica** dell'accuratezza del modello su istanze future.

In termini generali, la valutazione delle prestazioni risponde a domande del tipo:

- quanto è accurata l'ipotesi appresa?
- quanto è affidabile questa stima?
- come si confrontano due ipotesi o due algoritmi diversi?

È importante sottolineare che la valutazione delle prestazioni non restituisce un valore assoluto, ma una stima soggetta a incertezza, che dipende:

- dalla quantità di dati disponibili;
- dalla modalità di campionamento;
- dalla metrica di valutazione utilizzata.

La difficoltà principale nella valutazione delle prestazioni risiede nel fatto che l'errore vero di un modello non è osservabile direttamente. Per questo motivo si introducono stime basate su campioni finiti di dati, come l'errore campionario.

2.2 Errore vero ed errore campionario

Sia $f : X \rightarrow Y$ la funzione target ideale e sia D una distribuzione di probabilità sugli input. Dato un modello appreso h , l'obiettivo della valutazione delle prestazioni è stimare quanto spesso h fornisca una predizione errata su esempi futuri.

2.2.1 Errore vero

L'**errore vero** di un'ipotesi h è definito come la probabilità che h commetta un errore su un'istanza estratta casualmente secondo la distribuzione D :

$$error_D(h) = \Pr_{x \sim D}[f(x) \neq h(x)]$$

L'errore vero rappresenta la reale capacità di generalizzazione del modello. Tuttavia, non può essere calcolato direttamente, poiché la distribuzione D e la funzione target f sono sconosciute.

2.2.2 Errore campionario

Poiché l'errore vero non è osservabile, si introduce l'**errore campionario**, calcolato su un insieme finito di esempi S :

$$error_S(h) = \frac{1}{|S|} \sum_{x \in S} \delta(f(x) \neq h(x))$$

L'errore campionario è una quantità misurabile e rappresenta una stima dell'errore vero. Tuttavia, essendo calcolato su un campione finito, dipende dagli esempi contenuti in S .

Esempio

Si consideri un problema di classificazione binaria e un modello h . Supponiamo di testare il modello su un insieme di 20 esempi e osservare 2 errori. In questo caso:

$$error_S(h) = \frac{2}{20} = 0.1$$

Questo valore rappresenta l'errore campionario. Tuttavia, se il test fosse stato effettuato su un diverso campione di 20 esempi estratti dalla stessa distribuzione, il numero di errori osservati avrebbe potuto essere diverso.

L'errore vero $error_D(h)$ rappresenta invece la frazione di errori che il modello commetterebbe su un numero infinito di esempi estratti dalla distribuzione D . L'errore campionario è quindi una realizzazione casuale di una variabile aleatoria che approssima l'errore vero.

Interpretazione statistica

L'errore campionario $error_S(h)$ è una variabile casuale, poiché dipende dal campione S estratto dalla distribuzione D . Campioni diversi producono stime diverse dello stesso errore vero.

Il problema centrale della valutazione delle prestazioni consiste nel determinare:

- quanto bene $error_S(h)$ approssimi $error_D(h)$;
- quanto è affidabile questa approssimazione.

2.3 Intervalli di confidenza

Se il campione di test contiene almeno 30 esempi ed è indipendente dall'ipotesi, con probabilità approssimativamente pari a $N\%$, l'errore vero soddisfa:

$$error_D(h) \in error_S(h) \pm z_N \sqrt{\frac{error_S(h)(1 - error_S(h))}{n}}$$

dove z_N è un coefficiente dipendente dal livello di confidenza desiderato.

2.4 Stima non distorta e trade-off training/testing

Per ottenere una stima non distorta, i dati utilizzati per la valutazione devono essere indipendenti da quelli usati per l'addestramento. Una suddivisione tipica è:

- $\frac{2}{3}$ dei dati per il training;
- $\frac{1}{3}$ dei dati per il testing.

Esiste un trade-off: più dati di training migliorano il modello, ma peggiorano la stima; più dati di test migliorano la stima, ma peggiorano il modello.

2.5 k-Fold Cross Validation

La suddivisione dei dati in un singolo training set e un singolo test set fornisce una stima dell'errore vero, ma tale stima può avere elevata varianza, poiché dipende fortemente dalla particolare partizione dei dati.

La **k-Fold Cross Validation** è una tecnica che consente di ridurre questa variabilità, utilizzando in modo più efficiente l'intero dataset disponibile.

2.5.1 Descrizione dell'algoritmo

Dato un dataset D , esso viene suddiviso in k sottoinsiemi disgiunti:

$$D = S_1 \cup S_2 \cup \dots \cup S_k$$

Per ogni iterazione $i = 1, \dots, k$:

- il sottoinsieme S_i viene utilizzato come test set;
- i rimanenti $k - 1$ sottoinsiemi costituiscono il training set $T_i = D \setminus S_i$;
- viene addestrato un modello $h_i = L(T_i)$;
- viene calcolato l'errore $error_{S_i}(h_i)$.

L'errore finale dell'algoritmo di apprendimento L è stimato come:

$$error_{L,D} = \frac{1}{k} \sum_{i=1}^k error_{S_i}(h_i)$$

Interpretazione

La k-Fold Cross Validation fornisce una stima dell'**valore atteso** dell'errore campionario di un algoritmo di apprendimento.

Ogni esempio del dataset viene utilizzato:

- una sola volta come dato di test;
- $k - 1$ volte come dato di training.

In questo modo, la stima dell'errore:

- è meno dipendente da una singola partizione dei dati;
- presenta una varianza inferiore rispetto a un singolo split.

Esempio

Si consideri un dataset composto da 100 esempi e una 5-Fold Cross Validation. Il dataset viene suddiviso in 5 sottoinsiemi da 20 esempi ciascuno.

Ad ogni iterazione:

- 80 esempi vengono utilizzati per il training;
- 20 esempi vengono utilizzati per il testing.

Al termine delle 5 iterazioni, ciascun esempio è stato utilizzato esattamente una volta come test set. L'errore finale è la media degli errori osservati nelle singole iterazioni.

Schema concettuale



2.6 Overfitting

Nel Machine Learning l'obiettivo non è minimizzare l'errore sui dati di training, ma ottenere buone prestazioni su dati futuri non osservati. Il fenomeno dell'**overfitting** si verifica quando un modello si adatta eccessivamente ai dati di training, perdendo capacità di generalizzazione.

Formalmente, data una ipotesi $h \in H$, si dice che h presenta overfitting rispetto a un dataset S se esiste un'altra ipotesi $h' \in H$ tale che:

$$error_S(h) < error_S(h') \quad \text{e} \quad error_D(h) > error_D(h')$$

In altre parole, h è migliore sui dati di training, ma peggiore sui dati futuri rispetto a h' .

L'overfitting è una conseguenza di uno spazio delle ipotesi troppo complesso rispetto alla quantità di dati disponibili. Un modello con elevato potere rappresentativo può adattarsi anche al rumore presente nel dataset, trattandolo come se fosse informazione rilevante.

Esempio

Si consideri un problema di classificazione e due modelli h e h' . Il modello h è molto complesso, mentre h' è più semplice.

Supponiamo di osservare i seguenti risultati:

$$\begin{aligned} error_S(h) &= 0.02 & error_S(h') &= 0.08 \\ error_D(h) &= 0.25 & error_D(h') &= 0.10 \end{aligned}$$

Il modello h commette pochissimi errori sui dati di training, ma generalizza molto male. Al contrario, il modello h' ha un errore di training più alto, ma un errore vero più basso.

In questo caso, il modello h presenta overfitting, mentre h' rappresenta una soluzione preferibile.

L'overfitting non può essere individuato osservando esclusivamente l'errore di training. È necessario stimare l'errore vero mediante tecniche di valutazione come il test set indipendente o la k-Fold Cross Validation.

2.7 Metriche di valutazione

2.7.1 Limiti dell'accuratezza

Nel caso di problemi di classificazione, una metrica comunemente utilizzata è l'accuratezza:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Tuttavia, l'accuratezza può risultare fuorviante in presenza di dataset sbilanciati, ovvero quando una classe è molto più frequente delle altre. In questi casi, un classificatore banale che predice sempre la classe più frequente può ottenere un'alta accuratezza pur non essendo utile.

2.7.2 Precision, Recall e F1-score

Per valutare meglio le prestazioni in presenza di dataset sbilanciati, si introducono metriche alternative basate sulla matrice di confusione.

La **Recall** (o True Positive Rate) misura la capacità del modello di individuare correttamente i casi positivi:

$$Recall = \frac{TP}{TP + FN}$$

La **Precision** misura la frazione di predizioni positive corrette:

$$Precision = \frac{TP}{TP + FP}$$

La **F1-score** combina precision e recall in un'unica metrica:

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$$

L'importanza relativa di precision e recall dipende dal dominio applicativo.

2.7.3 Metriche derivate e curva ROC

Altre metriche utili sono:

- **True Negative Rate (Specificity):**

$$TNR = \frac{TN}{TN + FP}$$

- **False Positive Rate:**

$$FPR = \frac{FP}{TN + FP}$$

- **False Negative Rate:**

$$FNR = \frac{FN}{TP + FN}$$

La **ROC curve** rappresenta graficamente il compromesso tra True Positive Rate e False Positive Rate al variare della soglia di classificazione. L'**AUC** (Area Under the Curve) fornisce una misura sintetica delle prestazioni complessive del classificatore.

2.7.4 Dipendenza dal dominio applicativo

L'impatto degli errori dipende fortemente dall'applicazione. Ad esempio:

- in ambito medico, un falso negativo può avere conseguenze gravi;
- nella guida autonoma, un falso positivo può causare frenate inutili;
- in sistemi di sicurezza, un falso negativo può compromettere l'intero sistema.

La scelta della metrica di valutazione deve quindi essere guidata dal contesto applicativo e non basarsi esclusivamente sull'accuratezza.

2.7.5 Matrice di confusione multi-classe

La *confusion matrix* per un problema di classificazione multi-classe con K classi è una matrice quadrata

$$\mathbf{C} \in \mathbb{N}^{K \times K},$$

in cui le righe rappresentano le classi reali e le colonne le classi predette dal classificatore. L'elemento $C_{i,j}$ indica il numero di istanze appartenenti realmente alla classe C_i che vengono classificate come appartenenti alla classe C_j .

La diagonale principale contiene le classificazioni corrette per ciascuna classe, mentre gli elementi fuori diagonale indicano le classi che vengono più frequentemente confuse. L'analisi della matrice di confusione consente di individuare debolezze specifiche del classificatore.

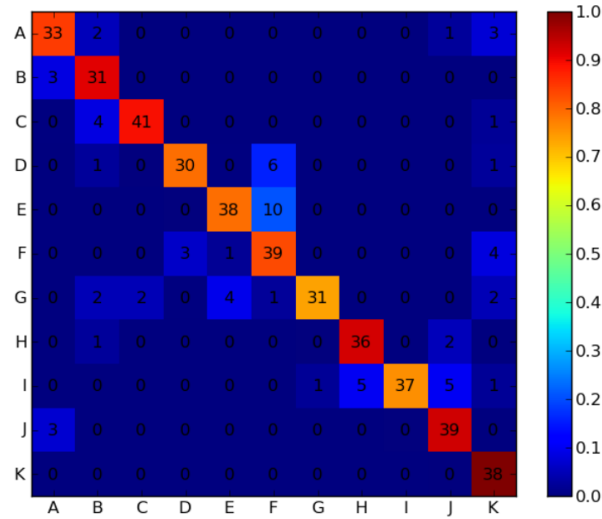


Figure 2.1: Esempio di matrice di confusione per un problema di classificazione. Gli elementi sulla diagonale rappresentano le predizioni corrette.

2.8 Metriche di valutazione per la regressione

Nel caso di problemi di regressione, la valutazione delle prestazioni si basa sulla differenza numerica tra il valore predetto $\hat{f}(x_i)$ e il valore reale t_i .

Le metriche più comuni sono:

- **Mean Absolute Error (MAE)**: misura l'errore medio assoluto tra i valori predetti e i valori reali

$$MAE = \frac{1}{n} \sum_{i=1}^n \left| \hat{f}(x_i) - t_i \right|$$

Il MAE assegna lo stesso peso a tutti gli errori, indipendentemente dalla loro ampiezza. Ogni errore contribuisce in modo lineare alla metrica.

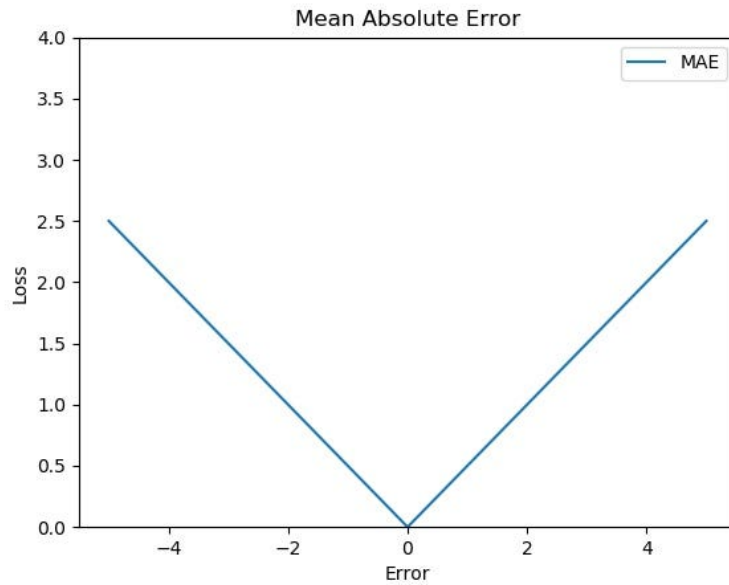


Figure 2.2: Rappresentazione grafica del MAE.

- **Mean Squared Error (MSE):** è definito come

$$MSE = \frac{1}{n} \sum_{i=1}^n \left(\hat{f}(x_i) - t_i \right)^2$$

In questa metrica, gli errori vengono elevati al quadrato, penalizzando maggiormente gli errori di grande entità.

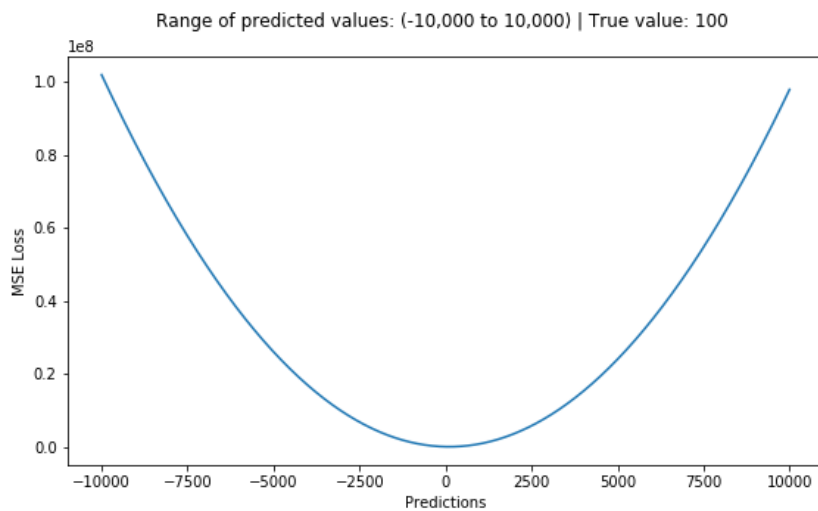


Figure 2.3: Rappresentazione grafica del MSE.

- **Root Mean Squared Error (RMSE):** è la radice quadrata del MSE:

$$RMSE = \sqrt{MSE}$$

Il RMSE mantiene la sensibilità agli errori grandi tipica del MSE, ma riporta l'errore nella stessa unità di misura del target.

Confronto tra le metriche

Le tre metriche differiscono per il modo in cui penalizzano gli errori:

- il MAE cresce linearmente con l'errore;
- il MSE cresce quadraticamente;
- il RMSE eredita la penalizzazione del MSE, ma è più interpretabile.

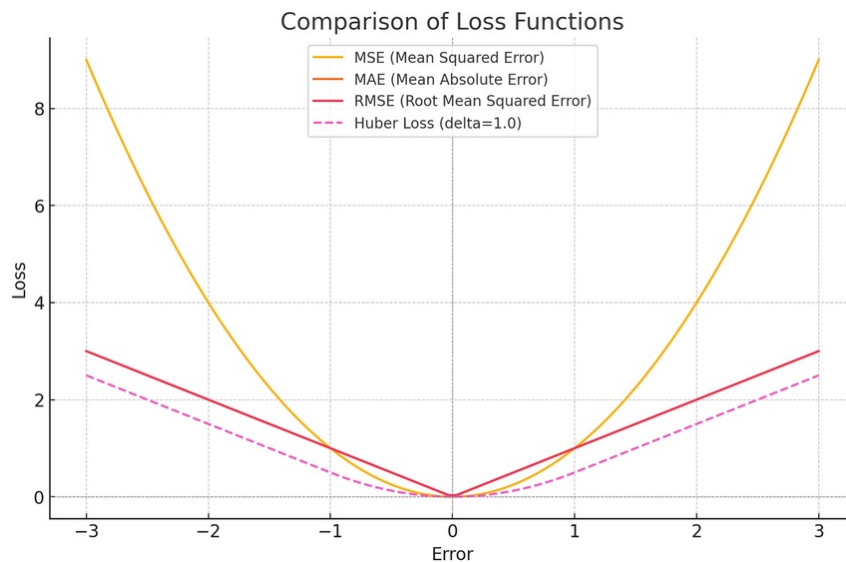


Figure 2.4: Rappresentazione del confronto tra le metriche.

2.8.1 Metriche percentuali e coefficiente di determinazione

In alcuni contesti è utile valutare l'errore in termini percentuali, ad esempio tramite il Mean Absolute Percentage Error (MAPE).

Un'ulteriore metrica importante è il coefficiente di determinazione:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - t_i)^2}{\sum_{i=1}^n (t_i - \mu)^2} \quad \text{con} \quad \mu = \frac{1}{n} \sum_{i=1}^n t_i$$

Il valore $R^2 = 0$ corrisponde a un modello che predice sempre la media dei dati.

2.8.2 Cross-validation per la regressione

La k-Fold Cross Validation può essere estesa ai problemi di regressione utilizzando metriche appropriate, come il MAE o l'RMSE. L'errore finale è ottenuto come media degli errori calcolati sui singoli fold.

Riepilogo

Valutazione delle prestazioni Stima della capacità di generalizzazione di un'ipotesi o di un algoritmo su dati non osservati.

Errore vero ed errore campionario

$$\begin{aligned} error_D(h) &= \Pr_{x \sim D}[f(x) \neq h(x)] \\ error_S(h) &= \frac{1}{|S|} \sum_{x \in S} \delta(f(x) \neq h(x)) \end{aligned}$$

Train / Test split I dati di valutazione devono essere indipendenti da quelli di training.

k-Fold Cross Validation

$$error_{L,D} = \frac{1}{k} \sum_{i=1}^k error_S(h_i)$$

Ogni esempio è usato una volta per il test e $k - 1$ volte per il training.

Overfitting

$$error_S(h) < error_S(h') \wedge error_D(h) > error_D(h')$$

Errore di training basso, errore vero alto.

Metriche per la classificazione

$$\begin{aligned} Accuracy &= \frac{TP + TN}{TP + TN + FP + FN} \\ Precision &= \frac{TP}{TP + FP} \quad Recall = \frac{TP}{TP + FN} \\ F1 &= \frac{2 \cdot Precision \cdot Recall}{Precision + Recall} \end{aligned}$$

ROC: TPR vs FPR — AUC misura la capacità discriminante.

Metriche per la regressione

$$\begin{aligned} MAE &= \frac{1}{n} \sum_{i=1}^n \left| \hat{f}(x_i) - t_i \right| \\ MSE &= \frac{1}{n} \sum_{i=1}^n \left(\hat{f}(x_i) - t_i \right)^2 \\ RMSE &= \sqrt{MSE} \\ R^2 &= 1 - \frac{\sum_{i=1}^n (\hat{f}(x_i) - t_i)^2}{\sum_{i=1}^n (t_i - \mu)^2} \end{aligned}$$

Chapter 3

Decision Tree

3.1 Definizione

Un *Decision Tree* è un modello di apprendimento supervisionato che rappresenta una funzione di decisione sotto forma di struttura ad albero. Il processo decisionale avviene mediante una sequenza di test su attributi che, partendo dalla radice, conducono a una foglia contenente la predizione finale.

Formalmente, un decision tree implementa una funzione:

$$h : X \rightarrow Y$$

dove X rappresenta lo spazio delle istanze descritte da un insieme di attributi e Y lo spazio delle etichette di output. Ogni cammino dalla radice a una foglia può essere interpretato come una regola di decisione del tipo:

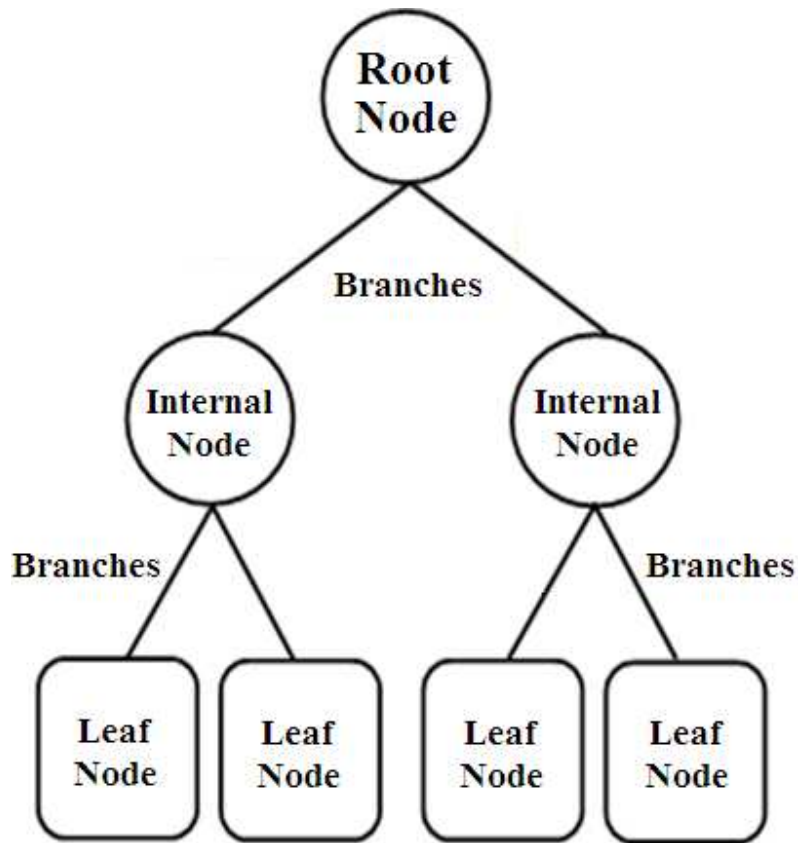
$$\text{IF } (\text{condizione}_1 \wedge \text{condizione}_2 \wedge \dots) \Rightarrow y$$

3.1.1 Struttura del Decision Tree

Un decision tree è composto dai seguenti elementi fondamentali:

- **Nodo radice**, che rappresenta il primo test su un attributo;
- **Nodi interni**, ciascuno dei quali corrisponde a un test su un attributo;
- **Archi**, che rappresentano i possibili esiti del test;
- **Foglie**, che contengono la decisione finale del modello.

Nel caso di problemi di classificazione, ogni foglia è associata a una classe, mentre nei problemi di regressione la foglia contiene un valore reale.



(a)

Figure 3.1: Struttura generale di un decision tree: nodo radice, nodi interni e foglie.

3.1.2 Caratteristiche principali

I decision tree presentano diverse proprietà che li rendono modelli particolarmente utilizzati:

- **Interpretabilità:** il modello è facilmente interpretabile, poiché il percorso seguito per effettuare una predizione è esplicito;
- **Assenza di assunzioni forti:** non richiedono ipotesi sulla distribuzione dei dati o relazioni lineari tra gli attributi;
- **Flessibilità:** possono gestire sia attributi discreti che continui;
- **Apprendimento non parametrico:** la complessità del modello cresce con i dati;
- **Rischio di overfitting:** alberi troppo profondi possono adattarsi eccessivamente ai dati di training, riducendo la capacità di generalizzazione.

Esempio

Si consideri un problema di classificazione in cui l'obiettivo è decidere se giocare a tennis in base alle condizioni meteorologiche. Gli attributi considerati sono:

- $Outlook \in \{Sunny, Overcast, Rain\}$;
- $Humidity \in \{High, Normal\}$;
- $Wind \in \{Weak, Strong\}$.

La classe di output è:

$$PlayTennis \in \{Yes, No\}$$

Un possibile decision tree è il seguente:

- se $Outlook = Overcast$, allora $PlayTennis = Yes$;
- se $Outlook = Sunny$:
 - se $Humidity = High$, allora $PlayTennis = No$;
 - se $Humidity = Normal$, allora $PlayTennis = Yes$;
- se $Outlook = Rain$:
 - se $Wind = Strong$, allora $PlayTennis = No$;
 - se $Wind = Weak$, allora $PlayTennis = Yes$.

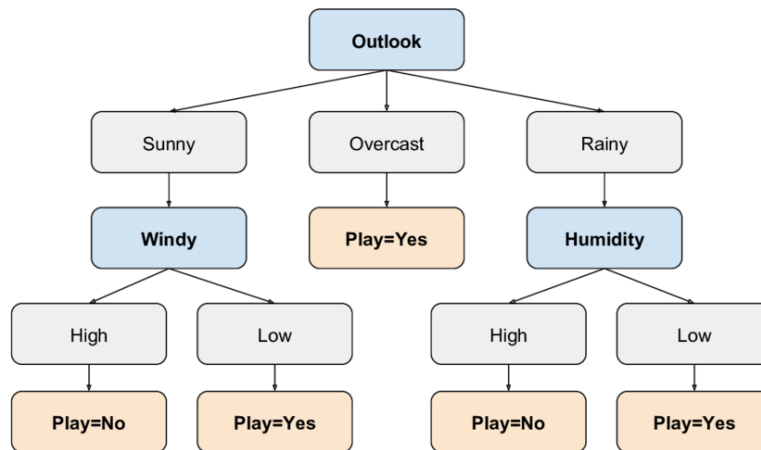


Figure 3.2: Esempio di decision tree per il problema Play Tennis.

3.2 Regole di decisione

Nel contesto dei decision tree, una **regola di decisione** è una proposizione logica che associa una combinazione di condizioni sugli attributi a una decisione finale.

Formalmente, una regola ha la forma:

$$\text{IF } (condizione_1 \wedge condizione_2 \wedge \dots \wedge condizione_k) \Rightarrow y$$

dove ciascuna condizione rappresenta un test su un attributo e y è il valore di output associato.

3.2.1 Decision Tree come insieme di regole

Un decision tree può essere interpretato come un insieme di regole di decisione. Ogni cammino che parte dal nodo radice e termina in una foglia definisce una regola distinta.

Pertanto, un decision tree rappresenta una funzione di decisione equivalente a una congiunzione di regole del tipo:

$$h(x) = \bigvee_{i=1}^m r_i(x)$$

dove ciascuna regola r_i è associata a una foglia dell'albero.

3.2.2 Conversione di un decision tree in regole

La conversione di un decision tree in un insieme di regole avviene secondo i seguenti passaggi:

1. si seleziona una foglia dell'albero;
2. si considera il cammino dalla radice a tale foglia;
3. per ogni nodo attraversato, si aggiunge alla regola la condizione associata all'arco seguito;
4. la classe associata alla foglia costituisce la conclusione della regola.

Ripetendo la procedura per tutte le foglie si ottiene un insieme di regole logicamente equivalenti al decision tree originale.

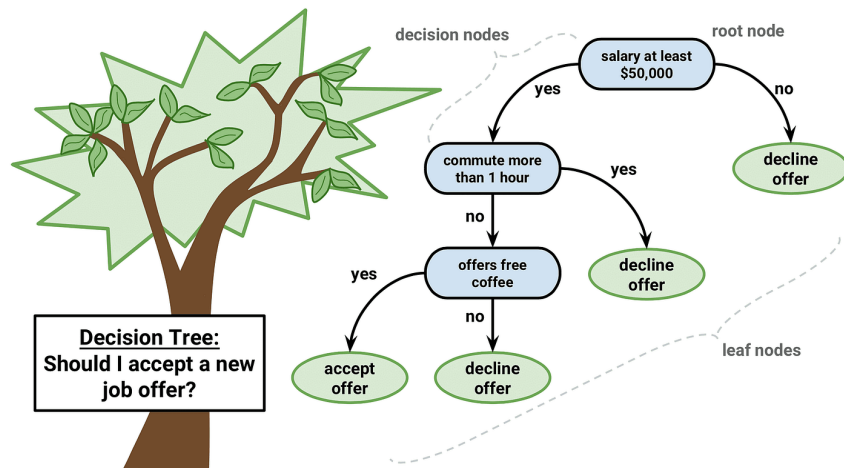


Figure 3.3: Conversione di un decision tree in un insieme di regole: ogni percorso radice–foglia corrisponde a una regola.

Esempio

Si consideri il decision tree del problema *Play Tennis*. Uno dei possibili cammini radice–foglia è il seguente:

$$Outlook = Sunny \wedge Humidity = High$$

La regola di decisione corrispondente è:

$$\text{IF } (Outlook = Sunny \wedge Humidity = High) \Rightarrow PlayTennis = No$$

Ripetendo il procedimento per ciascuna foglia dell'albero si ottiene l'insieme completo delle regole che descrivono il modello.

Osservazione

Un decision tree e l'insieme di regole da esso derivato sono logicamente equivalenti. Tuttavia, la rappresentazione a regole consente una maggiore flessibilità nella semplificazione del modello, ad esempio tramite l'eliminazione di condizioni non necessarie.

3.3 Algoritmo ID3

L'algoritmo *ID3* (Iterative Dichotomiser 3) è uno dei primi algoritmi proposti per l'apprendimento di decision tree a partire da dati etichettati. ID3 costruisce l'albero in modo incrementale, adottando una strategia **greedy top-down**, in cui ad ogni passo viene selezionato l'attributo che massimizza un criterio di bontà dello split.

L'idea alla base dell'algoritmo ID3 è quella di costruire l'albero selezionando, a ogni nodo, l'attributo che consente di separare al meglio gli esempi di training in base alle classi.

La costruzione dell'albero avviene:

- partendo da tutti gli esempi nel nodo radice;
- scegliendo l'attributo più informativo;
- suddividendo il dataset in sottoinsiemi;
- ripetendo ricorsivamente il procedimento su ciascun sottoinsieme.

3.3.1 Costruzione del decision tree

Sia S un insieme di esempi di training e sia A l'insieme degli attributi disponibili. L'algoritmo ID3 costruisce il decision tree secondo i seguenti passi:

1. Se tutti gli esempi in S appartengono alla stessa classe, crea una foglia etichettata con tale classe.
2. Se l'insieme degli attributi A è vuoto, crea una foglia etichettata con la classe più frequente in S .
3. Altrimenti, seleziona l'attributo $a \in A$ che massimizza il criterio di *Information Gain*.
4. Crea un nodo interno etichettato con l'attributo selezionato.
5. Per ciascun valore possibile v dell'attributo a :
 - crea un ramo corrispondente al test $a = v$;
 - considera il sottoinsieme $S_v \subseteq S$ degli esempi per cui $a = v$;
 - applica ricorsivamente l'algoritmo ID3 a S_v e all'insieme $A \setminus \{a\}$.

Condizioni di arresto

La crescita dell'albero termina quando si verifica almeno una delle seguenti condizioni:

- tutti gli esempi nel nodo appartengono alla stessa classe;
- non sono disponibili ulteriori attributi per effettuare lo split;
- il sottoinsieme di esempi associato al nodo è vuoto.

Pseudocodice

```
ID3(S, A):  
    if tutti gli esempi in S hanno la stessa classe:  
        return foglia con quella classe  
    if A è vuoto:  
        return foglia con la classe più frequente in S  
    seleziona a in A che massimizza Information Gain  
    crea un nodo con test sull'attributo a  
    for ogni valore v di a:  
        S_v = sottoinsieme di S con a = v  
        figlio = ID3(S_v, A \ {a})  
        collega figlio al nodo tramite ramo a = v  
    return nodo
```

3.3.2 Proprietà dell'algoritmo ID3

L'algoritmo ID3 presenta le seguenti caratteristiche:

- adotta una strategia greedy e non garantisce l'albero ottimo globale;
- utilizza solo attributi discreti;
- non prevede tecniche di pruning;
- è sensibile al rumore nei dati;
- tende a costruire alberi completi che possono soffrire di overfitting.

3.3.3 ID3 e regole di decisione

L'albero prodotto dall'algoritmo ID3 può essere convertito in un insieme di regole di decisione. Ogni foglia dell'albero corrisponde a una regola ottenuta congiungendo le condizioni lungo il cammino radice-foglia.

3.4 Entropia e Information Gain

Nell'algoritmo ID3 è necessario stabilire quale attributo utilizzare per suddividere gli esempi in ciascun nodo dell'albero. A tal fine viene introdotta una misura quantitativa della **purezza** di un insieme di esempi, basata su concetti di teoria dell'informazione.

3.4.1 Entropia

Sia S un insieme di esempi di training appartenenti a c classi distinte. Si indichi con p_i la proporzione di esempi di S appartenenti alla classe i .

L'**entropia** di S è definita come:

$$Entropy(S) = - \sum_{i=1}^c p_i \log_2 p_i$$

L'entropia misura il grado di incertezza o impurità dell'insieme di esempi ed assume i seguenti valori caratteristici:

- $Entropy(S) = 0$ se tutti gli esempi appartengono alla stessa classe;
- $Entropy(S)$ è massima quando le classi sono equamente distribuite;
- valori intermedi indicano un grado parziale di impurità.

Pertanto, un insieme di esempi è tanto più informativo quanto minore è la sua entropia.

Esempio

Si consideri un insieme S composto da 10 esempi, di cui 5 positivi e 5 negativi. In questo caso:

$$p_+ = 0.5 \quad p_- = 0.5$$

L'entropia risulta:

$$Entropy(S) = -0.5 \log_2 0.5 - 0.5 \log_2 0.5 = 1$$

Se invece tutti gli esempi appartenessero alla stessa classe, l'entropia sarebbe nulla.

3.4.2 Information Gain

L'**Information Gain** misura la riduzione di entropia ottenuta suddividendo un insieme di esempi in base a un attributo.

Dato un insieme di esempi S e un attributo A con valori possibili $\{v_1, v_2, \dots, v_k\}$, l'Information Gain di A rispetto a S è definito come:

$$Gain(S, A) = Entropy(S) - \sum_{j=1}^k \frac{|S_{v_j}|}{|S|} Entropy(S_{v_j})$$

dove $S_{v_j} \subseteq S$ è il sottoinsieme degli esempi per cui l'attributo A assume valore v_j .

L'Information Gain rappresenta la quantità di informazione ottenuta conoscendo il valore di un attributo. Un attributo è tanto più informativo quanto maggiore è la riduzione di entropia che produce.

Pertanto, l'attributo che massimizza l'Information Gain è quello che separa meglio gli esempi in sottoinsiemi più puri.

Utilizzo in ID3

Nell'algoritmo ID3, la selezione dell'attributo da utilizzare in un nodo avviene scegliendo quello che massimizza l'Information Gain.

Formalmente, dato l'insieme degli attributi disponibili A , viene selezionato:

$$a^* = \arg \max_{a \in A} \text{Gain}(S, a)$$

L'attributo selezionato viene utilizzato per suddividere il dataset e il procedimento viene applicato ricorsivamente ai sottoinsiemi generati.

3.4.3 Schema operativo

La costruzione dell'albero tramite ID3 può essere riassunta come segue:

1. calcolare l'entropia dell'insieme di esempi corrente;
2. calcolare l'Information Gain per ciascun attributo disponibile;
3. selezionare l'attributo con massimo Information Gain;
4. suddividere il dataset in base ai valori dell'attributo selezionato;
5. ripetere ricorsivamente il procedimento sui sottoinsiemi ottenuti.

3.5 Overfitting nei Decision Tree

I decision tree sono modelli altamente flessibili e possono adattarsi in modo molto accurato ai dati di training. Tuttavia, questa flessibilità comporta un elevato rischio di **overfitting**.

Un decision tree presenta overfitting quando continua a crescere fino a classificare correttamente tutti (o quasi) gli esempi di training, includendo anche il rumore presente nei dati. In tali casi, l'albero risulta molto accurato sui dati di training ma generalizza male su dati non osservati.

Definizione formale

Dato un insieme di esempi di training S , un decision tree h presenta overfitting se esiste un altro albero h' tale che:

$$\text{error}_S(h) < \text{error}_S(h') \wedge \text{error}_D(h) > \text{error}_D(h')$$

In altre parole, h è migliore sui dati di training, ma peggiore sui dati futuri rispetto a h' .

Cause principali

Le principali cause dell'overfitting nei decision tree sono:

- crescita eccessiva dell'albero;
- presenza di rumore nei dati di training;

- utilizzo di attributi poco informativi;
- assenza di vincoli sulla profondità dell'albero.

Poiché ID3 costruisce l'albero fino a quando è possibile separare ulteriormente gli esempi, esso tende a generare alberi completi e complessi.

3.6 Pruning

Il **pruning** è l'insieme di tecniche volte a ridurre la complessità del decision tree al fine di migliorare la capacità di generalizzazione del modello.

L'obiettivo del pruning non è minimizzare l'errore di training, bensì ridurre l'errore vero eliminando rami che modellano il rumore dei dati.

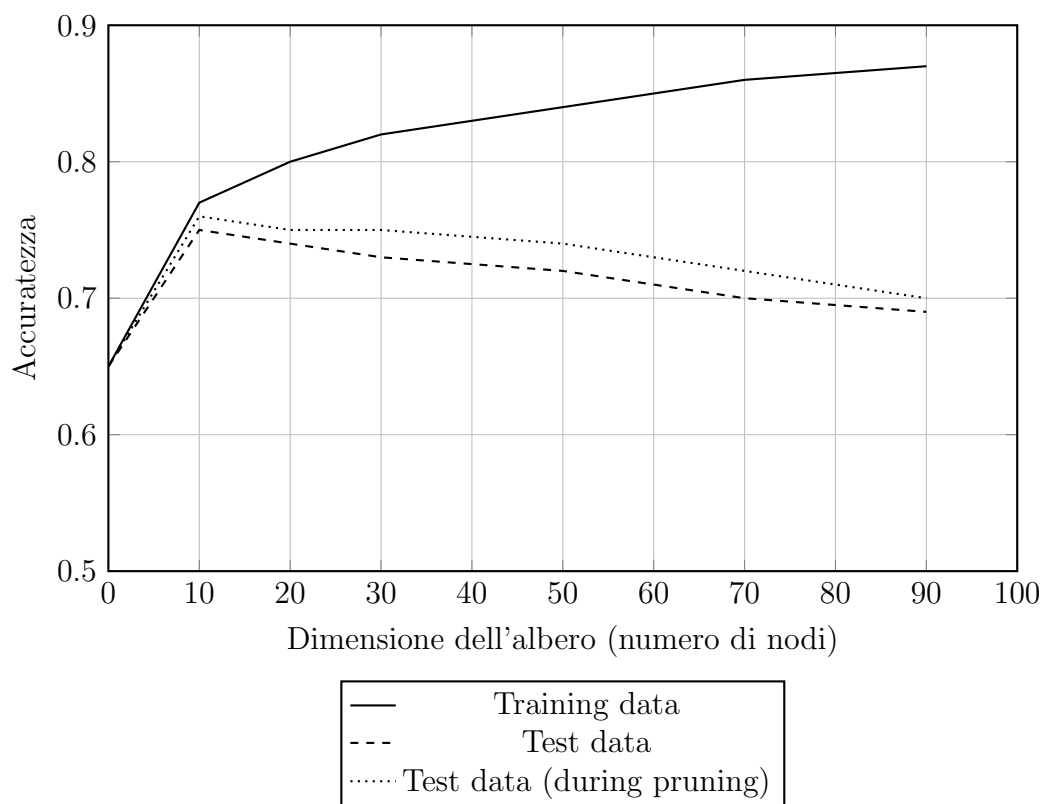


Figure 3.4: Effetto del Reduced-Error Pruning: l'accuratezza sui dati di training cresce con la complessità dell'albero, mentre l'accuratezza sui dati di test mostra un massimo per una complessità intermedia.

3.6.1 Pre-pruning

Il **pre-pruning** consiste nell'interrompere la crescita dell'albero prima che esso raggiunga una struttura completa. La decisione di arrestare la suddivisione viene presa durante la fase di costruzione dell'albero.

Criteri comuni di pre-pruning includono:

- profondità massima dell'albero;
- numero minimo di esempi in un nodo;

- Information Gain inferiore a una soglia;
- assenza di miglioramento significativo delle prestazioni.

Il pre-pruning riduce il rischio di overfitting e il costo computazionale, ma può introdurre underfitting se applicato in modo eccessivo.

3.6.2 Post-pruning

Il **post-pruning** consiste nel costruire inizialmente un decision tree completo e successivamente semplificarlo eliminando rami poco significativi.

Il procedimento tipico di post-pruning prevede:

- la valutazione delle prestazioni di un sottoalbero su un validation set;
- la sostituzione del sottoalbero con una foglia;
- il confronto tra le prestazioni prima e dopo la potatura;
- il mantenimento della potatura solo se migliora la generalizzazione.

Il post-pruning risulta generalmente più efficace del pre-pruning, poiché consente di valutare l'impatto reale delle potature sulla capacità di generalizzazione.

Confronto Pre-Pruning vs Post-Pruning

Metodo	Quando applicato	Rischio principale
Pre-pruning	Durante la crescita	Underfitting
Post-pruning	Dopo la crescita	Costo computazionale

3.7 Random Forest

Il *Random Forest* è un metodo di apprendimento basato su un insieme di decision tree, appartenente alla famiglia dei metodi di **ensemble**. L'idea di base è quella di combinare le predizioni di più alberi di decisione al fine di ottenere un modello più robusto e con migliori capacità di generalizzazione.

Idea fondamentale

Un singolo decision tree presenta elevata varianza ed è particolarmente sensibile ai dati di training. Il Random Forest riduce questo problema costruendo molti alberi diversi e aggregandone le predizioni.

Ogni albero viene addestrato:

- su un sottoinsieme casuale degli esempi di training (bootstrap);
- utilizzando, a ogni nodo, un sottoinsieme casuale degli attributi disponibili.

3.7.1 Aggregazione delle predizioni

Nel caso di classificazione, la predizione finale del Random Forest è ottenuta tramite voto di maggioranza tra le predizioni dei singoli alberi. Nel caso di regressione, la predizione finale è data dalla media delle predizioni.

Formalmente, dati $h_1, h_2, \dots, h_T$ decision tree:

$$\hat{y} = \text{majority_vote}\{h_1(x), \dots, h_T(x)\}$$

3.7.2 Riduzione dell'overfitting

Il Random Forest riduce l'overfitting combinando modelli ad alta varianza ma a basso bias. L'introduzione di casualità nella costruzione degli alberi rende le predizioni meno correlate tra loro, migliorando la stabilità del modello complessivo.

A differenza dei singoli decision tree, il Random Forest tende a generalizzare meglio senza richiedere un pruning esplicito.

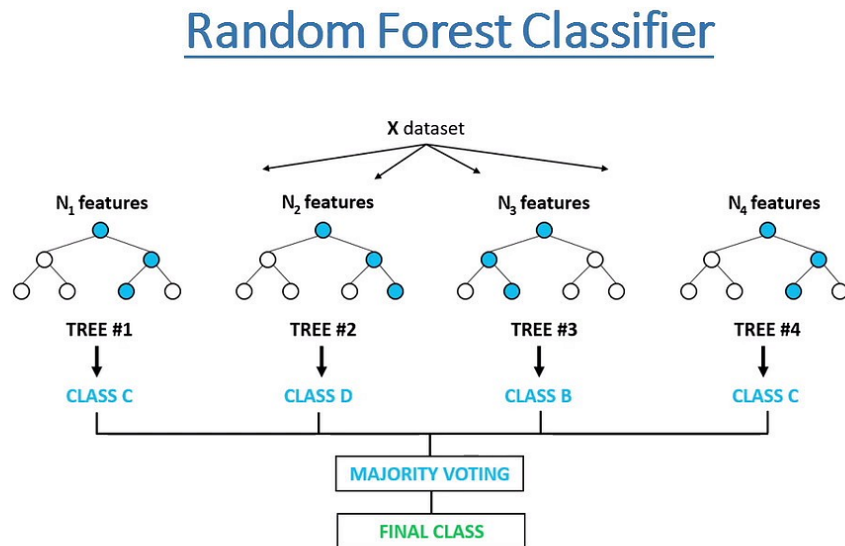


Figure 3.5: Schema concettuale di un Random Forest: più decision tree addestrati su sottoinsiemi diversi dei dati e degli attributi contribuiscono alla predizione finale.

Riepilogo

Decision Tree Modello supervisionato che rappresenta una funzione di decisione sotto forma di struttura ad albero. Ogni cammino radice–foglia corrisponde a una regola di decisione del tipo IF–THEN.

Regole di decisione Un decision tree è logicamente equivalente a un insieme di regole. Ogni foglia dell'albero definisce una regola ottenuta congiungendo le condizioni lungo il percorso radice–foglia.

Algoritmo ID3 Costruisce l'albero in modo greedy e top-down. A ogni nodo seleziona l'attributo che massimizza l'Information Gain e suddivide ricorsivamente il dataset. Non garantisce l'albero ottimo globale e non prevede tecniche di pruning.

Entropia

$$Entropy(S) = - \sum_{i=1}^c p_i \log_2 p_i$$

Misura l'impurità di un insieme di esempi. È nulla se tutti gli esempi appartengono alla stessa classe ed è massima quando le classi sono equamente distribuite.

Information Gain

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

Misura la riduzione di entropia ottenuta suddividendo gli esempi in base a un attributo. ID3 seleziona l'attributo con massimo Information Gain.

Overfitting Un decision tree può adattarsi eccessivamente ai dati di training, ottenendo errore campionario basso ma scarsa capacità di generalizzazione:

$$error_S(h) < error_S(h') \wedge error_D(h) > error_D(h')$$

Pruning Tecniche volte a ridurre la complessità dell'albero per migliorare la generalizzazione.

- *Pre-pruning*: arresto anticipato della crescita (rischio di underfitting);
- *Post-pruning*: potatura di un albero completo sulla base delle prestazioni su validation set.

Random Forest Metodo ensemble che combina più decision tree addestrati su sottoinsiemi casuali di dati e attributi. Riduce la varianza e l'overfitting dei singoli alberi, a costo di una minore interpretabilità.

RIEPILOGO — Probabilità e Bayes Networks

Incertezza In ambienti reali l'informazione è incompleta, rumorosa e parzialmente osservabile. Un approccio puramente logico risulta inadeguato per prendere decisioni efficaci.

Probabilità La probabilità fornisce una rappresentazione quantitativa dell'incertezza. Uno spazio di probabilità è definito da:

$$\Omega \text{ (spazio campionario),} \quad P : \Omega \rightarrow [0, 1], \quad \sum_{\omega \in \Omega} P(\omega) = 1$$

Eventi e variabili casuali Un evento è un sottoinsieme di Ω . Una variabile casuale è una funzione:

$$X : \Omega \rightarrow D(X)$$

che induce una distribuzione di probabilità $P(X = x_i)$.

Distribuzioni di probabilità

- *Distribuzione prior*: credenza prima di osservare evidenza;
- *Distribuzione congiunta*: probabilità di tutte le combinazioni di valori delle variabili;
- *Distribuzione condizionata*: probabilità di una variabile dato il valore di un'altra.

Probabilità condizionata e regola del prodotto

$$P(a \mid b) = \frac{P(a \wedge b)}{P(b)}, \quad P(a, b) = P(a \mid b)P(b)$$

Probabilità totali

$$P(X) = \sum_{y \in D(Y)} P(X \mid Y = y)P(Y = y)$$

Chain Rule

$$P(X_1, \dots, X_n) = \prod_{i=1}^n P(X_i \mid X_1, \dots, X_{i-1})$$

Indipendenza Due eventi sono indipendenti se:

$$P(A, B) = P(A)P(B)$$

L'indipendenza assoluta è rara nei sistemi complessi.

Indipendenza condizionata

$$X \perp Y \mid Z \iff P(X \mid Y, Z) = P(X \mid Z)$$

Consente una rappresentazione compatta delle distribuzioni congiunte.

Regola di Bayes

$$P(Y | X) = \frac{P(X | Y)P(Y)}{P(X)} = \alpha P(X | Y)P(Y)$$

Fondamentale per il ragionamento diagnostico (cause a partire dagli effetti).

Massima verosimiglianza

$$\arg \max_{x_i \in D(X)} P(X = x_i | Y) = \arg \max_{x_i} P(Y | X = x_i)P(X = x_i)$$

Bayesian Networks Modelli grafici che rappresentano variabili casuali e relazioni di dipendenza tramite:

- nodi (variabili);
- archi diretti (dipendenze causali);
- CPT: $P(X_i | Parents(X_i))$.

Le reti bayesiane permettono di rappresentare distribuzioni congiunte in modo compatto, riducendo la complessità da esponenziale a lineare (a parità di genitori).

Chapter 4

Bayesian Learning

4.1 Motivazione

Nel Machine Learning supervisionato, l'obiettivo principale è apprendere una funzione che consenta di effettuare predizioni corrette su nuove istanze non osservate. Nei capitoli precedenti, l'apprendimento è stato affrontato prevalentemente come un problema di ricerca nello spazio delle ipotesi, in cui si seleziona una singola ipotesi che ottimizza una misura di performance sui dati di training.

Tuttavia, in molti contesti reali l'informazione disponibile è incompleta, rumorosa e soggetta a incertezza. In questi casi, risulta limitante adottare un approccio deterministico che restituisca un'unica ipotesi o una singola predizione senza fornire indicazioni sul grado di confidenza associato.

Il **Bayesian Learning** affronta il problema dell'apprendimento introducendo una rappresentazione esplicita dell'incertezza tramite il calcolo delle probabilità. Invece di selezionare una singola ipotesi, l'approccio bayesiano considera una distribuzione di probabilità sulle ipotesi e utilizza i dati osservati per aggiornare tale distribuzione.

In questo quadro, l'apprendimento non consiste esclusivamente nella scelta di un modello, ma nella stima di quanto ogni possibile ipotesi sia plausibile alla luce dei dati disponibili.

4.2 Classificazione come stima probabilistica

Dal punto di vista bayesiano, la classificazione può essere interpretata come un problema di stima probabilistica. Dato un dataset di training D e una nuova istanza x' , l'obiettivo non è semplicemente assegnare un'etichetta, ma stimare la distribuzione di probabilità sulle possibili classi.

Sia $f : X \rightarrow V$ la funzione target, dove X è lo spazio delle istanze e V l'insieme dei possibili valori di output. La miglior predizione per una nuova istanza x' è definita come:

$$\hat{f}(x') = v^* \quad \text{con} \quad v^* = \arg \max_{v \in V} P(v \mid x', D)$$

In forma più generale, l'obiettivo del Bayesian Learning è stimare l'intera distribuzione:

$$P(V \mid x', D)$$

Questa formulazione consente di quantificare l'incertezza associata alle predizioni e costituisce la base per l'introduzione dei classificatori bayesiani.

4.3 ML e MAP

Dato un dataset di training D e una nuova istanza x' , l'obiettivo del Bayesian Learning è stimare la probabilità a posteriori delle classi:

$$P(v \mid x', D)$$

Utilizzando il teorema di Bayes, tale probabilità può essere riscritta come:

$$P(v \mid x', D) = \frac{P(x' \mid v, D) P(v \mid D)}{P(x' \mid D)}$$

Poiché il termine $P(x' \mid D)$ non dipende dalla classe v , esso può essere ignorato nel confronto tra classi.

4.3.1 Maximum Likelihood Classification

Nel criterio di **Maximum Likelihood (ML)**, si assume che tutte le classi siano ugualmente probabili a priori, ovvero:

$$P(v \mid D) = \text{costante}$$

In questo caso, la classificazione si riduce alla massimizzazione della verosimiglianza:

$$v_{ML} = \arg \max_{v \in V} P(x' \mid v, D)$$

Il classificatore ML assegna quindi a x' la classe che rende l'osservazione più probabile, indipendentemente dalla frequenza delle classi nel dataset.

Questo criterio quindi seleziona la classe che meglio spiega l'osservazione corrente. Tuttavia, esso non tiene conto di alcuna informazione a priori sulla distribuzione delle classi. Di conseguenza, il ML può risultare inadeguato in presenza di classi sbilanciate o quando il numero di esempi di training è limitato.

4.3.2 Maximum A Posteriori Classification

Il criterio di **Maximum A Posteriori (MAP)** estende il ML introducendo l'informazione a priori sulle classi. La classificazione avviene selezionando la classe che massimizza la probabilità a posteriori:

$$v_{MAP} = \arg \max_{v \in V} P(v \mid x', D)$$

Applicando il teorema di Bayes e trascurando il termine di normalizzazione, si ottiene:

$$v_{MAP} = \arg \max_{v \in V} P(x' \mid v, D) P(v \mid D)$$

Il criterio MAP combina due informazioni:

- la verosimiglianza $P(x' \mid v, D)$, che misura quanto bene la classe spiega l'osservazione;
- la probabilità a priori $P(v \mid D)$, che riflette la frequenza o plausibilità della classe.

In questo modo, il MAP penalizza classi rare e favorisce classi più probabili a priori, risultando più robusto rispetto al ML in presenza di dati sbilanciati.

Confronto tra ML e MAP

Criterio	Funzione obiettivo	Informazione a priori
ML	$\arg \max_v P(x' v, D)$	No
MAP	$\arg \max_v P(x' v, D) P(v D)$	Sì

Osservazione

Il criterio ML può essere visto come un caso particolare del MAP in cui tutte le classi hanno probabilità a priori uguali. L'introduzione di informazioni a priori consente al MAP di migliorare la qualità delle predizioni in contesti realistici.

4.4 Bayes Optimal Classifier

Nel contesto del Bayesian Learning, i criteri ML e MAP selezionano una singola classe sulla base di una stima puntuale delle probabilità. Tuttavia, tali approcci assumono implicitamente di conoscere con certezza il modello probabilistico sottostante.

Il **Bayes Optimal Classifier** rappresenta il classificatore teoricamente ottimale, in quanto tiene conto di *tutte* le ipotesi possibili pesate dalla loro probabilità a posteriori. In un approccio completamente bayesiano, infatti, non si seleziona una singola ipotesi h , ma si considera l'intera distribuzione di probabilità sulle ipotesi dato il dataset di training. La classificazione ottimale si ottiene combinando le predizioni di tutte le ipotesi, ciascuna pesata dalla propria probabilità a posteriori.

Definizione formale

Sia H lo spazio delle ipotesi e sia D il dataset di training. La probabilità che una nuova istanza x' appartenga alla classe v è data da:

$$P(v | x', D) = \sum_{h \in H} P(v | x', h) P(h | D)$$

Il **Bayes Optimal Classifier** assegna a x' la classe che massimizza tale probabilità:

$$v^* = \arg \max_{v \in V} \sum_{h \in H} P(v | x', h) P(h | D)$$

Questo metodo minimizza l'errore di classificazione atteso rispetto alla distribuzione a posteriori sulle ipotesi. Esso rappresenta quindi il limite superiore delle prestazioni ottenibili da qualsiasi classificatore basato sulle stesse assunzioni probabilistiche.

In pratica, tuttavia, il calcolo esplicito del Bayes Optimal Classifier è spesso intrattabile, poiché richiede di considerare tutte le ipotesi nello spazio H .

Esempio

Si consideri un problema di classificazione binaria con due ipotesi h_1 e h_2 . Supponiamo che, dato il dataset D , le probabilità a posteriori delle ipotesi siano:

$$P(h_1 | D) = 0.6 \quad P(h_2 | D) = 0.4$$

Per una nuova istanza x' , le ipotesi forniscono le seguenti predizioni:

$$P(v = 1 \mid x', h_1) = 0.9 \quad P(v = 1 \mid x', h_2) = 0.2$$

Il Bayes Optimal Classifier calcola:

$$P(v = 1 \mid x', D) = 0.9 \cdot 0.6 + 0.2 \cdot 0.4 = 0.62$$

$$P(v = 0 \mid x', D) = 1 - 0.62 = 0.38$$

Pertanto, la classe assegnata è $v = 1$, ottenuta combinando le predizioni di entrambe le ipotesi.

Osservazione

Il Bayes Optimal Classifier rappresenta il classificatore teoricamente migliore in termini di errore atteso. Nella pratica, modelli come MAP e Naive Bayes possono essere interpretati come approssimazioni computazionalmente efficienti di questo approccio ideale.

4.5 Naive Bayes Classifier

4.5.1 Definizione

Il *Naive Bayes Classifier* è un classificatore probabilistico basato sul teorema di Bayes e rappresenta un'approssimazione pratica del Bayes Optimal Classifier.

Il Bayes Optimal Classifier infatti richiede di considerare tutte le ipotesi possibili nello spazio H , risultando spesso intrattabile dal punto di vista computazionale. Il Naive Bayes supera questo problema assumendo che gli attributi siano condizionatamente indipendenti dato il valore della classe.

Formalmente, se un'istanza è descritta da $x = (x_1, x_2, \dots, x_n)$, si assume che:

$$P(x_1, x_2, \dots, x_n \mid v) = \prod_{i=1}^n P(x_i \mid v)$$

Questa assunzione implica che, fissata la classe, il valore di ciascun attributo non fornisce informazioni sugli altri attributi. In altre parole, tutta la dipendenza tra gli attributi è mediata dalla variabile di classe.

Nella maggior parte dei problemi reali però, gli attributi non sono realmente indipendenti. Ad esempio, nel testo di un documento o nelle caratteristiche di un'immagine, la presenza di una certa informazione rende più probabile la presenza di altre. Pertanto, l'assunzione di indipendenza condizionata è raramente soddisfatta in modo rigoroso.

Nonostante l'assunzione di indipendenza sia spesso violata, il Naive Bayes mostra buone prestazioni in molti contesti pratici. Ciò è dovuto a diversi fattori:

- l'obiettivo della classificazione è confrontare probabilità relative tra classi, non stimare probabilità assolute accurate;
- errori sistematici nelle stime delle probabilità tendono a compensarsi nel confronto tra classi;

- il modello ha bassa varianza, risultando particolarmente robusto in presenza di pochi dati di training;
- la semplicità del modello riduce il rischio di overfitting.

Di conseguenza, anche con stime probabilistiche imprecise, il Naive Bayes è spesso in grado di individuare correttamente la classe più probabile.

4.5.2 Formulazione formale

Sia $x = (x_1, x_2, \dots, x_n)$ una istanza descritta da n attributi e sia $v \in V$ una possibile classe. L'obiettivo è stimare la probabilità a posteriori:

$$P(v \mid x)$$

Applicando il teorema di Bayes, si ottiene:

$$P(v \mid x) = \frac{P(x \mid v) P(v)}{P(x)}$$

Poiché il termine $P(x)$ non dipende dalla classe, esso può essere ignorato nel confronto tra classi.

Utilizzando l'assunzione di indipendenza, la regola di classificazione del Naive Bayes diventa:

$$v_{NB} = \arg \max_{v \in V} P(v) \prod_{i=1}^n P(x_i \mid v)$$

Il classificatore assegna all'istanza x la classe che massimizza il prodotto tra la probabilità a priori della classe e le probabilità condizionate degli attributi.

Algoritmo Naive Bayes

```

Target function f : X -> V
X = A1 x A2 x ... x An
V = {v1, v2, ..., vk}
Training set D
New instance x = <a1, a2, ..., an>

Naive_Bayes_Learn(A, V, D)
  for each class vj in V
    estimate P(vj | D)
    for each attribute Ak
      for each possible value ai of Ak
        estimate P(ai | vj, D)

Classify_New_Instance(x)
  return vNB = argmax_vj P(vj | D) *
    product over ai in x of P(ai | vj, D)

```

L'algoritmo Naive Bayes si articola in due fasi distinte: una fase di apprendimento e una fase di classificazione.

Durante la fase di apprendimento (`Naive_Bayes_Learn`), il modello stima:

- la probabilità a priori di ciascuna classe $P(v_j \mid D)$, calcolata come frequenza relativa della classe nel dataset;
- le probabilità condizionate $P(a_i \mid v_j, D)$, che descrivono la distribuzione dei valori degli attributi all'interno di ciascuna classe.

Nella fase di classificazione (**Classify_New_Instance**), per una nuova istanza x , il classificatore calcola per ogni classe il prodotto tra la probabilità a priori e le probabilità condizionate degli attributi, secondo l'assunzione di indipendenza. La classe assegnata è quella che massimizza tale quantità.

Osservazione

Lo pseudocodice mostra esplicitamente come il Naive Bayes implementi la regola di decisione:

$$v_{NB} = \arg \max_{v \in V} P(v) \prod_{i=1}^n P(x_i \mid v)$$

L'algoritmo non apprende una struttura complessa, ma si limita a stimare un numero limitato di parametri probabilistici, rendendolo semplice ed efficiente.

4.5.3 Stima delle probabilità

Le probabilità utilizzate dal Naive Bayes vengono stimate a partire dal dataset di training:

- $P(v)$: frequenza relativa della classe v ;
- $P(x_i \mid v)$: frequenza dell'attributo x_i all'interno della classe v .

Nel caso di attributi continui, si assume spesso una distribuzione parametrica (ad esempio gaussiana) per modellare $P(x_i \mid v)$.

Esempio

Si consideri un problema di classificazione con due classi $V = \{\text{Spam}, \text{Ham}\}$ e due attributi binari:

$$x = (\text{contains_offer}, \text{contains_link})$$

Supponiamo di aver stimato:

$$P(\text{Spam}) = 0.4 \quad P(\text{Ham}) = 0.6$$

$$P(\text{contains_offer} = 1 \mid \text{Spam}) = 0.8$$

$$P(\text{contains_link} = 1 \mid \text{Spam}) = 0.7$$

$$P(\text{contains_offer} = 1 \mid \text{Ham}) = 0.1$$

$$P(\text{contains_link} = 1 \mid \text{Ham}) = 0.2$$

Per un nuovo messaggio con $x = (1, 1)$, il Naive Bayes calcola:

$$P(\text{Spam} \mid x) \propto 0.4 \cdot 0.8 \cdot 0.7 = 0.224$$

$$P(\text{Ham} \mid x) \propto 0.6 \cdot 0.1 \cdot 0.2 = 0.012$$

Pertanto, il messaggio viene classificato come *Spam*.

Limiti

I limiti del Naive Bayes possono essere riassunti come un compromesso tra semplicità ed espressività:

- **ipotesi di indipendenza spesso violata;**

- **sensibilità a dati sparsi e probabilità nulle;**

Il Naive Bayes stima le probabilità condizionate tramite frequenze osservate. Se una certa combinazione attributo–classe non compare nel dataset di training, la probabilità stimata risulta nulla:

$$P(x_i | v) = 0$$

Poiché la regola di decisione prevede il prodotto delle probabilità condizionate, una singola probabilità nulla è sufficiente ad annullare l'intero termine. Questo problema è particolarmente rilevante in dataset ad alta dimensionalità o con pochi esempi.

- **scarsa calibrazione delle probabilità;**

Le prestazioni del Naive Bayes dipendono fortemente dalla rappresentazione degli attributi. Attributi ridondanti o altamente correlati violano l'assunzione di indipendenza e possono distorcere il calcolo delle probabilità.

Inoltre, per attributi continui, l'assunzione di una specifica distribuzione parametrica (ad esempio gaussiana) può risultare inadeguata se i dati reali non seguono tale distribuzione.

- **esistono modelli più espressivi;**

Nonostante ciò, il Naive Bayes rimane un modello di riferimento per problemi di classificazione ad alta dimensionalità e con dati limitati.

RIEPILOGO

Bayesian Learning Approccio probabilistico all'apprendimento che rappresenta esplicitamente l'incertezza. L'obiettivo non è selezionare una singola ipotesi, ma stimare distribuzioni di probabilità aggiornate sulla base dei dati osservati.

Classificazione come stima probabilistica Dato un dataset D e una nuova istanza x' , la classificazione consiste nel calcolare:

$$v^* = \arg \max_{v \in V} P(v \mid x', D)$$

ovvero la classe con massima probabilità a posteriori.

Maximum Likelihood (ML) Assume classi equiprobabili a priori. La decisione si basa esclusivamente sulla verosimiglianza:

$$v_{ML} = \arg \max_{v \in V} P(x' \mid v, D)$$

Maximum A Posteriori (MAP) Estende il ML includendo le probabilità a priori delle classi:

$$v_{MAP} = \arg \max_{v \in V} P(x' \mid v, D) P(v \mid D)$$

Il ML è un caso particolare del MAP con prior uniformi.

Bayes Optimal Classifier Classificatore teoricamente ottimale che combina le predizioni di tutte le ipotesi pesate dalla loro probabilità a posteriori:

$$v^* = \arg \max_{v \in V} \sum_{h \in H} P(v \mid x', h) P(h \mid D)$$

È generalmente intrattabile nella pratica.

Naive Bayes Classifier

Approssimazione pratica del Bayes Optimal Classifier basata sull'assunzione di indipendenza condizionata degli attributi dato il valore della classe:

$$P(x \mid v) = \prod_{i=1}^n P(x_i \mid v)$$

La regola di decisione è:

$$v_{NB} = \arg \max_{v \in V} P(v) \prod_{i=1}^n P(x_i \mid v)$$

Assunzione di indipendenza

Se un'istanza è descritta da $x = (x_1, x_2, \dots, x_n)$, si assume che:

$$P(x_1, x_2, \dots, x_n \mid v) = \prod_{i=1}^n P(x_i \mid v)$$

Questa assunzione implica che, fissata la classe, il valore di ciascun attributo non fornisce informazioni sugli altri attributi.

Perché funziona Nonostante l'ipotesi semplificativa:

- confronta probabilità relative tra classi;
- ha bassa varianza;
- è robusto con pochi dati e in spazi ad alta dimensionalità.

Limiti principali

- ipotesi di indipendenza spesso violata;
- sensibilità a dati sparsi e probabilità nulle;
- scarsa calibrazione delle probabilità;
- limitata capacità espressiva rispetto a modelli più complessi.

Chapter 5

Probabilistic Models for Classification

Nel problema di classificazione supervisionata si assume l'esistenza di una funzione target

$$f : X \rightarrow \mathcal{C},$$

dove $X \subseteq \mathbb{R}^d$ è lo spazio delle istanze e $\mathcal{C} = \{C_1, \dots, C_K\}$ è l'insieme delle classi. Dato un dataset di training $D = \{(x_n, t_n)\}_{n=1}^N$ e una nuova istanza $x \notin D$, l'obiettivo è stimare la probabilità a posteriori

$$P(C_k | x),$$

e assegnare la classe più probabile.

I modelli probabilistici per la classificazione si suddividono in due grandi famiglie: modelli generativi e modelli discriminativi.

5.1 Modelli probabilistici generativi

I modelli generativi stimano esplicitamente la distribuzione dei dati condizionata alla classe $P(x | C_k)$ e la probabilità a priori delle classi $P(C_k)$. La probabilità a posteriori viene poi ottenuta tramite il teorema di Bayes.

Nel caso di due classi C_1, C_2 , assumendo un modello parametrico gaussiano, si ha:

$$P(C_1) = \pi, \quad P(C_2) = 1 - \pi,$$

$$P(x | C_i) = \mathcal{N}(x; \mu_i, \Sigma).$$

La probabilità a posteriori può essere scritta nella forma:

$$P(C_1 | x) = \sigma(a), \quad a = \ln \frac{P(x | C_1)P(C_1)}{P(x | C_2)P(C_2)},$$

dove $\sigma(\cdot)$ è la funzione sigmoide.

5.1.1 Caso gaussiano con covarianza condivisa

Se le due classi condividono la stessa matrice di covarianza Σ , il termine a diventa una funzione lineare dell'input:

$$a = w^T x + w_0,$$

con

$$w = \Sigma^{-1}(\mu_1 - \mu_2), \quad w_0 = -\frac{1}{2}\mu_1^T \Sigma^{-1} \mu_1 + \frac{1}{2}\mu_2^T \Sigma^{-1} \mu_2 + \ln \frac{\pi}{1 - \pi}.$$

La classificazione avviene confrontando $P(C_1 | x)$ con una soglia (tipicamente 0.5).

5.1.2 Stima dei parametri tramite Maximum Likelihood

Assumendo un modello gaussiano, i parametri $\pi, \mu_1, \mu_2, \Sigma$ vengono stimati massimizzando la likelihood sul dataset di training. Nel caso binario, le soluzioni ML sono:

$$\hat{\pi} = \frac{N_1}{N}, \quad \hat{\mu}_1 = \frac{1}{N_1} \sum_{n:t_n=1} x_n, \quad \hat{\mu}_2 = \frac{1}{N_2} \sum_{n:t_n=0} x_n, \\ \hat{\Sigma} = \frac{N_1}{N} S_1 + \frac{N_2}{N} S_2,$$

dove S_i sono le matrici di covarianza empiriche delle classi.

5.1.3 Estensione a K classi

Nel caso multi-classe, la probabilità a posteriori assume la forma:

$$P(C_k | x) = \frac{\exp(a_k)}{\sum_j \exp(a_j)}, \quad a_k = \ln P(x | C_k) P(C_k),$$

che corrisponde alla funzione *softmax*.

Anche in questo caso, assumendo distribuzioni gaussiane con covarianza condivisa, i parametri possono essere stimati tramite Maximum Likelihood.

5.1.4 Gaussian Naive Bayes

Nel Gaussian Naive Bayes si assume l'indipendenza condizionata delle feature dato il valore della classe:

$$P(x | C_k) = \prod_{j=1}^m P(x_j | C_k),$$

con

$$P(x_j | C_k) = \mathcal{N}(x_j; \mu_{j,k}, \sigma_{j,k}^2).$$

Questa assunzione riduce drasticamente il numero di parametri da stimare e rende il modello particolarmente adatto a problemi ad alta dimensionalità.

5.2 Modelli probabilistici discriminativi

I modelli discriminativi stimano direttamente la probabilità a posteriori $P(C_k | x)$ senza modellare $P(x | C_k)$. Un esempio fondamentale è la regressione logistica.

5.2.1 Logistic Regression

Nel caso binario, la regressione logistica modella la probabilità come:

$$P(C_1 | \tilde{x}) = \sigma(\tilde{w}^T \tilde{x}),$$

dove $\tilde{x} = (1, x^T)^T$.

I parametri vengono stimati minimizzando la cross-entropy (negative log-likelihood):

$$E(\tilde{w}) = - \sum_{n=1}^N [t_n \ln y_n + (1 - t_n) \ln(1 - y_n)].$$

L'ottimizzazione viene risolta tramite metodi iterativi, in particolare l'algoritmo *Iterative Reweighted Least Squares* (IRLS).

5.2.2 Regressione logistica multi-classe

Per K classi, la probabilità a posteriori è modellata tramite softmax:

$$P(C_k | \tilde{x}) = \frac{\exp(\tilde{w}_k^T \tilde{x})}{\sum_j \exp(\tilde{w}_j^T \tilde{x})}.$$

Anche in questo caso, i parametri sono stimati minimizzando la cross-entropy multi-classe tramite ottimizzazione iterativa.

5.2.3 Generalizzazione e spazio delle feature

Tutti i modelli descritti possono essere applicati in uno spazio delle feature trasformato. Dato un mapping

$$\phi : X \rightarrow \Phi,$$

le istanze vengono proiettate nello spazio Φ e l'apprendimento avviene sulle nuove rappresentazioni.

Questo approccio consente di aumentare la capacità espressiva del modello senza modificarne la struttura probabilistica.

5.3 RIEPILOGO

Classificazione probabilistica

Nel problema di classificazione si stima la probabilità a posteriori delle classi dato un input x :

$$P(C_k | x)$$

La decisione consiste nell'assegnare la classe con probabilità massima.

Modelli generativi

Stimano esplicitamente la distribuzione dei dati e delle classi:

$$P(x | C_k), \quad P(C_k)$$

La probabilità a posteriori è ottenuta tramite il teorema di Bayes. Esempi: Gaussian Classifier, Gaussian Naive Bayes.

Classificatore gaussiano

Assumendo distribuzioni gaussiane per ciascuna classe:

$$P(x | C_k) = \mathcal{N}(x; \mu_k, \Sigma_k)$$

Se le classi condividono la stessa covarianza, la funzione discriminante è lineare:

$$a_k = w_k^T x + w_{k0}$$

Stima dei parametri (ML)

I parametri del modello generativo vengono stimati tramite Maximum Likelihood:

$$\hat{\pi}_k = \frac{N_k}{N}, \quad \hat{\mu}_k = \frac{1}{N_k} \sum_{n:t_n=k} x_n$$

Estensione multi-classe

Nel caso di K classi, la probabilità a posteriori assume la forma softmax:

$$P(C_k | x) = \frac{\exp(a_k)}{\sum_j \exp(a_j)}$$

Gaussian Naive Bayes

Assume indipendenza condizionata delle feature dato il valore della classe:

$$P(x | C_k) = \prod_{j=1}^m P(x_j | C_k)$$

Riduce drasticamente il numero di parametri ed è adatto a problemi ad alta dimensionalità.

Modelli discriminativi

Stimano direttamente la probabilità a posteriori $P(C_k | x)$, senza modellare la distribuzione dei dati. Esempio fondamentale: regressione logistica.

Logistic Regression

Nel caso binario:

$$P(C_1 | x) = \sigma(w^T x)$$

I parametri sono stimati minimizzando la cross-entropy tramite ottimizzazione iterativa (IRLS).

Logistic Regression multi-classe

Utilizza la funzione softmax:

$$P(C_k | x) = \frac{\exp(w_k^T x)}{\sum_j \exp(w_j^T x)}$$

Spazio delle feature

I modelli probabilistici possono operare in uno spazio trasformato tramite una mappa:

$$\phi : X \rightarrow \Phi$$

Aumentando la capacità espressiva senza modificare la struttura del modello.

Chapter 6

Modelli lineari per la classificazione

6.1 Dati linearmente separabili

Un insieme di dati è detto **linearmente separabile** se esiste un iperpiano che separa perfettamente le istanze appartenenti a classi diverse.

Formalmente, dati esempi $x \in \mathbb{R}^d$ appartenenti a due classi, essi sono linearmente separabili se esistono un vettore $w \in \mathbb{R}^d$ e uno scalare $w_0 \in \mathbb{R}$ tali che:

$$w^T x + w_0 > 0 \quad \text{per tutti gli esempi della prima classe}$$

$$w^T x + w_0 < 0 \quad \text{per tutti gli esempi della seconda classe}$$

In questo caso, l'iperpiano definito dall'equazione $w^T x + w_0 = 0$ separa esattamente le due classi.

Interpretazione geometrica

L'iperpiano $w^T x + w_0 = 0$ divide lo spazio delle feature in due semispazi. Il segno della funzione discriminante determina la classe assegnata all'istanza.

Nel caso bidimensionale, l'iperpiano corrisponde a una retta; nel caso tridimensionale, a un piano. In dimensioni superiori, la separazione avviene tramite un iperpiano.

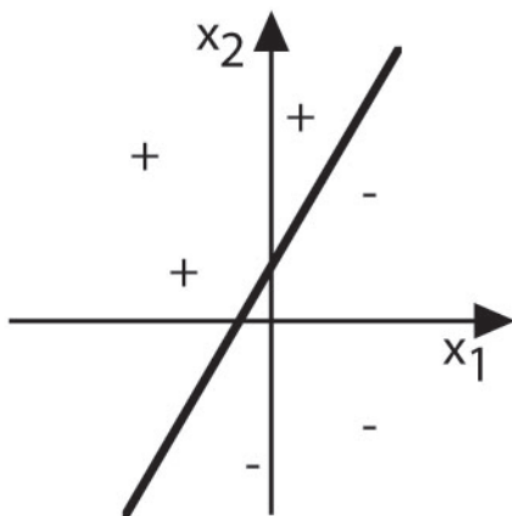


Figure 6.1: Esempio di dati linearmente separabili tramite una retta.

6.1.1 Dati non linearmente separabili

Un insieme di dati è detto **non linearmente separabile** se non esiste alcun iperpiano in grado di separare perfettamente le classi.

In tali casi, una funzione discriminante lineare non è sufficiente a risolvere il problema di classificazione, rendendo necessaria l'introduzione di modelli non lineari o di trasformazioni dello spazio delle feature.

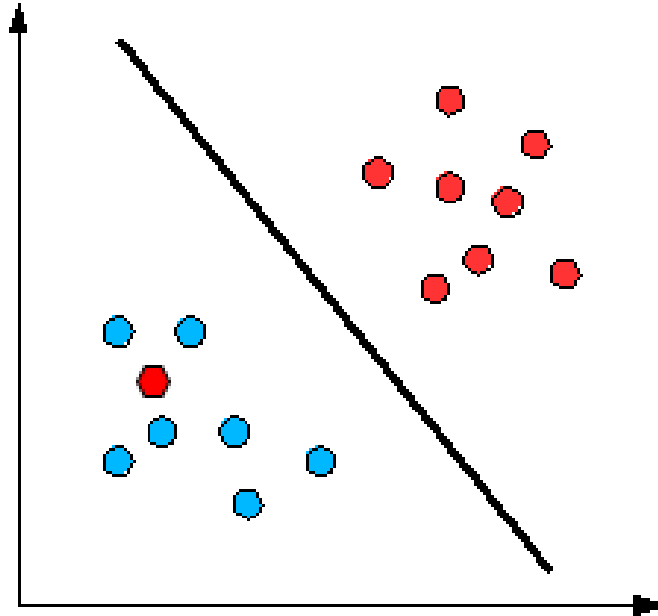


Figure 6.2: Esempio di dati non linearmente separabili.

6.2 Linear Discriminant Function

Una **funzione discriminante lineare** è una funzione del tipo:

$$g(x) = w^T x + w_0$$

dove:

- $w \in \mathbb{R}^d$ è il vettore dei pesi;
- $w_0 \in \mathbb{R}$ è il termine di bias;
- $g(x) \in \mathbb{R}$ è il valore della funzione discriminante.

Essa associa a ogni istanza un valore reale, il cui segno viene utilizzato per effettuare la classificazione.

6.2.1 Regola di decisione

La regola di classificazione basata su una funzione discriminante lineare è:

$$\text{assegna } x \text{ alla classe } \begin{cases} C_1 & \text{se } g(x) \geq 0 \\ C_2 & \text{se } g(x) < 0 \end{cases}$$

Il valore assoluto di $g(x)$ fornisce una misura della distanza dell'istanza dall'iperpiano di separazione.

Interpretazione geometrica

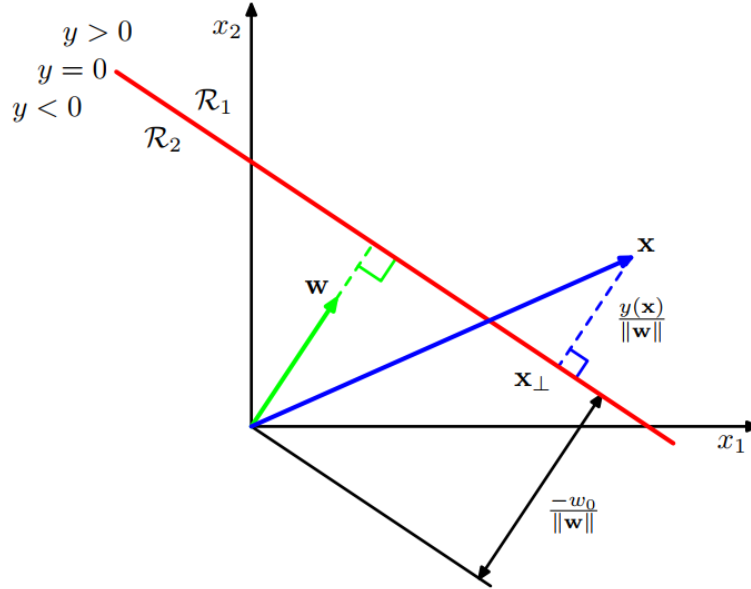


Figure 6.3: Interpretazione geometrica di una funzione discriminante lineare in due dimensioni.

Il vettore dei pesi w è ortogonale all'iperpiano di decisione. Esso indica la direzione di massima crescita della funzione discriminante.

Nella figura, il vettore w è rappresentato come una freccia perpendicolare alla retta di separazione.

Dato un punto x , il valore della funzione discriminante $g(x)$ è proporzionale alla distanza con segno del punto dall'iperpiano.

La distanza geometrica di x dall'iperpiano è data da:

$$\frac{g(x)}{\|w\|}$$

Nella figura, $x_{\perp}$ rappresenta la proiezione ortogonale del punto x sull'iperpiano.

Il termine di bias w_0 determina la posizione dell'iperpiano rispetto all'origine. In particolare, la distanza dell'iperpiano dall'origine è:

$$-\frac{w_0}{\|w\|}$$

Questo valore controlla la traslazione dell'iperpiano senza modificarne l'orientamento.

6.3 Learning di funzioni discriminanti lineari

Nei paragrafi precedenti è stata introdotta la funzione discriminante lineare come strumento per separare due classi mediante un iperpiano nello spazio delle feature. Resta ora da affrontare il problema fondamentale dell'apprendimento: come determinare i parametri w e w_0 a partire da un insieme di dati etichettati.

I **Linear Discriminant Learning Algorithms** comprendono una famiglia di metodi che apprendono una funzione discriminante lineare a partire da esempi di training. Tali algoritmi condividono l'obiettivo di costruire una funzione del tipo

$$g(x) = w^T x + w_0$$

in grado di separare, nel miglior modo possibile, le istanze appartenenti a classi diverse.

Esistono diversi criteri secondo cui è possibile apprendere una funzione discriminante lineare. Alcuni algoritmi mirano a separare correttamente i dati, altri a massimizzare una misura di separazione tra le classi, altri ancora a minimizzare un errore globale.

Tra i principali approcci rientrano:

- il **criterio dei minimi quadrati**, che apprende i parametri minimizzando l'errore quadratico tra output desiderato e output del modello;
- il **Perceptron**, che aggiorna iterativamente i pesi correggendo gli errori di classificazione;
- il **Fisher Linear Discriminant**, che massimizza la separazione statistica tra le classi.
- il **Support Vector Machine**, che trova l'iperpiano ottimale che separa i dati in classi diverse massimizzando il margine tra esse.

Nonostante le differenze nei criteri di ottimizzazione, tutti questi metodi producono una funzione discriminante lineare.

6.4 Least Squares per la classificazione

Il criterio dei **minimi quadrati** rappresenta uno dei metodi più semplici e fondamentali per apprendere una funzione discriminante lineare a partire da un insieme di dati etichettati. L'idea di base consiste nel trattare il problema di classificazione come un problema di approssimazione funzionale.

6.4.1 Definizione del problema

Si consideri un dataset di training:

$$\mathcal{D} = \{(x_n, t_n)\}_{n=1}^N$$

dove:

- $x_n \in \mathbb{R}^d$ è il vettore delle feature dell' n -esimo esempio;
- $t_n \in \mathbb{R}$ è il valore target associato all'esempio.

Nel contesto della classificazione binaria, le etichette sono tipicamente codificate come:

$$t_n \in \{-1, +1\}$$

Si assume un modello lineare della forma:

$$y(x) = w^T x + w_0$$

Introducendo il vettore delle feature esteso:

$$\tilde{x} = \begin{pmatrix} 1 \\ x \end{pmatrix}, \quad \tilde{w} = \begin{pmatrix} w_0 \\ w \end{pmatrix},$$

il modello può essere riscritto in forma compatta come:

$$y(x) = \tilde{w}^T \tilde{x}$$

6.4.2 Funzione di costo

Il criterio dei minimi quadrati consiste nel minimizzare la somma degli errori quadratici tra l'output del modello e i target desiderati:

$$E(\tilde{w}) = \frac{1}{2} \sum_{n=1}^N (y(x_n) - t_n)^2$$

La funzione di costo misura quanto il modello si discosta dai valori target desiderati sull'insieme di training. Per ciascun esempio x_n , il termine

$$(y(x_n) - t_n)^2$$

rappresenta l'errore commesso dal modello su quell'esempio.

La somma degli errori quadratici tiene conto di tutti gli esempi del dataset e penalizza maggiormente gli errori di grande entità. Minimizzare la funzione di costo equivale quindi a trovare i parametri w e w_0 che rendono l'output del modello il più vicino possibile ai target, in media, su tutti i dati di training.

6.4.3 Soluzione ottima

Il minimo della funzione di costo si ottiene imponendo l'annullamento del gradiente:

$$\nabla E(\tilde{w}) = 0$$

Da cui si ricavano le **equazioni normali**:

$$\tilde{X}^T \tilde{X} \tilde{w} = \tilde{X}^T t$$

Se la matrice $\tilde{X}^T \tilde{X}$ è invertibile, la soluzione è:

$$\tilde{w} = (\tilde{X}^T \tilde{X})^{-1} \tilde{X}^T t$$

La soluzione ottima del problema dei minimi quadrati è il vettore dei parametri che minimizza globalmente la funzione di costo. Poiché la funzione di costo è quadratica nei parametri, essa è convessa e ammette un unico minimo globale.

Questo significa che la soluzione ottenuta risolvendo le equazioni normali è la migliore possibile secondo il criterio dei minimi quadrati, e non dipende dall'inizializzazione dei parametri.

6.4.4 Regola di classificazione

Una volta stimati i parametri $\tilde{w}$, la classificazione di una nuova istanza x avviene utilizzando il segno dell'output del modello:

$$\hat{t} = \text{sign}(y(x)) = \text{sign}(w^T x + w_0)$$

Il valore continuo $y(x)$ può essere interpretato come una misura di confidenza rispetto alla decisione di classificazione.

6.4.5 Sensibilità agli outlier

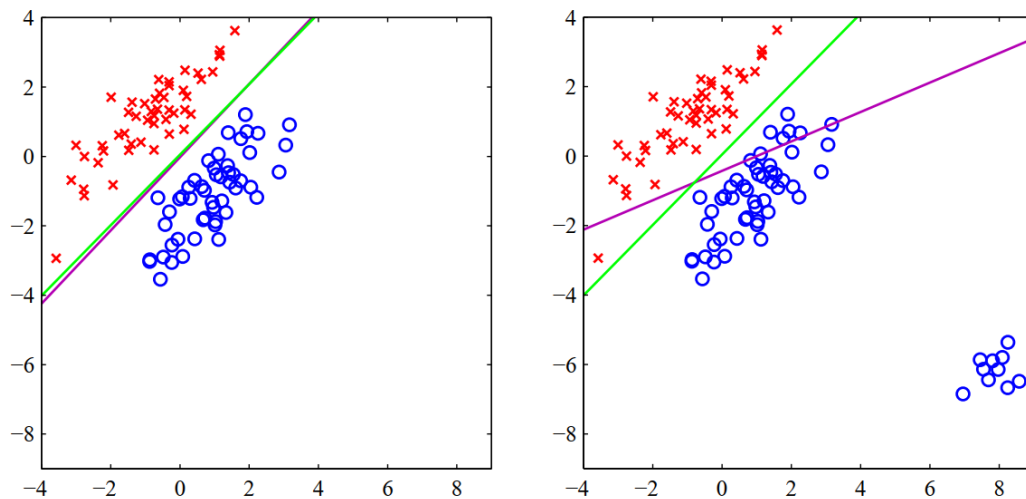


Figure 6.4: Effetto degli outlier sul metodo dei minimi quadrati. L'aggiunta di pochi punti anomali può modificare significativamente la frontiera di decisione.

La figura illustra l'effetto degli outlier sull'apprendimento di una funzione discriminante lineare tramite il criterio dei minimi quadrati.

Nel pannello di sinistra, le due classi sono ben separate e la funzione discriminante appresa (retta verde) fornisce una separazione ragionevole dei dati. Nel pannello di destra, l'introduzione di pochi punti anomali (outlier) appartenenti alla classe blu, ma lontani dalla distribuzione principale, provoca una variazione significativa della retta di separazione (retta viola).

Questo comportamento è una conseguenza diretta della funzione di costo dei minimi quadrati: Poiché l'errore è elevato al quadrato, gli esempi per cui la differenza tra output del modello e target è grande contribuiscono in modo sproporzionato al valore complessivo della funzione di costo. Di conseguenza, pochi outlier possono dominare l'ottimizzazione e influenzare fortemente la posizione dell'iperpiano appreso.

NOTA BENE:

Un metodo di apprendimento si dice **non robusto agli outlier** quando la presenza di pochi esempi anomali è sufficiente a modificare in modo significativo il modello appreso.

Nel caso del Least Squares, l'algoritmo cerca di ridurre l'errore quadratico complessivo, anche a costo di peggiorare la classificazione della maggioranza dei dati, pur di adattarsi ai punti anomali. Questo limite rende il criterio dei minimi quadrati poco adatto ai problemi di classificazione, in cui l'obiettivo principale è la corretta separazione delle classi piuttosto che l'approssimazione numerica dei target.

Tale osservazione motiva l'introduzione di algoritmi alternativi, come il perceptron o i metodi basati su funzioni di costo non quadratiche, più robusti alla presenza di outlier.

6.5 Perceptron

Il **Perceptron** è uno dei più semplici algoritmi di apprendimento supervisionato per la classificazione binaria. Esso apprende una funzione discriminante lineare a partire da esempi etichettati, aggiornando iterativamente i parametri del modello in base agli errori di classificazione.

Il Perceptron può essere interpretato sia come un algoritmo di apprendimento per funzioni discriminanti lineari, sia come un modello elementare di neurone artificiale.

6.5.1 Definizione del problema

Si consideri un insieme di dati di training:

$$\mathcal{D} = \{(x_n, t_n)\}_{n=1}^N$$

dove:

- $x_n \in \mathbb{R}^d$ è il vettore delle feature dell' n -esimo esempio;
- $t_n \in \{-1, +1\}$ è l'etichetta di classe associata.

L'obiettivo è apprendere una funzione:

$$f : \mathbb{R}^d \rightarrow \{-1, +1\}$$

che assegni correttamente un'etichetta di classe a nuove istanze non osservate.

6.5.2 Modello del Perceptron

Il Perceptron utilizza un modello lineare della forma:

$$g(x) = w^T x + w_0$$

L'output del classificatore è ottenuto applicando una funzione di soglia:

$$o(x) = \begin{cases} +1 & \text{se } g(x) > 0 \\ -1 & \text{altrimenti} \end{cases} = \text{sign}(w^T x + w_0)$$

La funzione di soglia separa lo spazio delle feature in due regioni distinte, corrispondenti alle due classi. Il valore assoluto di $g(x)$ indica quanto il punto si trovi lontano dall'iperpiano di decisione, ma non influisce direttamente sulla decisione finale.

Questo comportamento distingue il Perceptron da modelli probabilistici, nei quali l'output rappresenta una probabilità.

6.5.3 Perceptron Learning Rule

L'apprendimento nel Perceptron avviene in modo iterativo, aggiornando i pesi solo quando un esempio di training viene classificato in modo errato.

Dato un esempio (x_n, t_n) , con $t_n \in \{-1, +1\}$, il modello produce un output:

$$o(x_n) = \text{sign}(w^T x_n + w_0).$$

Se $o(x_n) = t_n$, l'esempio è classificato correttamente e non viene effettuato alcun aggiornamento. Se invece $o(x_n) \neq t_n$, i parametri vengono aggiornati secondo la seguente regola.

In presenza di errore di classificazione, i pesi vengono aggiornati come:

$$w \leftarrow w + \eta t_n x_n$$

$$w_0 \leftarrow w_0 + \eta t_n$$

dove $\eta > 0$ è il tasso di apprendimento.

L'aggiornamento dei pesi modifica l'iperpiano di decisione in modo da ridurre l'errore commesso sull'esempio corrente.

In particolare:

- se un esempio positivo viene classificato come negativo, il termine $t_n x_n$ spinge l'iperpiano verso l'esempio;
- se un esempio negativo viene classificato come positivo, l'aggiornamento avviene nella direzione opposta.

In questo modo, l'iperpiano viene progressivamente adattato per separare correttamente i dati.

Forma compatta

La regola di aggiornamento del Perceptron può essere scritta in forma compatta come:

$$w \leftarrow w + \eta t_n x_n \quad \text{se } t_n(w^T x_n + w_0) \leq 0.$$

Questa espressione evidenzia che l'aggiornamento avviene esclusivamente in presenza di errore.

6.5.4 Algoritmo del Perceptron

Input:

Training set $D = \{(x_n, t_n)\}$, $t_n \in \{-1, +1\}$
Learning rate $\eta > 0$

Initialize:

$w = 0$
 $w_0 = 0$

Repeat until convergence:

```
for each  $(x_n, t_n)$  in  $D$ :  
    if  $t_n * (w^T x_n + w_0) \leq 0$  then  
         $w \leftarrow w + \eta * t_n * x_n$   
         $w_0 \leftarrow w_0 + \eta * t_n$   
    end if  
end for
```

L'algoritmo del Perceptron procede iterativamente scorrendo il dataset di training. Per ciascun esempio, viene verificata la correttezza della classificazione.

Se un esempio è classificato correttamente, i parametri restano invariati. In caso contrario, i pesi e il bias vengono aggiornati per correggere l'errore. L'algoritmo termina quando un intero passaggio sui dati non produce aggiornamenti.

Esempio di applicazione

Si consideri un semplice dataset bidimensionale con target binari:

$$\mathcal{D} = \{(x_1, t_1), (x_2, t_2), (x_3, t_3)\}$$

con:

$$\begin{aligned}x_1 &= (1, 1), & t_1 &= +1 \\x_2 &= (2, 0), & t_2 &= +1 \\x_3 &= (-1, -1), & t_3 &= -1\end{aligned}$$

Si inizializzano i parametri:

$$w = (0, 0), \quad w_0 = 0$$

e si assume $\eta = 1$.

Per il primo esempio $x_1 = (1, 1)$:

$$g(x_1) = w^T x_1 + w_0 = 0$$

Poiché:

$$t_1 g(x_1) = 0 \leq 0,$$

l'esempio è considerato errato e si applica l'aggiornamento:

$$w \leftarrow (0, 0) + 1 \cdot (+1) \cdot (1, 1) = (1, 1)$$

$$w_0 \leftarrow 0 + 1 \cdot (+1) = 1$$

Per $x_2 = (2, 0)$:

$$g(x_2) = (1, 1)^T(2, 0) + 1 = 3$$

Poiché:

$$t_2 g(x_2) = (+1) \cdot 3 > 0,$$

l'esempio è classificato correttamente e non viene effettuato alcun aggiornamento.

Per $x_3 = (-1, -1)$:

$$g(x_3) = (1, 1)^T(-1, -1) + 1 = -1$$

Poiché:

$$t_3 g(x_3) = (-1) \cdot (-1) > 0,$$

anche questo esempio è classificato correttamente.

Al termine del primo ciclo sui dati, non si verificano ulteriori errori. L'iperpiano appreso è quindi:

$$w^T x + w_0 = 0 \quad \Rightarrow \quad x_1 + x_2 + 1 = 0$$

Questo iperpiano separa correttamente le due classi del dataset.

Visualizzazione dell'algoritmo del Perceptron

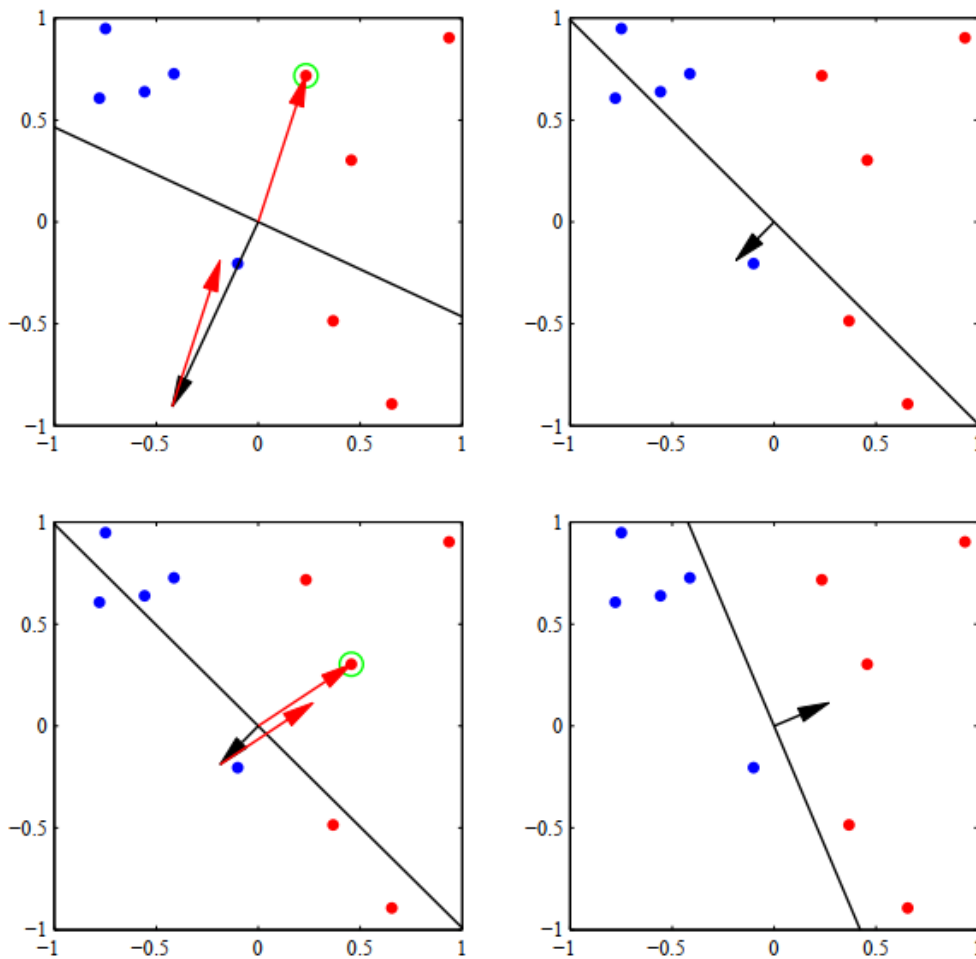


Figure 6.5: Evoluzione dell'iperpiano di decisione durante l'apprendimento del Perceptron. Ogni pannello rappresenta un aggiornamento successivo dei pesi in risposta a un errore di classificazione.

La figura mostra il comportamento dell'algoritmo del Perceptron durante il processo di apprendimento su un dataset bidimensionale.

In ciascun pannello è rappresentato l'iperpiano di decisione corrente, insieme agli esempi di training appartenenti alle due classi. Le frecce indicano la direzione dell'aggiornamento dei pesi quando viene incontrato un esempio classificato erroneamente. Quando un esempio viene classificato in modo errato, il Perceptron modifica i parametri del modello secondo la regola:

$$w \leftarrow w + \eta t_n x_n$$

Questo aggiornamento sposta l'iperpiano nella direzione dell'esempio mal classificato se $t_n = +1$, o nella direzione opposta se $t_n = -1$. Nella figura, i punti cerchiati evidenziano gli esempi che causano l'aggiornamento dei pesi.

Dal punto di vista geometrico, ogni aggiornamento ruota e trasla l'iperpiano di decisione nel tentativo di ridurre il numero di errori di classificazione.

Il Perceptron non ottimizza una funzione di costo globale, ma corregge localmente l'errore commesso sull'esempio corrente. Di conseguenza, la traiettoria seguita dall'iperpiano dipende dall'ordine con cui vengono presentati i dati di training.

6.5.5 Interpretazione neurale

Il perceptron è la forma più semplice di rete neurale: consiste in un singolo neurone artificiale che riceve input pesati, li somma, applica una funzione di attivazione (tipicamente a gradino) e produce un output binario per classificare linearmente i dati. Storicamente è stato il primo modello di rete neurale (proposto da Rosenblatt nel 1958) e ha posto le basi concettuali per le reti neurali moderne. La sua limitazione principale è che può risolvere solo problemi linearmente separabili - per questo motivo le reti neurali si sono evolute in architetture multi-strato (multilayer perceptron o MLP) capaci di apprendere relazioni non lineari grazie ai neuroni nascosti e alla retropropagazione dell'errore.

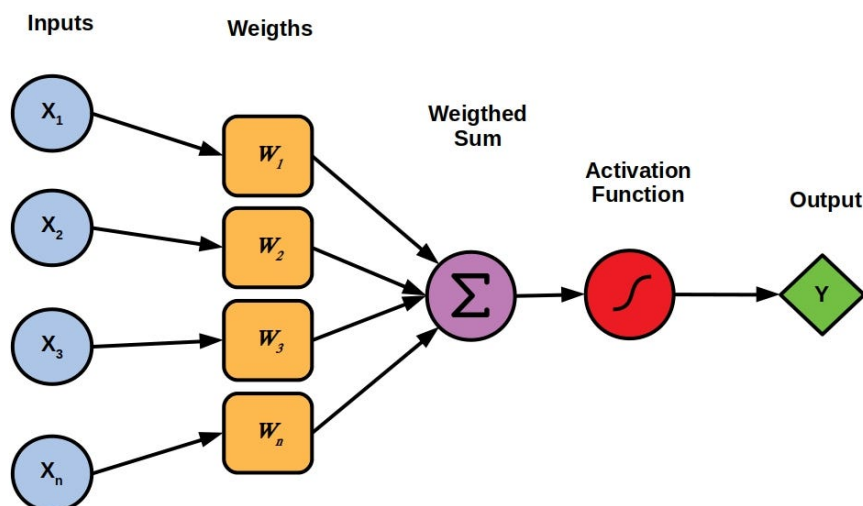


Figure 6.6: Visualizzazione grafica del Perceptron come neurone artificiale.

Nell'immagine vediamo come ciascun input x_i è associato a un peso w_i , mentre il termine di bias è modellato tramite un input costante $x_0 = 1$ con peso w_0 .

Il neurone calcola una combinazione lineare degli input:

$$\sum_{i=0}^d w_i x_i$$

e applica una funzione di attivazione a soglia che restituisce l'etichetta di classe.

6.6 Fisher's Linear Discriminant

Il **Fisher's Linear Discriminant** è un metodo di classificazione lineare basato su un criterio statistico. A differenza del Perceptron e del Least Squares, che si basano rispettivamente sugli errori di classificazione e sull'errore quadratico, il metodo di Fisher mira a massimizzare la separazione tra le classi in termini di distribuzione dei dati.

Impostazione del problema

Si consideri un problema di classificazione binaria con due classi:

$$\mathcal{C}_1 \quad \text{e} \quad \mathcal{C}_2$$

Ciascun esempio $x \in \mathbb{R}^d$ viene proiettato su una retta mediante una funzione lineare:

$$y = w^T x$$

L'obiettivo è determinare il vettore w tale che, dopo la proiezione, le due classi risultino il più possibile separate.

L'idea centrale del Fisher's Linear Discriminant consiste nel trovare una direzione di proiezione che:

- massimizzi la distanza tra le medie delle classi proiettate;
- minimizzi la dispersione interna a ciascuna classe.

In questo modo, le classi risultano ben separate lungo la direzione di proiezione scelta.

6.6.1 Medie delle classi

Siano:

$$\mu_1 = \frac{1}{N_1} \sum_{x \in \mathcal{C}_1} x, \quad \mu_2 = \frac{1}{N_2} \sum_{x \in \mathcal{C}_2} x$$

le medie dei dati appartenenti alle due classi nello spazio originale. Dopo la proiezione sulla direzione w , le medie diventano:

$$m_1 = w^T \mu_1, \quad m_2 = w^T \mu_2$$

6.6.2 Varianza intra-classe

La dispersione dei dati di ciascuna classe dopo la proiezione è misurata dalla varianza:

$$s_i^2 = \sum_{x \in \mathcal{C}_i} (w^T x - m_i)^2, \quad i = 1, 2$$

La varianza totale intra-classe è definita come:

$$s_w^2 = s_1^2 + s_2^2$$

6.6.3 Criterio di Fisher

Il criterio di Fisher è definito come il rapporto tra la distanza quadratica tra le medie proiettate e la varianza intra-classe:

$$J(w) = \frac{(m_1 - m_2)^2}{s_w^2}$$

Il problema di apprendimento consiste nel trovare il vettore w che massimizza tale criterio.

Forma matriciale

Introducendo le matrici di dispersione intra-classe:

$$S_i = \sum_{x \in \mathcal{C}_i} (x - \mu_i)(x - \mu_i)^T, \quad i = 1, 2$$

e la matrice di dispersione totale:

$$S_W = S_1 + S_2,$$

il criterio di Fisher può essere riscritto come:

$$J(w) = \frac{w^T (\mu_1 - \mu_2)(\mu_1 - \mu_2)^T w}{w^T S_W w}$$

Interpretazione geometrica del criterio di Fisher

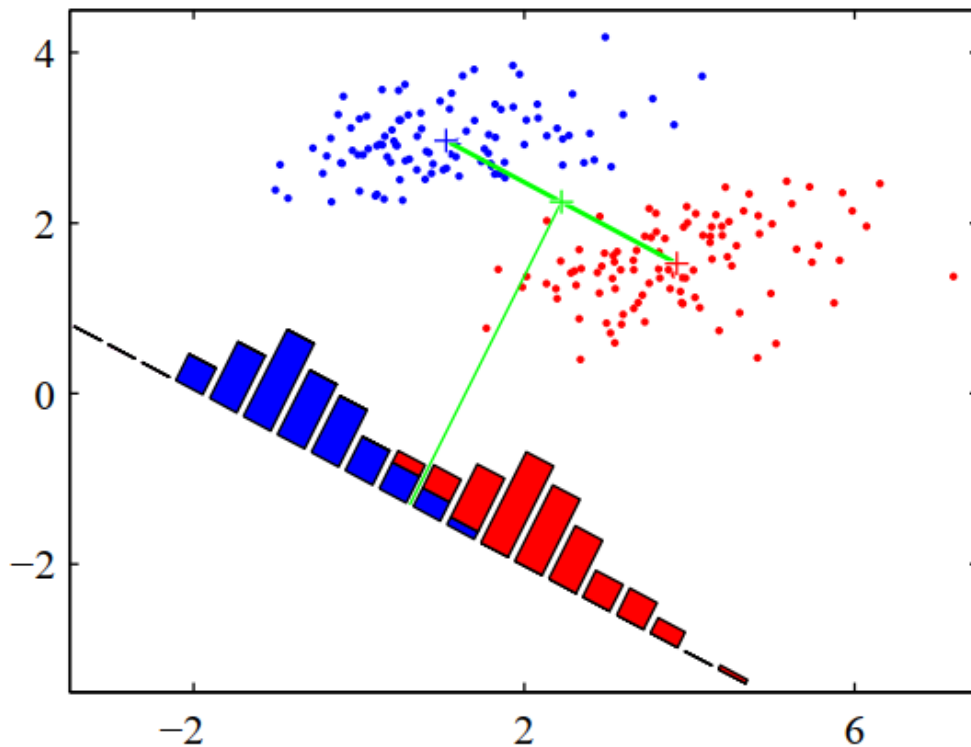


Figure 6.7: *Proiezione lungo la direzione della differenza delle medie. La dispersione intra-classe non è considerata.*

Questa prima figura mostra una proiezione dei dati lungo la direzione che unisce le medie delle due classi. Questa scelta massimizza la distanza tra le medie proiettate, ma non tiene conto della dispersione interna delle classi.

Se una classe presenta un'elevata varianza lungo tale direzione, le distribuzioni proiettate possono sovrapporsi significativamente, riducendo la capacità discriminante.

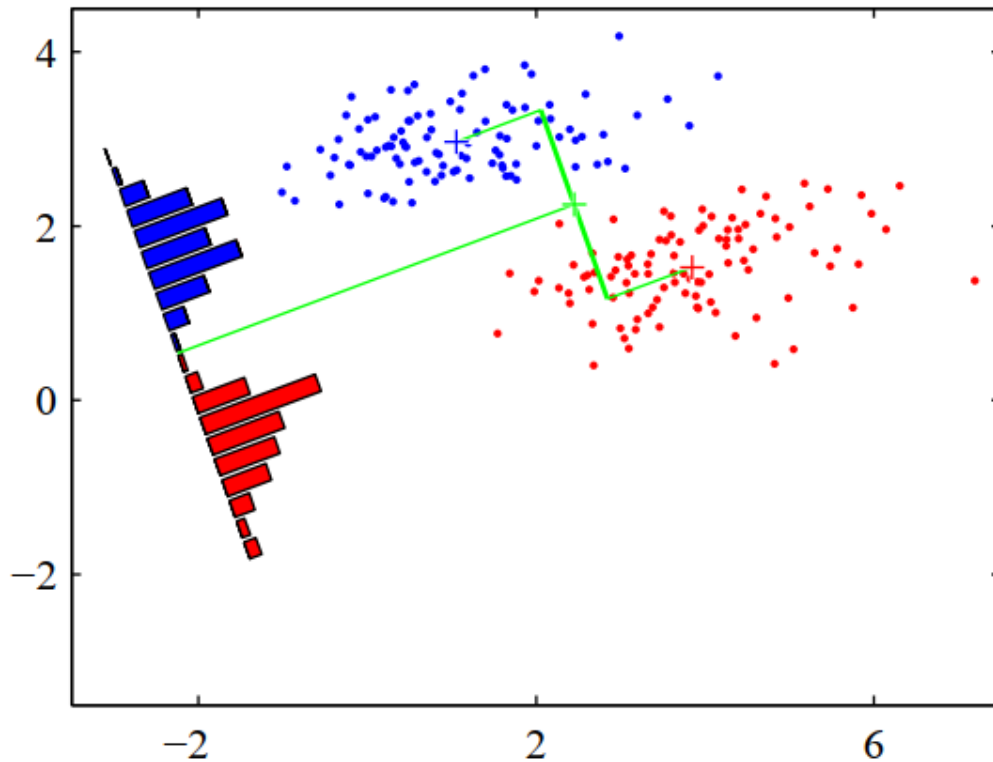


Figure 6.8: Proiezione secondo il Fisher Linear Discriminant. La dispersione intra-classe viene tenuta in conto.

Questa seconda figura invece illustra la soluzione del Fisher Linear Discriminant. L'introduzione della matrice di dispersione intra-classe S_W consente di penalizzare le direzioni caratterizzate da elevata varianza.

La direzione di proiezione risultante massimizza la separazione tra le classi tenendo conto sia della distanza tra le medie sia della compattezza delle distribuzioni.

Soluzione ottima

Massimizzando il criterio di Fisher, si ottiene che il vettore ottimo è proporzionale a:

$$w \propto S_W^{-1}(\mu_1 - \mu_2)$$

Questa direzione massimizza la separazione tra le classi dopo la proiezione.

Regola di classificazione

Una volta determinata la direzione w , un nuovo esempio x viene classificato proiettandolo su w e confrontando il risultato con una soglia θ :

$$\text{classe}(x) = \begin{cases} \mathcal{C}_1 & \text{se } w^T x \geq \theta \\ \mathcal{C}_2 & \text{altrimenti} \end{cases}$$

La soglia può essere scelta, ad esempio, come il punto medio tra le medie proiettate delle due classi.

6.7 Support Vector Machine

Le **Support Vector Machine (SVM)** sono una classe di algoritmi di apprendimento supervisionato per la classificazione (e regressione), basati sulla costruzione di un iperpiano di decisione ottimale.

Come nel caso del Perceptron e del Fisher Linear Discriminant, anche le SVM costruiscono una funzione discriminante lineare della forma:

$$g(x) = w^T x + b$$

Tuttavia, mentre il Perceptron si limita a correggere gli errori di classificazione e il metodo di Fisher massimizza una misura statistica di separazione, le SVM introducono un criterio geometrico esplicito basato sul margine.

Il margine di un iperpiano di decisione è definito come la distanza minima tra l'iperpiano stesso e i punti del dataset più vicini ad esso.

L'idea alla base delle SVM è che un iperpiano con margine massimo presenti migliori proprietà di generalizzazione, risultando meno sensibile al rumore e alle variazioni nei dati di training.

6.7.1 Formulazione formale

Si consideri un problema di classificazione binaria con un dataset di training

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N,$$

dove $x_i \in \mathbb{R}^d$ rappresenta il vettore delle feature e $y_i \in \{-1, +1\}$ l'etichetta di classe.

L'obiettivo delle Support Vector Machine è apprendere una funzione discriminante lineare della forma

$$f(x) = w^T x + b,$$

in grado di separare correttamente le due classi mediante un iperpiano di decisione definito dall'equazione

$$w^T x + b = 0.$$

Un punto x viene classificato in base al segno della funzione discriminante:

$$\text{classe}(x) = \text{sign}(w^T x + b).$$

La distanza geometrica di un punto x_i dall'iperpiano di decisione è data da

$$\frac{|w^T x_i + b|}{\|w\|}.$$

Il margine di classificazione è definito come la distanza minima tra l'iperpiano e i punti del dataset. Le Support Vector Machine cercano l'iperpiano che massimizza tale margine.

Nel caso di dati linearmente separabili, è possibile imporre i vincoli di separazione

$$y_i(w^T x_i + b) \geq 1, \quad \forall i = 1, \dots, N.$$

Questa scelta fissa la scala dei parametri e consente di definire in modo univoco il margine geometrico, che risulta pari a $\frac{2}{\|w\|}$.

Il problema di apprendimento di una SVM hard-margin può quindi essere formulato come il seguente problema di ottimizzazione convessa:

$$\begin{aligned} \min_{w,b} \quad & \frac{1}{2} \|w\|^2 \\ \text{soggetto a} \quad & y_i(w^T x_i + b) \geq 1, \quad \forall i. \end{aligned}$$

La funzione obiettivo consiste nella minimizzazione della norma del vettore dei pesi. Poiché il margine è inversamente proporzionale a $\|w\|$, minimizzare $\|w\|^2$ equivale a massimizzare il margine di separazione tra le classi.

I punti del dataset che soddisfano il vincolo con uguaglianza,

$$y_i(w^T x_i + b) = 1,$$

sono detti **support vectors**. Essi sono gli unici esempi che contribuiscono direttamente alla determinazione dell'iperpiano ottimale, mentre tutti gli altri punti non influenzano la soluzione finale una volta fissato il margine.

Interpretazione geometrica del margine

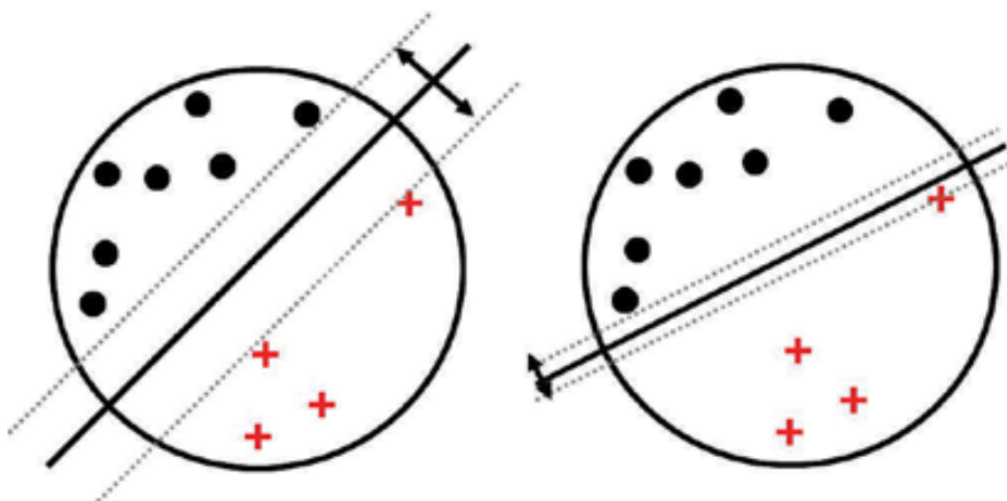


Figure 6.9: Confronto tra diversi iperpiani separatori: a sinistra un iperpiano con margine ridotto, a destra l'iperpiano a margine massimo individuato dalle SVM.

La figura mostra il principio fondamentale alla base delle Support Vector Machine. Nel pannello di sinistra è rappresentato un iperpiano che separa correttamente le due classi, ma presenta un margine ridotto. Piccole variazioni nei dati o la presenza di rumore potrebbero compromettere la corretta classificazione.

Nel pannello di destra è invece mostrato l'iperpiano a margine massimo. Questo iperpiano massimizza la distanza tra le due classi, risultando più stabile e robusto rispetto a perturbazioni nei dati.

6.7.2 Soft-Margin Support Vector Machine

Nel caso reale, i dati di training non sono quasi mai perfettamente linearmente separabili. La presenza di rumore, outlier o sovrapposizione tra le classi rende impossibile soddisfare rigidamente i vincoli dell'hard-margin SVM. Per questo motivo, le Support Vector Machine introducono il concetto di **soft margin**, che consente alcune violazioni controllate dei vincoli di separazione mantenendo il principio del margine massimo.

Nel soft-margin SVM vengono introdotte delle variabili di slack $\xi_i \geq 0$, una per ciascun esempio di training, che misurano il grado di violazione del vincolo di separazione. I vincoli diventano quindi:

$$y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \forall i = 1, \dots, N.$$

Il significato delle variabili di slack è il seguente: un esempio con $\xi_i = 0$ è correttamente classificato e si trova al di fuori del margine o esattamente sul margine; un esempio con $0 < \xi_i < 1$ è correttamente classificato ma si trova all'interno del margine; un esempio con $\xi_i \geq 1$ è classificato erroneamente.

Il problema di apprendimento del soft-margin SVM può essere formulato come il seguente problema di ottimizzazione convessa:

$$\begin{aligned} \min_{w, b, \xi} \quad & \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \xi_i \\ \text{soggetto a} \quad & y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \forall i, \\ & \xi_i \geq 0, \quad \forall i. \end{aligned}$$

La funzione obiettivo è composta da due termini in competizione tra loro. Il termine $\frac{1}{2} \|w\|^2$ tende a massimizzare il margine di separazione tra le classi, mentre il termine $C \sum_i \xi_i$ penalizza le violazioni dei vincoli di separazione. Il parametro $C > 0$ controlla il compromesso tra l'ampiezza del margine e l'accuratezza sui dati di training.

Valori elevati di C assegnano un peso maggiore agli errori di classificazione, portando a un margine più stretto ma a un numero ridotto di violazioni. Al contrario, valori piccoli di C consentono margini più ampi, tollerando un numero maggiore di errori sul training set, con potenziali benefici in termini di generalizzazione.

Anche nel caso soft-margin, i support vectors sono costituiti dagli esempi per cui il vincolo è attivo, ovvero quelli che soddisfano:

$$y_i(w^T x_i + b) \leq 1.$$

Questi includono sia i punti sul margine sia quelli che si trovano all'interno del margine o lo violano, e sono gli unici esempi che contribuiscono direttamente alla definizione dell'iperpiano ottimale.

Il soft-margin SVM generalizza l'hard-margin SVM, che può essere visto come un caso particolare ottenuto per $C \rightarrow \infty$. Grazie all'introduzione delle variabili di slack, le Support Vector Machine risultano applicabili a problemi reali caratterizzati da rumore e non perfetta separabilità delle classi.

6.7.3 Interpretazione geometrica

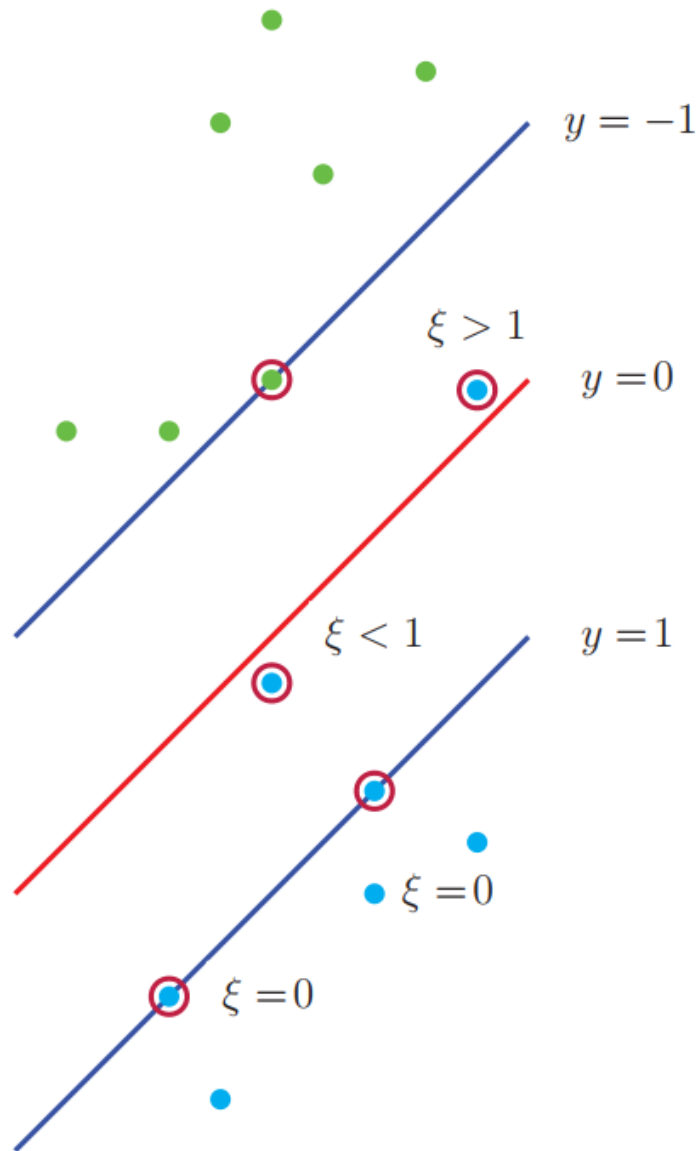


Figure 6.10: Interpretazione geometrica delle variabili di slack nel soft-margin SVM.

La figura illustra il significato geometrico delle variabili di slack ξ_i nel contesto del soft-margin SVM. Le tre rette rappresentano, rispettivamente, l'iperpiano di decisione $w^T x + b = 0$ e i due margini $w^T x + b = \pm 1$.

I punti appartenenti alle due classi sono distribuiti nello spazio delle feature e la loro posizione rispetto ai margini determina il valore della variabile di slack associata. Un punto per cui $0 < \xi_i \leq 1$ si trova all'interno del margine, ma sul lato corretto dell'iperpiano di decisione. In questo caso il punto è correttamente classificato, ma contribuisce comunque alla penalizzazione nella funzione obiettivo.

Infine, un punto per cui $\xi_i > 1$ si trova sul lato sbagliato dell'iperpiano di decisione ed è quindi classificato erroneamente. In particolare, quando $\xi_i = 1$ il punto giace esattamente sull'iperpiano di decisione, mentre per $\xi_i > 1$ la classificazione risulta errata. La figura evidenzia come il soft-margin SVM consenta di tollerare violazioni dei vincoli di

separazione, penalizzandole in modo controllato attraverso le variabili di slack, anziché imporre una separazione rigida come nel caso dell'hard-margin.

6.7.4 Basis Functions e separazione non lineare

Nei casi reali, i dati non sono spesso linearmente separabili nello spazio originale delle feature. In queste situazioni, una funzione discriminante lineare non è sufficiente a descrivere una frontiera di decisione efficace. L'idea alla base dell'uso delle **basis functions** consiste nel trasformare le istanze originali in uno spazio delle feature di dimensione maggiore, nel quale una separazione lineare diventa possibile.

Formalmente, si introduce una mappa non lineare:

$$\phi : \mathbb{R}^d \rightarrow \mathbb{R}^M,$$

che associa a ciascun input x un nuovo vettore di feature $\phi(x)$. La funzione discriminante assume quindi la forma:

$$f(x) = w^T \phi(x) + b.$$

In questo nuovo spazio delle feature, la funzione di decisione rimane lineare rispetto ai parametri w , ma risulta non lineare rispetto alle variabili originali. Questo consente di modellare frontiere di decisione complesse nello spazio di input.

Un primo esempio di basis functions è dato dalle **polynomial basis functions**. Nel caso monodimensionale, una trasformazione polinomiale di grado due può essere scritta come:

$$\phi(x) = (1, x, x^2).$$

In questo modo, una funzione lineare nello spazio delle feature corrisponde a una funzione quadratica nello spazio originale.

Un secondo esempio molto utilizzato è rappresentato dalle **Gaussian basis functions**, definite come:

$$\phi_j(x) = \exp\left(-\frac{\|x - \mu_j\|^2}{2\sigma^2}\right),$$

dove μ_j rappresenta il centro della funzione e σ ne controlla l'ampiezza. Queste funzioni consentono di modellare regioni locali nello spazio delle feature e risultano particolarmente efficaci in presenza di distribuzioni complesse.

Un'ulteriore classe di trasformazioni è data dalle **sigmoidal basis functions**, definite come:

$$\phi_j(x) = \tanh(a_j^T x + b_j),$$

che introducono una non linearità simile a quella dei neuroni artificiali.

L'uso esplicito delle basis functions presenta tuttavia un limite pratico: la dimensione dello spazio delle feature può diventare molto elevata o addirittura infinita. Le Support Vector Machine superano questo problema attraverso il **kernel trick**, che consente di calcolare prodotti scalari nello spazio delle feature senza esplicitare la mappa $\phi(x)$.

In questo modo, le SVM riescono a costruire classificatori non lineari mantenendo una formulazione computazionalmente efficiente e una solida base teorica.

6.8 RIEPILOGO

Support Vector Machine – Concetti chiave

Le **Support Vector Machine (SVM)** sono classificatori supervisionati basati sulla ricerca di un iperpiano di decisione che massimizza il **margin** tra le classi, migliorando la capacità di generalizzazione.

Hard-margin SVM Nel caso di dati linearmente separabili, il problema consiste nel minimizzare:

$$\frac{1}{2} \|w\|^2$$

soggetto ai vincoli:

$$y_i(w^T x_i + b) \geq 1.$$

I punti che soddisfano il vincolo con uguaglianza sono detti **support vectors** e determinano univocamente l'iperpiano ottimale.

Soft-margin SVM Per dati non perfettamente separabili, vengono introdotte le variabili di slack $\xi_i \geq 0$ e il problema diventa:

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_i \xi_i, \quad y_i(w^T x_i + b) \geq 1 - \xi_i.$$

Il parametro C regola il compromesso tra ampiezza del margine e penalizzazione degli errori.

Interpretazione delle variabili di slack

- $\xi_i = 0$: punto correttamente classificato e fuori dal margine;
- $0 < \xi_i \leq 1$: punto dentro il margine ma sul lato corretto;
- $\xi_i > 1$: punto misclassificato.

Separazione non lineare e basis functions Quando i dati non sono linearmente separabili nello spazio originale, è possibile applicare una trasformazione non lineare:

$$\phi : \mathbb{R}^d \rightarrow \mathbb{R}^M, \quad f(x) = w^T \phi(x) + b.$$

Esempi di basis functions includono trasformazioni polinomiali, gaussiane e sigmoidali.

In sintesi, le SVM combinano un criterio geometrico solido, una formulazione di ottimizzazione convessa e la possibilità di estendere il modello a separazioni non lineari, rendendole uno dei metodi di riferimento nel machine learning classico.

Chapter 7

Linear Models for Regression

L'obiettivo della regressione è apprendere una funzione

$$f : \mathcal{X} \subseteq \mathbb{R}^d \rightarrow \mathbb{R}$$

a partire da un insieme di dati

$$\mathcal{D} = \{(x_n, t_n)\}_{n=1}^N,$$

dove x_n rappresenta un vettore di input e t_n il corrispondente valore target reale. I modelli lineari per la regressione assumono che l'output possa essere espresso come combinazione lineare di parametri, eventualmente applicata a trasformazioni non lineari degli input.

7.1 Modelli lineari per funzioni lineari

Nel caso più semplice, il modello è lineare rispetto alle variabili di input:

$$y(x; w) = w_0 + w_1 x_1 + \dots + w_d x_d = w^T x,$$

dove il vettore degli input è esteso con un termine costante

$$x = [1 \quad x_1 \quad \dots \quad x_d]^T,$$

e il vettore dei parametri è

$$w = [w_0 \quad w_1 \quad \dots \quad w_d]^T.$$

Questo modello consente, ad esempio, di approssimare una retta nel caso bidimensionale, come nel classico problema di *line fitting*.

Nel caso di una singola variabile di input, il modello lineare assume la forma

$$y(x; w) = w_0 + w_1 x_1,$$

che corrisponde geometricamente a una retta nel piano. Come illustrato in Figura 7.1, i dati osservati non giacciono esattamente sulla retta a causa della presenza di rumore sulle misure. Il problema della regressione consiste quindi nel determinare i parametri w_0 e w_1 tali da approssimare nel modo migliore possibile i valori target.

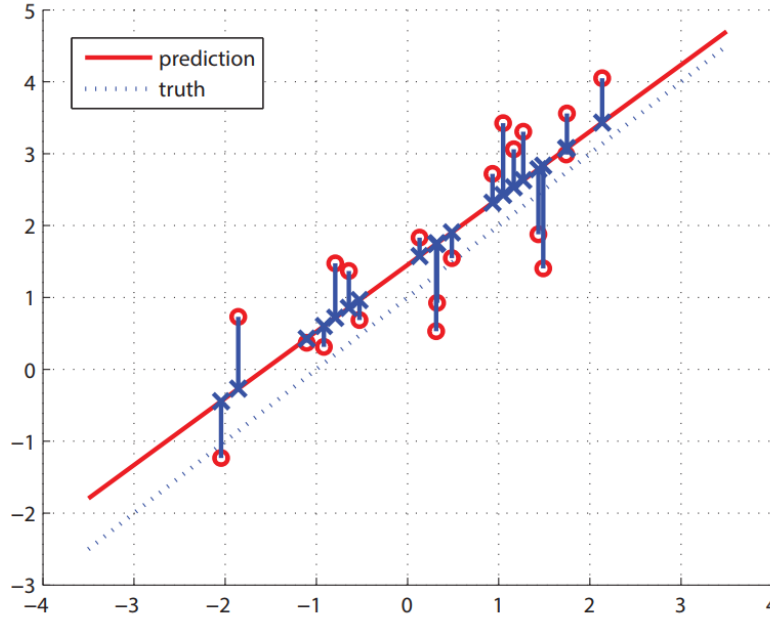


Figure 7.1: Esempio di regressione lineare bidimensionale. I punti blu rappresentano i valori target osservati, affetti da rumore, mentre la retta rossa corrisponde alla funzione di predizione $y(x; w) = w_0 + w_1 x_1$ appresa dal modello. I segmenti verticali indicano i residui, ovvero gli errori $t_n - y(x_n; w)$ che vengono minimizzati nel criterio dei minimi quadrati.

Gli scarti verticali tra i punti osservati e la retta di predizione rappresentano i residui del modello. La stima dei parametri tramite il metodo dei minimi quadrati equivale a minimizzare la somma dei quadrati di tali residui, fornendo la soluzione ottima in senso statistico sotto l'ipotesi di rumore gaussiano.

7.2 Modelli lineari con basis functions

Per aumentare la flessibilità del modello senza rinunciare alla linearità nei parametri, si introducono le *basis functions*. Il modello assume la forma

$$y(x; w) = \sum_{j=0}^M w_j \phi_j(x) = w^T \phi(x),$$

dove $\phi_j(x)$ sono funzioni (in generale non lineari) degli input, con $\phi_0(x) = 1$. Nonostante la possibile non linearità rispetto a x , il modello rimane lineare rispetto ai parametri w .

Un caso particolarmente rilevante di modelli lineari con basis functions è rappresentato dall'uso di polinomi. In questo contesto, le funzioni base sono definite come

$$\phi_j(x) = x^j,$$

e il modello assume la forma

$$y(x; w) = \sum_{j=0}^M w_j x^j.$$

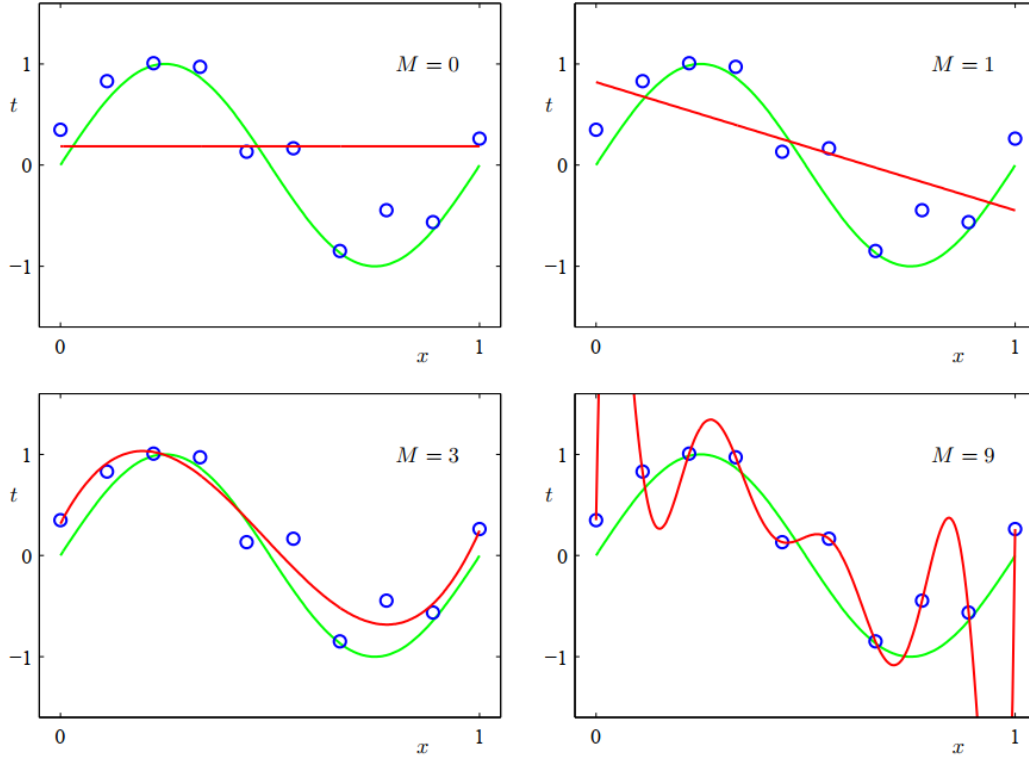


Figure 7.2: Esempio di regressione mediante basis functions polinomiali di diverso grado M . I punti blu rappresentano i dati osservati, la curva verde indica la funzione target sottostante, mentre la curva rossa corrisponde alla funzione appresa dal modello. All'aumentare del grado del polinomio cresce la flessibilità del modello, passando da situazioni di underfitting ($M = 0, 1$) a una buona approssimazione ($M = 3$), fino a fenomeni di overfitting ($M = 9$).

La Figura 7.3 mostra come la scelta del grado M influenzi in modo critico il comportamento del modello. Per valori piccoli di M , il modello risulta eccessivamente rigido e non è in grado di catturare la struttura dei dati, producendo un'elevata distorsione (bias). Al crescere di M , la capacità di approssimazione aumenta e il modello riesce a seguire più fedelmente l'andamento della funzione target.

Tuttavia, per valori elevati di M , il modello tende ad adattarsi anche al rumore presente nei dati di training, manifestando oscillazioni non giustificate e una scarsa capacità di generalizzazione. Questo fenomeno prende il nome di overfitting e motiva la necessità di introdurre tecniche di controllo della complessità, come la regolarizzazione, discusse nelle sezioni successive.

Altri esempi di basis functions includono funzioni radiali (RBF), sigmoidi e funzioni tanh, che permettono di modellare strutture più complesse nei dati.

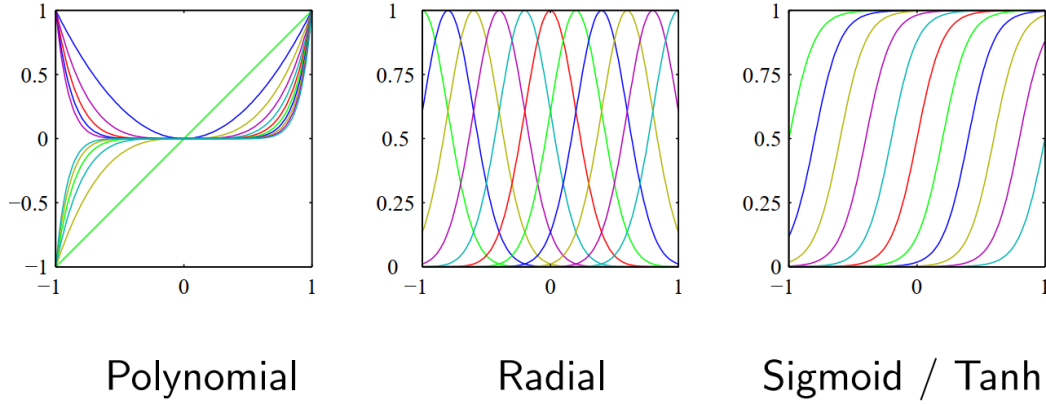


Figure 7.3: Visualizzazione grafica di alcuni esempi di basis function.

7.3 Stima dei parametri: massima verosimiglianza e minimi quadrati

Si assume che il valore target sia affetto da rumore additivo gaussiano:

$$t = y(x; w) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \beta^{-1}),$$

da cui segue:

$$P(t|x, w, \beta) = \mathcal{N}(t|y(x; w), \beta^{-1}).$$

Assumendo osservazioni indipendenti e identicamente distribuite, la massimizzazione della verosimiglianza equivale alla minimizzazione dell'errore quadratico:

$$E_D(w) = \frac{1}{2} \sum_{n=1}^N (t_n - w^T \phi(x_n))^2.$$

Introducendo la matrice di progetto

$$\Phi = \begin{bmatrix} \phi_0(x_1) & \dots & \phi_M(x_1) \\ \vdots & & \vdots \\ \phi_0(x_N) & \dots & \phi_M(x_N) \end{bmatrix},$$

la soluzione in forma chiusa è:

$$w_{ML} = (\Phi^T \Phi)^{-1} \Phi^T t,$$

che coincide con la soluzione ai minimi quadrati ottenuta tramite pseudo-inversa.

7.4 Apprendimento sequenziale

In alternativa alla soluzione analitica, i parametri possono essere appresi in modo incrementale tramite *stochastic gradient descent*:

$$\hat{w} \leftarrow \hat{w} - \eta \nabla E_S,$$

dove η è il learning rate e ∇E_S è una stima del gradiente calcolata su un sottoinsieme casuale dei dati. L'algoritmo converge per valori sufficientemente piccoli di η .

7.4.1 Regularizzazione

Per contrastare l'overfitting si introduce un termine di penalizzazione sui parametri:

$$\arg \min_w E_D(w) + \lambda E_W(w),$$

con $\lambda > 0$ fattore di regolarizzazione.

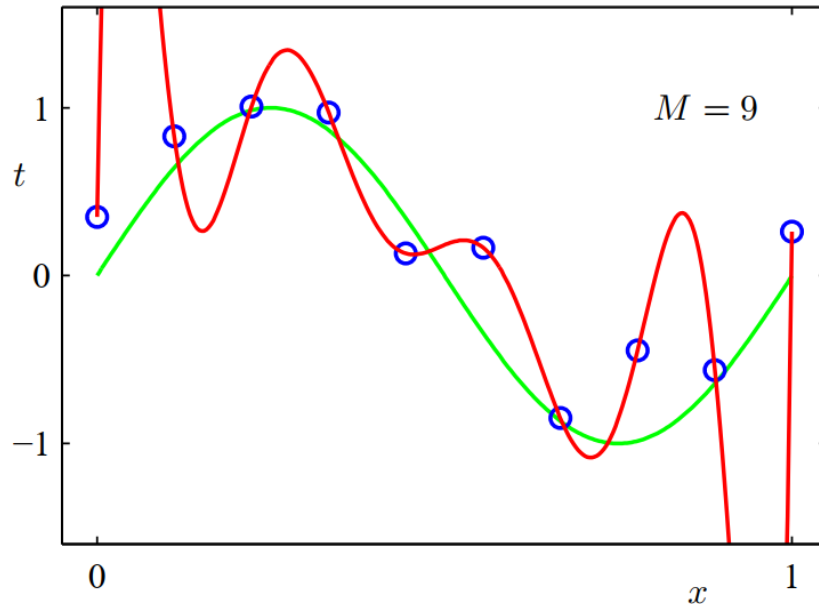


Figure 7.4: Effetto della regolarizzazione su un modello polinomiale di grado elevato ($M = 9$). In alto è mostrata la soluzione non regolarizzata, caratterizzata da forti oscillazioni e overfitting.

In assenza di penalizzazione sui parametri, un modello polinomiale di grado elevato tende ad adattarsi in modo eccessivo ai dati di training, producendo oscillazioni non giustificate e una scarsa capacità di generalizzazione.

Introducendo un termine di regolarizzazione del tipo

$$E_W(w) = \frac{1}{2} w^T w,$$

la funzione obiettivo diventa

$$E(w) = E_D(w) + \lambda E_W(w),$$

dove il parametro λ controlla il compromesso tra fedeltà ai dati e complessità del modello. Per valori piccoli di λ , la penalizzazione è debole e il comportamento del modello è simile a quello non regolarizzato. All'aumentare di λ , i coefficienti vengono progressivamente ridotti, limitando le oscillazioni della funzione appresa.

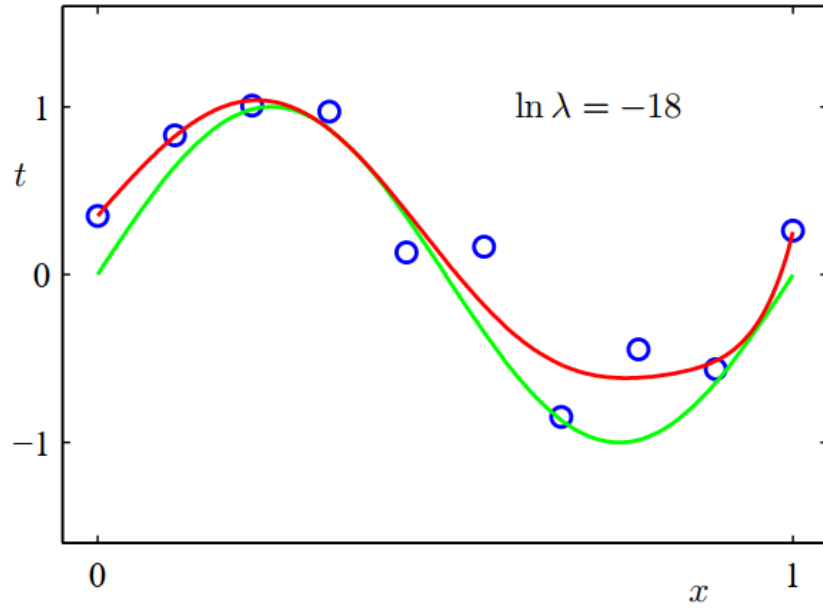


Figure 7.5: Una regolarizzazione debole ($\ln \lambda = -18$), che consente una buona approssimazione della funzione target.

Un valore eccessivamente grande di λ porta tuttavia a un modello troppo semplice, incapace di catturare la struttura dei dati, evidenziando un fenomeno di underfitting. La scelta del parametro di regolarizzazione risulta quindi cruciale per ottenere buone prestazioni di generalizzazione.

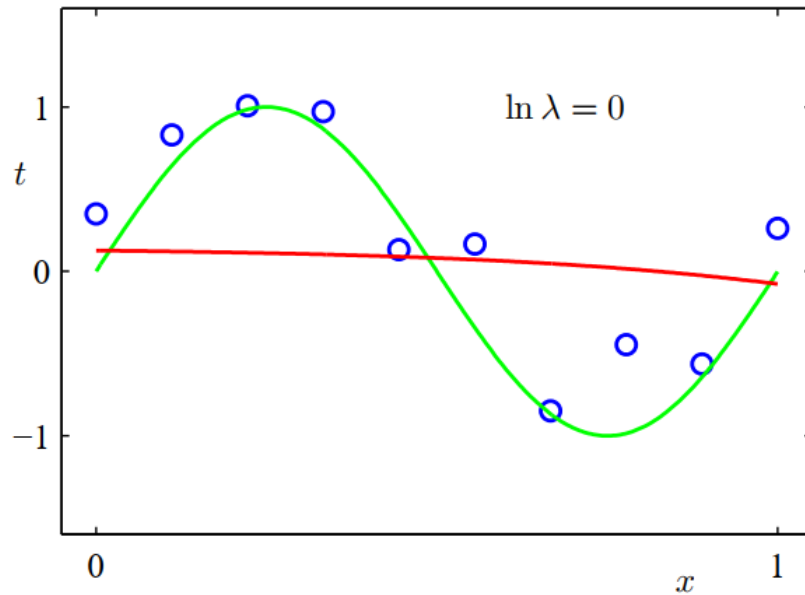


Figure 7.6: Un esempio caso di regolarizzazione forte ($\ln \lambda = 0$), in cui il modello diventa eccessivamente rigido e manifesta underfitting.

7.4.2 Regressione con output multipli

Nel caso di output vettoriali $y \in \mathbb{R}^K$, il modello diventa:

$$y(x; W) = W^T \phi(x),$$

dove W è una matrice dei parametri. Sotto le stesse ipotesi gaussiane, la stima di massima verosimiglianza è:

$$W_{ML} = (\Phi^T \Phi)^{-1} \Phi^T T.$$

RIEPILOGO

Linear Models for Regression – Concetti chiave

I modelli lineari per la regressione mirano ad apprendere una funzione

$$f : \mathbb{R}^d \rightarrow \mathbb{R}$$

a partire da un insieme di dati etichettati, assumendo una dipendenza lineare dai parametri.

Modello lineare

$$y(x; w) = w^T x$$

rappresenta il caso più semplice di regressione lineare.

Linear Basis Function Models Introducendo funzioni base $\phi_j(x)$, il modello diventa:

$$y(x; w) = \sum_{j=0}^M w_j \phi_j(x) = w^T \phi(x),$$

consentendo di approssimare relazioni non lineari pur mantenendo la linearità nei parametri.

Stima dei parametri Assumendo rumore gaussiano additivo, la massima verosimiglianza conduce alla minimizzazione dell'errore quadratico:

$$E_D(w) = \frac{1}{2} \sum_{n=1}^N (t_n - w^T \phi(x_n))^2,$$

con soluzione in forma chiusa:

$$w_{ML} = (\Phi^T \Phi)^{-1} \Phi^T t.$$

Regolarizzazione Per controllare l'overfitting si introduce una penalizzazione sui parametri:

$$\arg \min_w E_D(w) + \lambda \frac{1}{2} w^T w,$$

dove λ regola il compromesso tra accuratezza sui dati e complessità del modello.

In sintesi, i modelli lineari per la regressione costituiscono un quadro teorico solido e computazionalmente efficiente, ponendo le basi per tecniche più avanzate di apprendimento supervisionato.

Chapter 8

Kernel Methods

I metodi kernel rappresentano un'estensione fondamentale dei modelli lineari, consentendo di apprendere funzioni non lineari mantenendo una formulazione matematica elegante e computazionalmente trattabile. L'idea centrale è quella di esprimere il modello esclusivamente in termini di prodotti scalari tra campioni di input, permettendo così di sostituire tali prodotti con funzioni di similarità più generali.

8.1 Dai modelli lineari alla formulazione duale

Consideriamo un modello lineare per la regressione:

$$y(x; w) = w^T x,$$

e una funzione di costo regolarizzata:

$$E(w) = \frac{1}{2} \sum_{n=1}^N (w^T x_n - t_n)^2 + \lambda \|w\|^2.$$

La soluzione ottima si ottiene imponendo $\nabla E(w) = 0$, da cui segue che il vettore dei pesi può essere espresso come combinazione lineare dei dati di training:

$$w^* = \sum_{n=1}^N \alpha_n x_n.$$

Sostituendo nella funzione di predizione, si ottiene:

$$y(x) = \sum_{n=1}^N \alpha_n x_n^T x.$$

Questa espressione mostra che la predizione non dipende più esplicitamente dal vettore dei pesi w , ma solo da **quanto l'input x è simile ai dati di training**. Ogni termine $x_n^T x$ misura una similarità, e i coefficienti α_n ne pesano l'importanza.

8.2 Gram matrix e formulazione kernelizzata

Definendo la matrice di Gram:

$$K = XX^T, \quad K_{nm} = x_n^T x_m,$$

la soluzione diventa:

$$\alpha = (K + \lambda I_N)^{-1} t, \quad y(x) = \sum_{n=1}^N \alpha_n x_n^T x.$$

Questa formulazione è detta *duale* e dipende esclusivamente dai prodotti scalari tra dati.

Il modello non “vede” più le coordinate degli input, ma solo **le relazioni reciproche** tra i punti del dataset. Questo apre la strada a una generalizzazione potente: sostituire il prodotto scalare con una misura di similarità più sofisticata.

Esempio

Dato un insieme di dati di training $\{x_1, \dots, x_N\}$, la Gram matrix associata a un kernel k è definita come

$$K_{nm} = k(x_n, x_m).$$

Consideriamo, ad esempio, tre punti bidimensionali:

$$x_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad x_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad x_3 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Utilizzando il kernel lineare $k(x, x') = x^T x'$, la Gram matrix risulta:

$$K = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{bmatrix}.$$

Sostituendo il kernel lineare con un kernel gaussiano

$$k(x, x') = \exp(-\beta \|x - x'\|^2),$$

si ottiene una Gram matrix con la stessa struttura ma valori differenti, che riflettono una diversa nozione di similarità tra i dati.

La Gram matrix codifica quindi tutte le relazioni tra gli esempi del dataset ed è l'unico oggetto necessario per l'apprendimento nei modelli kernelizzati.

8.3 Kernel functions

Una *kernel function* è una funzione:

$$k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R},$$

che misura la similarità tra due input. Il kernel lineare è un caso particolare:

$$k(x, x') = x^T x'.$$

Sostituendo il prodotto scalare con un kernel generico, il modello diventa:

$$y(x) = \sum_{n=1}^N \alpha_n k(x_n, x),$$

con

$$\alpha = (K + \lambda I_N)^{-1} t, \quad K_{nm} = k(x_n, x_m).$$

Il kernel può essere visto come un confronto intelligente tra esempi: invece di chiedersi “quanto sono vicini in termini di coordinate?”, il modello si chiede “quanto si assomigliano secondo un certo criterio?”.

8.3.1 Kernel trick

Il *kernel trick* consiste nell’osservare che, se un algoritmo dipende dagli input **solo tramite prodotti scalari**, questi possono essere sostituiti da kernel:

$$x^T x' \longrightarrow k(x, x').$$

In molti casi esiste una mappa implicita $\phi(x)$ tale che:

$$k(x, x') = \phi(x)^T \phi(x'),$$

senza che sia necessario conoscere o calcolare esplicitamente $\phi(x)$.

È come lavorare in uno spazio di dimensione molto alta (o infinita) senza mai entrarci davvero. Il kernel fornisce direttamente il risultato finale, evitando il costo computazionale della trasformazione esplicita.

8.3.2 Esempio: applicazione concreta del kernel trick

Consideriamo un semplice dataset di training composto da tre esempi monodimensionali:

$$\mathcal{D} = \{(x_1, t_1), (x_2, t_2), (x_3, t_3)\}, \quad x_1 = 0, \quad x_2 = 1, \quad x_3 = 2, \quad t = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}.$$

Supponiamo che un modello lineare nello spazio originale non sia sufficiente e scegliamo un kernel gaussiano (RBF):

$$k(x, x') = \exp(-\beta(x - x')^2),$$

con $\beta = 1$.

Costruzione della Gram matrix

La Gram matrix K è definita come:

$$K_{nm} = k(x_n, x_m).$$

Calcolando tutte le similarità tra i punti del training, otteniamo:

$$K = \begin{bmatrix} k(0,0) & k(0,1) & k(0,2) \\ k(1,0) & k(1,1) & k(1,2) \\ k(2,0) & k(2,1) & k(2,2) \end{bmatrix} = \begin{bmatrix} 1 & e^{-1} & e^{-4} \\ e^{-1} & 1 & e^{-1} \\ e^{-4} & e^{-1} & 1 \end{bmatrix}.$$

Questa matrice rappresenta completamente il dataset dal punto di vista del modello kernelizzato.

Apprendimento

Nel caso della regressione kernelizzata con regolarizzazione, i coefficienti α si ottengono risolvendo:

$$\alpha = (K + \lambda I)^{-1}t,$$

dove $\lambda > 0$ è il parametro di regolarizzazione. Il modello non utilizza esplicitamente i dati x_n , ma solo la matrice K e il vettore dei target t .

Predizione su un nuovo punto

Dato un nuovo punto di test $x_* = 0.5$, la predizione è:

$$y(x_*) = \sum_{n=1}^3 \alpha_n k(x_n, x_*).$$

In particolare:

$$k(0, 0.5) = e^{-0.25}, \quad k(1, 0.5) = e^{-0.25}, \quad k(2, 0.5) = e^{-2.25}.$$

La predizione dipende quindi esclusivamente dalla similarità tra x_* e i punti di training, senza mai calcolare esplicitamente una trasformazione non lineare dello spazio degli input.

8.4 Famiglie di kernel

Tra le famiglie di kernel più utilizzate si trovano:

Lineare:	$k(x, x') = x^T x'$
Polinomiale:	$k(x, x') = (\beta x^T x' + \gamma)^d$
RBF (Gaussiano):	$k(x, x') = \exp(-\beta \ x - x'\ ^2)$
Sigmoide:	$k(x, x') = \tanh(\beta x^T x' + \gamma).$

Ogni kernel introduce una diversa nozione di vicinanza: il kernel RBF, ad esempio, assegna un'influenza locale ai punti, mentre il kernel polinomiale cattura interazioni globali di grado crescente.

8.5 Kernelized SVM

Nel caso delle Support Vector Machine, la soluzione ottima ha la forma:

$$y(x) = \text{sign} \left(w_0 + \sum_{n=1}^N \alpha_n k(x_n, x) \right),$$

dove solo i *support vectors* hanno coefficienti $\alpha_n \neq 0$.

La decisione finale dipende solo da pochi esempi critici. Il kernel consente di costruire frontiere di decisione non lineari nello spazio originale, pur mantenendo una formulazione lineare nello spazio implicito.

8.5.1 Kernelized SVM per la regressione

Nel caso regressivo, si introduce la funzione di perdita ε -insensitive e un tubo di tolleranza attorno alla funzione appresa. Solo i punti che violano tale tubo contribuiscono alla soluzione:

$$y(x) = \sum_{n=1}^N (\hat{\alpha}_n - \hat{\alpha}'_n) k(x_n, x) + w_0.$$

Il modello ignora piccoli errori e si concentra solo sui punti che forniscono informazione rilevante, ottenendo soluzioni sparse e robuste al rumore.

RIEPILOGO

Kernel Methods – Concetti chiave

I metodi kernel permettono di estendere i modelli lineari a funzioni non lineari esprimendo il modello esclusivamente in termini di prodotti scalari tra dati.

Idea fondamentale

$$x^T x' \rightarrow k(x, x')$$

senza calcolare esplicitamente la trasformazione dello spazio degli input.

Kernel trick Consente di lavorare implicitamente in spazi di dimensione elevata o infinita mantenendo una complessità computazionale gestibile.

Kernelized models

$$y(x) = \sum_{n=1}^N \alpha_n k(x_n, x),$$

con soluzione dipendente solo dalla matrice di Gram.

Applicazioni principali Le SVM kernelizzate rappresentano uno dei metodi più efficaci per classificazione e regressione non lineare, pur richiedendo una attenta selezione degli iperparametri.

Chapter 9

Instance Based

Gli **instance-based methods** sono modelli *non-parametrici* che non costruiscono esplicitamente un modello globale durante la fase di training, ma *memorizzano l'intero dataset* e rimandano il processo di apprendimento alla fase di predizione (*lazy learning*).

Nei modelli parametrici il numero di parametri è fissato e indipendente dalla dimensione del dataset. Nei modelli non parametrici, viceversa, il numero di parametri cresce con il numero di dati disponibili.

K-Nearest Neighbors (K-NN)

Si consideri un problema di classificazione:

$$f : \mathcal{X} \rightarrow \mathcal{C}, \quad \mathcal{D} = \{(x_n, t_n)\}_{n=1}^N$$

Dato un nuovo campione x , l'algoritmo K-NN opera come segue:

1. si individuano i K campioni più vicini a x nel dataset $\mathcal{D}$;
2. si assegna a x la classe più frequente tra i suoi K vicini.

La probabilità della classe c dato x è stimata come:

$$p(c \mid x, \mathcal{D}, K) = \frac{1}{K} \sum_{x_n \in \mathcal{N}_K(x, \mathcal{D})} \mathbb{I}(t_n = c)$$

dove $\mathcal{N}_K(x, \mathcal{D})$ è l'insieme dei K nearest neighbors di x e $\mathbb{I}(\cdot)$ è la funzione indicatrice.

Osservazioni

- è necessario memorizzare l'intero dataset;
- il metodo dipende fortemente dalla funzione di distanza scelta;
- per $K = 1$ la frontiera di decisione induce una tassellazione di Voronoi;
- aumentando K si ottengono regioni di decisione più lisce, riducendo l'overfitting.

9.1 K-Nearest Neighbors: interpretazione geometrica

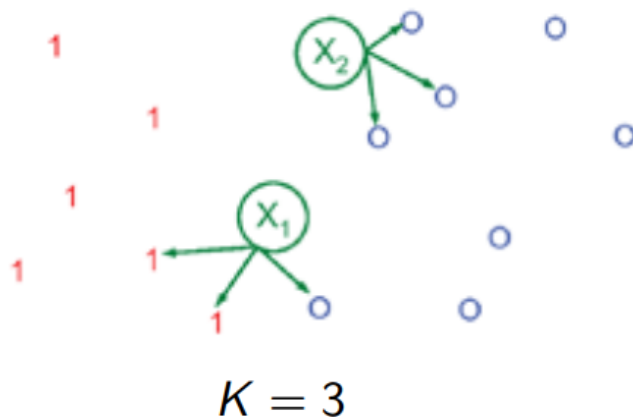
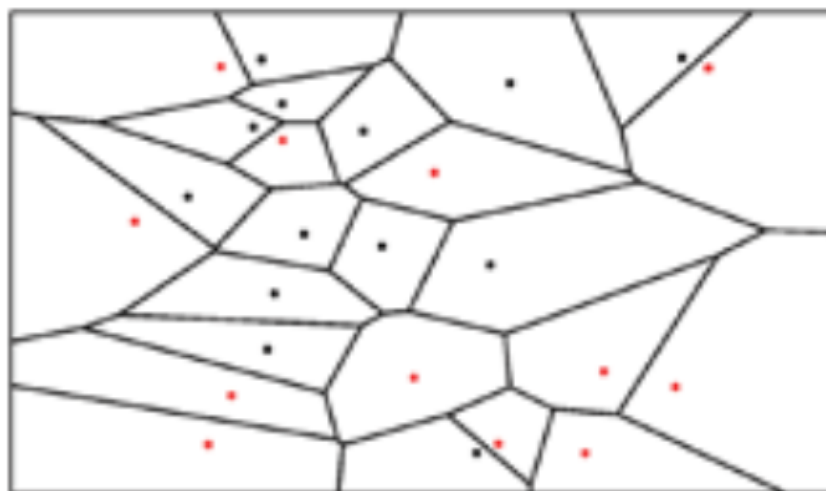


Figure 9.1: Classificazione K-NN con $K = 3$. Per ciascun punto di query (x_1, x_2) vengono selezionati i tre campioni più vicini nel dataset; la classe assegnata è quella di maggioranza tra i vicini. La decisione è locale e dipende fortemente dalla distribuzione dei punti nello spazio delle feature.

Nel caso $K > 1$, la classificazione si basa su un voto di maggioranza locale, che introduce un effetto di regolarizzazione: all'aumentare di K , le regioni di decisione diventano più lisce e la sensibilità al rumore diminuisce, riducendo il rischio di overfitting.



Voronoi tessellation for $K = 1$

Figure 9.2: Tassellazione di Voronoi indotta da un classificatore 1-NN ($K = 1$). Ogni punto del dataset genera una regione dello spazio contenente tutti i punti più vicini a esso rispetto a qualunque altro campione. La frontiera di decisione risulta altamente irregolare e perfettamente adattata ai dati di training.

Per $K = 1$, il classificatore assegna a ogni punto la classe del nearest neighbor più vicino, inducendo una tassellazione di Voronoi dello spazio delle feature. Questo rappresenta il caso limite di massima varianza, in cui il modello è estremamente sensibile al rumore e tende all'overfitting.

9.1.1 Effetto del parametro K sulle regioni di decisione

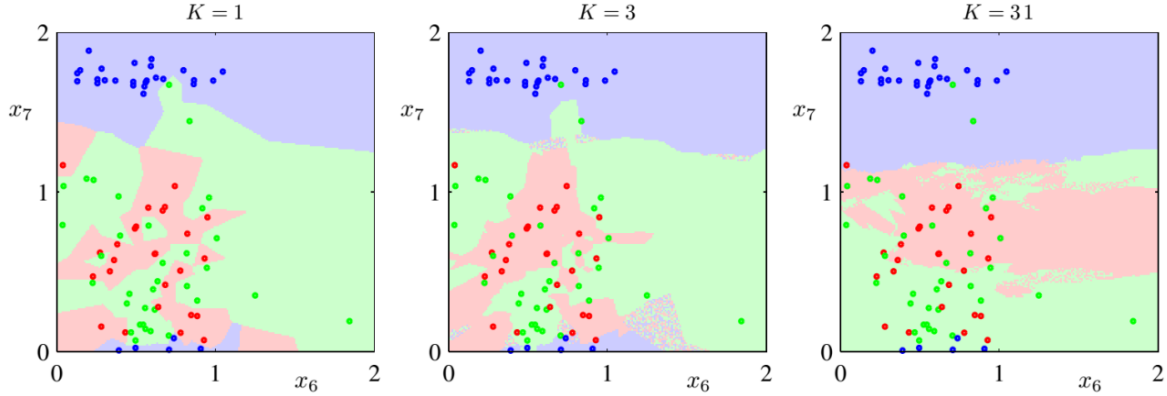


Figure 9.3: Effetto dell'aumento del parametro K nel classificatore K -NN. Da sinistra a destra: $K = 1$, $K = 3$, $K = 31$. All'aumentare di K , le regioni di decisione diventano progressivamente più lisce, riducendo la complessità del modello e l'overfitting.

La Figura 9.3 mostra come il valore del parametro K influenzi direttamente la forma delle regioni di decisione nel classificatore K -NN.

Nel caso $K = 1$, la classificazione è determinata esclusivamente dal nearest neighbor più vicino. Le regioni di decisione risultano altamente irregolari e frammentate, adattandosi in modo quasi perfetto ai dati di training. Questo comportamento corrisponde a un modello ad alta varianza, fortemente sensibile al rumore e incline all'overfitting.

Per valori intermedi di K (ad esempio $K = 3$), la decisione si basa su una maggioranza locale. Le regioni di decisione diventano più regolari, mantenendo comunque una buona capacità di catturare la struttura locale dei dati.

Per valori elevati di K (ad esempio $K = 31$), la classificazione tiene conto di un intorno molto ampio del punto di query. Le regioni di decisione risultano notevolmente più lisce e meno sensibili alle fluttuazioni locali del dataset. In questo regime, la varianza del modello si riduce, ma aumenta il bias, con il rischio di underfitting.

In sintesi, il parametro K controlla il compromesso bias-variance del classificatore K -NN:

$$K \uparrow \Rightarrow \text{decision boundaries più lisce} \Rightarrow \text{minore overfitting}$$

9.2 Kernelized Nearest Neighbors

Nel classificatore K -NN, la selezione dei nearest neighbors dipende dalla funzione di distanza utilizzata nello spazio delle feature. Nel caso della distanza euclidea al quadrato tra due punti x e x_n :

$$\|x - x_n\|^2 = (x - x_n)^T(x - x_n) = x^T x + x_n^T x_n - 2x^T x_n$$

Si osserva che la distanza può essere espressa esclusivamente in termini di prodotti scalari. Questo consente di applicare il *kernel trick*, sostituendo il prodotto scalare con una funzione kernel:

$$x^T x_n \longrightarrow k(x, x_n)$$

In questo modo, la distanza viene calcolata implicitamente in uno spazio delle feature ad alta (o infinita) dimensionalità:

$$\|\phi(x) - \phi(x_n)\|^2 = k(x, x) + k(x_n, x_n) - 2k(x, x_n)$$

dove $\phi(\cdot)$ è una mappatura implicita associata al kernel $k(\cdot, \cdot)$.

Interpretazione Il kernelized K-NN non modifica la regola di classificazione, ma ridefinisce la nozione di vicinanza tra i punti. Due campioni risultano vicini se hanno un'elevata similarità nel feature space indotto dal kernel, anche se non sono vicini nello spazio originale.

Conseguenze

- è possibile catturare strutture non lineari nei dati;
- la frontiera di decisione può diventare non lineare anche nello spazio originale;
- la scelta del kernel influenza direttamente la geometria dello spazio delle feature e quindi i nearest neighbors.

Il kernelized K-NN rappresenta quindi un'estensione non lineare del classificatore K-NN standard, analoga concettualmente all'uso dei kernel nelle Support Vector Machines.

9.3 Locally Weighted Regression

La *Locally Weighted Regression* (LWR) è un metodo di apprendimento *instance-based* e *non parametrico*, in cui la funzione di regressione non viene appresa globalmente una volta per tutte, ma viene stimata *localmente* in corrispondenza di ciascun punto di query.

A differenza della regressione lineare classica, che apprende un unico modello valido su tutto il dominio, la LWR costruisce un modello locale utilizzando principalmente le istanze di training vicine al punto x_{new} per cui si vuole effettuare la predizione.

Idea di base

Dato un punto di query x_{new} , si assegnano pesi diversi agli esempi di training (x_i, y_i) in funzione della loro distanza da x_{new} . Gli esempi più vicini contribuiscono maggiormente alla stima, mentre quelli lontani hanno un'influenza trascurabile.

La regressione viene quindi risolta come un problema di *least squares pesati*, che cambia per ogni punto di query.

Formulazione

Si consideri un modello lineare locale:

$$\hat{y}(x) = \mathbf{w}^\top \mathbf{x}$$

I parametri $\mathbf{w}$ sono stimati minimizzando la funzione di costo:

$$J(\mathbf{w}) = \sum_{i=1}^n w_i(x_{\text{new}}) (y_i - \mathbf{w}^\top \mathbf{x}_i)^2$$

dove $w_i(x_{\text{new}})$ è una funzione di peso che dipende dalla distanza tra x_i e il punto di query x_{new} .

Una scelta tipica è una funzione kernel gaussiana:

$$w_i(x_{\text{new}}) = \exp\left(-\frac{\|x_i - x_{\text{new}}\|^2}{2\tau^2}\right)$$

dove τ è il parametro di bandwidth che controlla l'ampiezza della regione locale.

Interpretazione geometrica

Per ogni punto di query, la LWR seleziona una regione locale del dataset e adatta una retta (o un iperpiano) che approssima bene i dati in quella zona, ignorando la struttura globale della funzione.

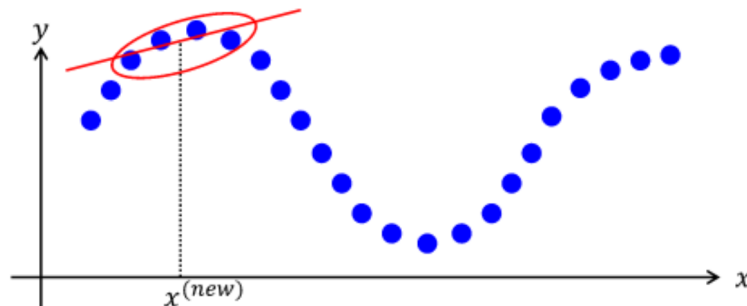


Figure 9.4: Interpretazione geometrica della Locally Weighted Regression. Per il punto di query x_{new} viene costruito un modello locale utilizzando principalmente i punti vicini (ellisse rossa), ottenendo una retta di regressione locale che approssima la funzione solo in quella regione.

La figura mostra come, in presenza di una funzione non lineare globale, la LWR riesca a catturare correttamente l'andamento locale del dataset, a differenza di un modello lineare globale che risulterebbe inadeguato.

Caratteristiche principali

- non esiste un modello globale esplicito;
- l'addestramento è differito al momento della predizione;

- elevata flessibilità e capacità di adattamento locale;
- costo computazionale elevato in fase di test;
- forte dipendenza dal parametro di bandwidth τ .

La Locally Weighted Regression può essere vista come una generalizzazione continua dei metodi *k-nearest neighbors* per la regressione, in cui l'influenza degli esempi non è binaria (vicino/lontano), ma decresce progressivamente con la distanza.

RIEPILOGO

Instance-Based Learning – Concetti chiave

I metodi di *Instance-Based Learning* non apprendono un modello globale esplicito durante la fase di training, ma memorizzano il dataset e rimandano l'apprendimento al momento della predizione.

Idea fondamentale

La predizione per un punto di query x_{new} viene ottenuta utilizzando direttamente gli esempi di training più simili, secondo una misura di distanza definita nello spazio delle feature.

K-Nearest Neighbors (KNN)

Nel metodo KNN la stima dell'output è ottenuta aggregando le etichette dei k vicini più prossimi a x_{new} . Il parametro k controlla il compromesso tra:

- modelli molto flessibili e sensibili al rumore (k piccolo);
- modelli più stabili ma meno adattivi (k grande).

Kernelized Nearest Neighbors

L'estensione kernelizzata dei KNN consente di definire la nozione di similarità tramite funzioni kernel, permettendo di catturare strutture non lineari nei dati senza esplicitare la mappatura nello spazio delle feature.

Locally Weighted Regression

La Locally Weighted Regression costruisce, per ogni punto di query, un modello di regressione locale risolvendo un problema di minimi quadrati pesati. Gli esempi di training contribuiscono alla stima in modo proporzionale alla loro vicinanza a x_{new} , secondo una funzione di peso (tipicamente gaussiana).

Chapter 10

Multiple Learners

Nei capitoli precedenti abbiamo considerato modelli di apprendimento singoli, progettati per approssimare una funzione obiettivo a partire da un insieme di dati. In questo contesto, la qualità delle prestazioni dipende fortemente dalla scelta del modello, dalla sua complessità e dalla quantità di dati disponibili. Tuttavia, anche modelli ben progettati possono risultare limitati quando affrontano problemi complessi o dati rumorosi.

L'approccio dei *multiple learners*, noto anche come *ensemble learning*, si basa su un'idea diversa: invece di costruire un singolo modello molto complesso, si addestrano più modelli (detti *base learners*) e si combinano le loro predizioni per ottenere un risultato complessivo più accurato e robusto.

Formalmente, si considera un insieme di modelli

$$\{y_1(x), y_2(x), \dots, y_M(x)\},$$

addestrati sullo stesso problema, e una funzione di combinazione che produce la predizione finale a partire dalle singole uscite. I modelli possono essere addestrati in modo indipendente oppure in modo sequenziale, a seconda della strategia adottata.

L'idea alla base dell'ensemble learning è analoga al processo decisionale collettivo: una singola opinione può essere errata o influenzata dal rumore, mentre la combinazione di più punti di vista indipendenti tende a ridurre gli errori individuali. Anche se ciascun modello è relativamente semplice o imperfetto, l'insieme può produrre una decisione più affidabile.

Un aspetto cruciale è che i modelli combinati devono essere in qualche misura *diversi* tra loro. Se tutti i modelli commettono gli stessi errori, la loro combinazione non porta alcun beneficio. Le tecniche di multiple learners mirano quindi a introdurre diversità tra i modelli, sfruttandola per migliorare le prestazioni complessive.

Non è sufficiente avere molti modelli: è importante che ciascun modello “sbagli in modo diverso”. L'ensemble funziona quando gli errori dei singoli modelli si compensano a vicenda, lasciando emergere una predizione più stabile.

Le principali strategie di ensemble learning si distinguono in base a come vengono addestrati e combinati i modelli:

- combinazione di modelli addestrati in parallelo (ad esempio tramite voting o bagging);
- combinazione di modelli addestrati in sequenza, in cui ogni modello corregge gli errori dei precedenti (boosting).

Queste strategie, pur condividendo l'idea di base, affrontano in modo diverso il compromesso tra bias e varianza e saranno analizzate nel dettaglio nelle sezioni successive.

10.1 Voting

Una delle strategie più semplici per combinare più modelli di apprendimento è il *voting*. L'idea di base consiste nell'addestrare più modelli sullo stesso dataset e combinare le loro predizioni attraverso una media (nel caso di regressione) o una maggioranza (nel caso di classificazione).

Sia dato un insieme di M modelli

$$\{y_1(x), y_2(x), \dots, y_M(x)\},$$

addestrati a partire dallo stesso insieme di dati $\mathcal{D}$. Nel caso della regressione, la predizione combinata è definita come:

$$y_{\text{voting}}(x) = \sum_{m=1}^M w_m y_m(x),$$

dove $w_m \geq 0$ sono pesi assegnati ai singoli modelli, tali che:

$$\sum_{m=1}^M w_m = 1.$$

Nel caso della classificazione, la predizione finale è ottenuta tramite una maggioranza pesata:

$$y_{\text{voting}}(x) = \arg \max_c \sum_{m=1}^M w_m \mathbb{I}(y_m(x) = c),$$

dove $\mathbb{I}(\cdot)$ è la funzione indicatrice.

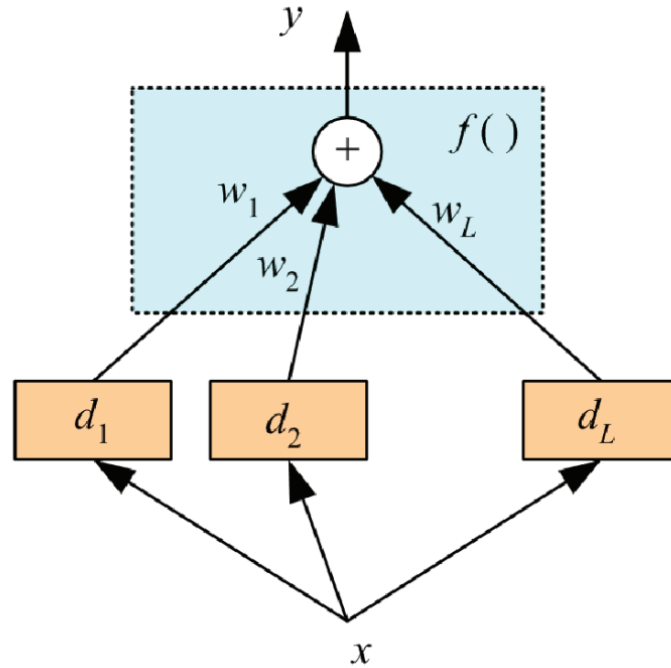


Figure 10.1: Schema concettuale del voting. L'input x viene fornito a più modelli ($d_1, d_2, \dots, d_L$), le cui predizioni vengono combinate tramite una somma pesata per produrre l'output finale y .

Come illustrato in Figura 10.1, ogni modello produce una predizione indipendente a partire dallo stesso input. Le singole uscite vengono poi aggregate da una funzione di combinazione, che può essere vista come un modello lineare nello spazio delle predizioni.

Il voting può essere interpretato come una media delle opinioni di più esperti. Ogni modello fornisce una stima potenzialmente imperfetta, ma combinando più predizioni si riduce l'impatto degli errori individuali. Se i modelli commettono errori non correlati, questi tendono a compensarsi, producendo una predizione complessivamente più stabile.

Nel caso in cui tutti i pesi siano uguali ($w_m = 1/M$), si parla di *uniform voting*. L'introduzione di pesi differenti consente invece di assegnare maggiore importanza ai modelli ritenuti più affidabili.

Osservazione Il voting non modifica i modelli di base né il processo di addestramento: l'intervento avviene esclusivamente a livello di combinazione delle predizioni. Per questo motivo, il voting rappresenta una tecnica semplice, flessibile e facilmente parallelizzabile, ma la sua efficacia dipende fortemente dalla diversità tra i modelli combinati.

10.2 Mixture of Experts

Una generalizzazione del voting è rappresentata dal modello di *mixture of experts*, in cui la combinazione delle predizioni non avviene tramite pesi fissi, ma attraverso una funzione dipendente dall'input.

Siano $d_1(x), d_2(x), \dots, d_L(x)$ un insieme di modelli di base. La predizione finale è

definita come:

$$y(x) = \sum_{l=1}^L g_l(x) d_l(x),$$

dove $g_l(x)$ è una funzione di *gating* tale che:

$$g_l(x) \geq 0, \quad \sum_{l=1}^L g_l(x) = 1.$$

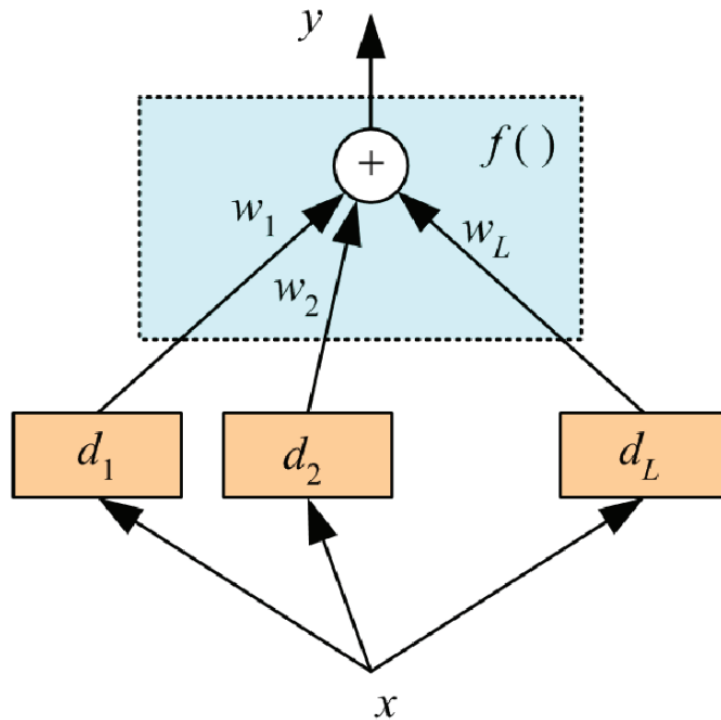


Figure 10.2: Schema del modello di mixture of experts. L'input x viene fornito sia ai modelli di base ($d_1, \dots, d_L$) sia alla funzione di gating, che determina dinamicamente i pesi con cui le singole predizioni vengono combinate per produrre l'output finale y .

A differenza del voting, in cui i pesi sono costanti, nel mixture of experts la funzione di gating apprende come selezionare o pesare i diversi modelli in base all'input.

Il mixture of experts può essere visto come un sistema adattivo che, per ogni input, decide quali modelli siano più affidabili. In questo modo, ciascun esperto può specializzarsi in una particolare regione dello spazio degli input, mentre il gating coordina il loro contributo, migliorando la capacità di modellare comportamenti complessi.

10.3 Stacking

Un'ulteriore estensione delle tecniche di ensemble è rappresentata dallo *stacking*, in cui la funzione di combinazione delle predizioni viene essa stessa appresa dai dati.

Siano $d_1(x), d_2(x), \dots, d_L(x)$ un insieme di modelli di base addestrati sullo stesso problema. Nello stacking, le predizioni dei modelli di base costituiscono l'input di un

meta-modello $f(\cdot)$, che produce la predizione finale:

$$y(x) = f(d_1(x), d_2(x), \dots, d_L(x)).$$

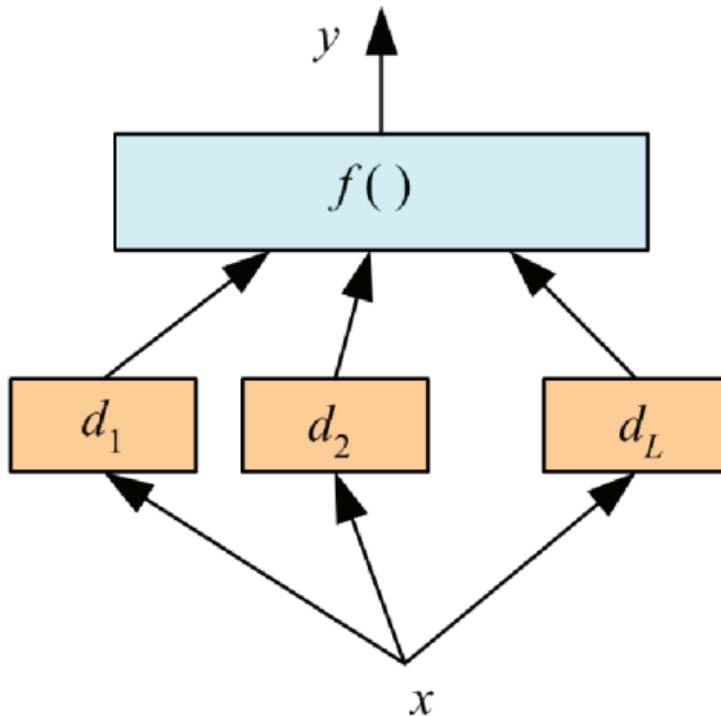


Figure 10.3: Schema dello stacking. L'input x viene fornito a più modelli di base $(d_1, d_2, \dots, d_L)$, le cui predizioni sono utilizzate come input di un meta-modello $f(\cdot)$ che apprende come combinarle per produrre l'output finale y .

A differenza del voting e del mixture of experts, nello stacking non viene imposta alcuna forma specifica alla funzione di combinazione. Il meta-modello può essere qualsiasi algoritmo di apprendimento supervisionato, come una regressione lineare, una SVM o una rete neurale.

Lo stacking può essere visto come un processo di apprendimento su più livelli: i modelli di base apprendono una prima rappresentazione del problema, mentre il meta-learner apprende come sfruttare al meglio tali rappresentazioni, correggendone sistematicamente gli errori.

10.4 Cascading

Il *cascading* è una tecnica di ensemble in cui i modelli di base sono organizzati in sequenza e interrogati progressivamente in base a un criterio di confidenza. A differenza di altre strategie di combinazione, nel cascading non vengono aggregate più predizioni: l'output finale è fornito da un singolo modello selezionato dinamicamente.

Siano $d_1(x), d_2(x), \dots, d_L(x)$ un insieme di modelli di base e $c_l(x)$ una misura di con-

fidenza associata al modello d_i . La predizione finale è definita come:

$$y(x) = \begin{cases} d_1(x) & \text{se } c_1(x) > \theta_1 \\ d_2(x) & \text{se } c_2(x) > \theta_2 \\ \vdots & \\ d_L(x) & \text{altrimenti.} \end{cases}$$

Il cascading può essere visto come una strategia decisionale a più stadi: modelli semplici e veloci vengono utilizzati per risolvere i casi facili, mentre modelli più complessi intervengono solo quando necessario. Questo approccio consente di ridurre il costo computazionale medio mantenendo buone prestazioni complessive.

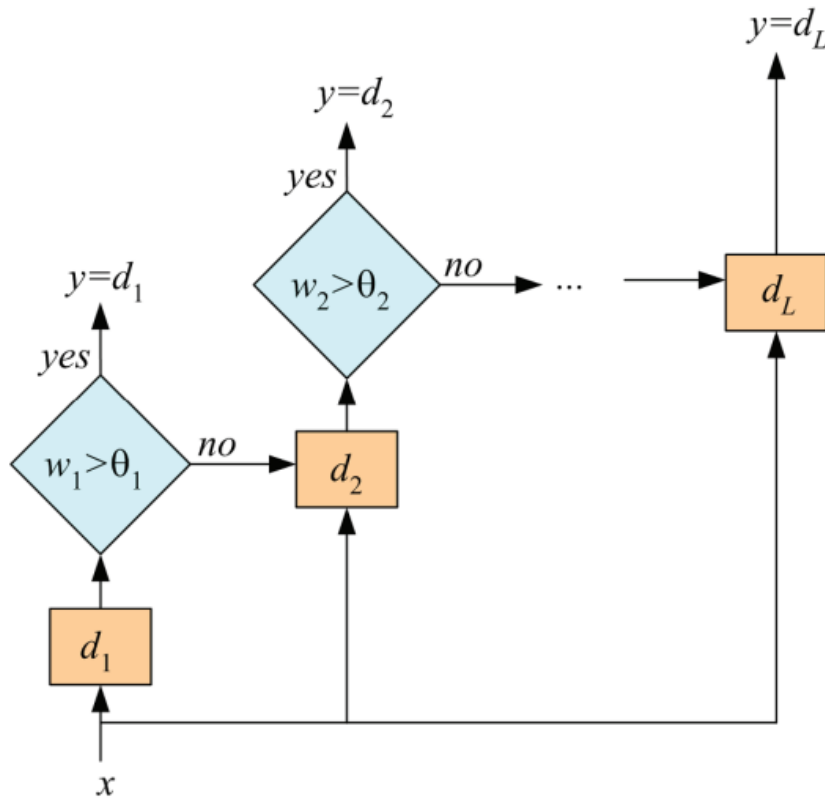


Figure 10.4: Schema del cascading. L'input x viene elaborato da una sequenza di modelli. Ogni modello restituisce una predizione solo se la sua confidenza supera una soglia prefissata; in caso contrario, l'input viene passato al modello successivo.

10.5 Bagging

Il *bagging* (Bootstrap Aggregating) è una tecnica di ensemble learning progettata per migliorare la stabilità e la capacità di generalizzazione di un modello di apprendimento, riducendone la varianza. A differenza delle strategie viste in precedenza, il bagging introduce diversità tra i modelli non attraverso la combinazione delle predizioni o funzioni adattive, ma tramite la costruzione di differenti insiemi di addestramento.

Dato un dataset di training

$$\mathcal{D} = \{(x_1, t_1), \dots, (x_N, t_N)\},$$

il bagging procede secondo i seguenti passi:

1. si generano M dataset bootstrap $\mathcal{D}_1, \dots, \mathcal{D}_M$ mediante campionamento casuale con rimpiazzo a partire da $\mathcal{D}$;
2. su ciascun dataset $\mathcal{D}_m$ viene addestrato un modello $y_m(x)$;
3. le predizioni dei modelli vengono combinate tramite una media (regressione) o una maggioranza (classificazione).

Nel caso della regressione, la predizione finale è:

$$y_{\text{bagging}}(x) = \frac{1}{M} \sum_{m=1}^M y_m(x).$$

Nel caso della classificazione, si utilizza una maggioranza di voto:

$$y_{\text{bagging}}(x) = \arg \max_c \sum_{m=1}^M \mathbb{I}(y_m(x) = c).$$

Tutti i modelli di base utilizzati nel bagging sono tipicamente dello stesso tipo e vengono addestrati in modo indipendente e parallelo. Il bagging sfrutta il fatto che molti modelli di apprendimento, in particolare quelli complessi e flessibili (come alberi di decisione profondi), sono altamente sensibili alle variazioni del dataset di training. Piccole modifiche nei dati possono produrre modelli molto diversi.

Creando molte versioni leggermente diverse del dataset tramite bootstrap, il bagging induce una variabilità artificiale tra i modelli. La media delle loro predizioni tende a cancellare le fluttuazioni dovute al rumore, producendo una stima più stabile.

In termini di decomposizione bias-varianza, il bagging agisce principalmente sulla *varianza* del modello: non riduce significativamente il bias, ma attenua l'overfitting tipico dei modelli ad alta complessità. Il bagging è particolarmente efficace quando i modelli di base sono instabili, ovvero quando piccole variazioni nei dati di training producono grandi variazioni nella funzione appresa. In questi casi, l'aggregazione delle predizioni consente di ottenere un miglioramento consistente delle prestazioni di generalizzazione.

10.6 Boosting

Il *boosting* è una tecnica di ensemble learning in cui i modelli di base vengono addestrati in modo sequenziale, assegnando progressivamente maggiore importanza ai campioni difficili da classificare. A differenza del bagging, i modelli non sono indipendenti: ciascun learner è influenzato dalle prestazioni dei precedenti.

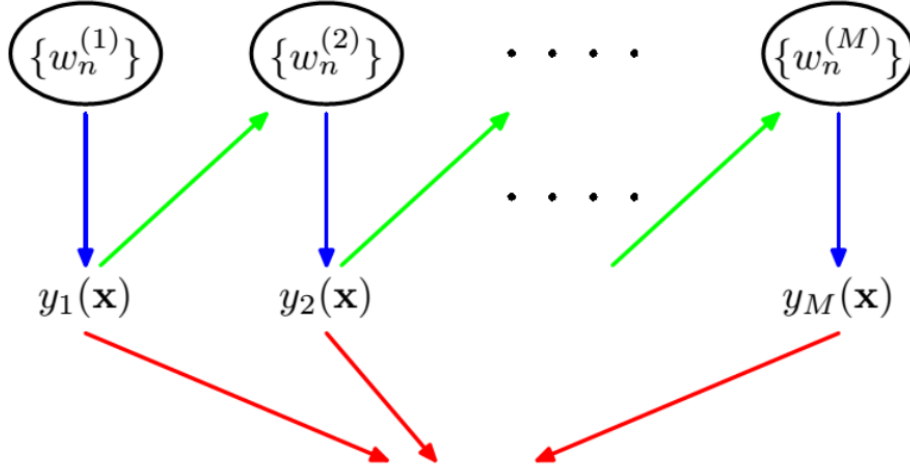


Figure 10.5: Schema concettuale del boosting. A ogni iterazione m , il modello $y_m(x)$ viene addestrato su un dataset pesato $\{w_n^{(m)}\}$. I pesi dei campioni mal classificati vengono aumentati, costringendo i learner successivi a concentrarsi sugli errori precedenti. La predizione finale è una combinazione pesata delle predizioni dei singoli modelli.

Sia $\mathcal{D} = \{(x_n, t_n)\}_{n=1}^N$ un dataset con etichette $t_n \in \{-1, +1\}$. Il boosting costruisce una sequenza di weak learners $y_1(x), \dots, y_M(x)$, ciascuno addestrato minimizzando un errore pesato. Al termine, le predizioni vengono combinate come:

$$Y_M(x) = \text{sign} \left(\sum_{m=1}^M \alpha_m y_m(x) \right).$$

Il boosting può essere visto come un processo di apprendimento cooperativo: ogni modello cerca di correggere le debolezze dei precedenti, focalizzandosi sui campioni più difficili. Questo meccanismo consente di trasformare un insieme di learner deboli in un classificatore forte.

10.6.1 AdaBoost

AdaBoost (Adaptive Boosting) è uno degli algoritmi di boosting più noti ed efficaci. Il suo obiettivo è costruire un classificatore forte combinando una sequenza di classificatori deboli, ciascuno dei quali viene addestrato ponendo maggiore enfasi sui campioni che risultano difficili da classificare.

Si consideri un dataset di training

$$\mathcal{D} = \{(x_n, t_n)\}_{n=1}^N, \quad t_n \in \{-1, +1\}.$$

AdaBoost mantiene una distribuzione di pesi sui campioni, indicata con

$$w_n^{(m)}, \quad n = 1, \dots, N,$$

che viene aggiornata a ogni iterazione m .

Algoritmo AdaBoost

1. Inizializzare i pesi dei campioni:

$$w_n^{(1)} = \frac{1}{N}, \quad n = 1, \dots, N.$$

2. Per $m = 1, \dots, M$:

- (a) addestrare un weak learner $y_m(x)$ minimizzando l'errore pesato:

$$J_m = \sum_{n=1}^N w_n^{(m)} \mathbb{I}(y_m(x_n) \neq t_n);$$

- (b) calcolare l'errore del classificatore:

$$\varepsilon_m = \frac{\sum_{n=1}^N w_n^{(m)} \mathbb{I}(y_m(x_n) \neq t_n)}{\sum_{n=1}^N w_n^{(m)}};$$

- (c) calcolare il peso del classificatore:

$$\alpha_m = \ln\left(\frac{1 - \varepsilon_m}{\varepsilon_m}\right);$$

- (d) aggiornare i pesi dei campioni:

$$w_n^{(m+1)} = w_n^{(m)} \exp(\alpha_m \mathbb{I}(y_m(x_n) \neq t_n)).$$

La predizione finale è ottenuta come:

$$Y_M(x) = \text{sign}\left(\sum_{m=1}^M \alpha_m y_m(x)\right).$$

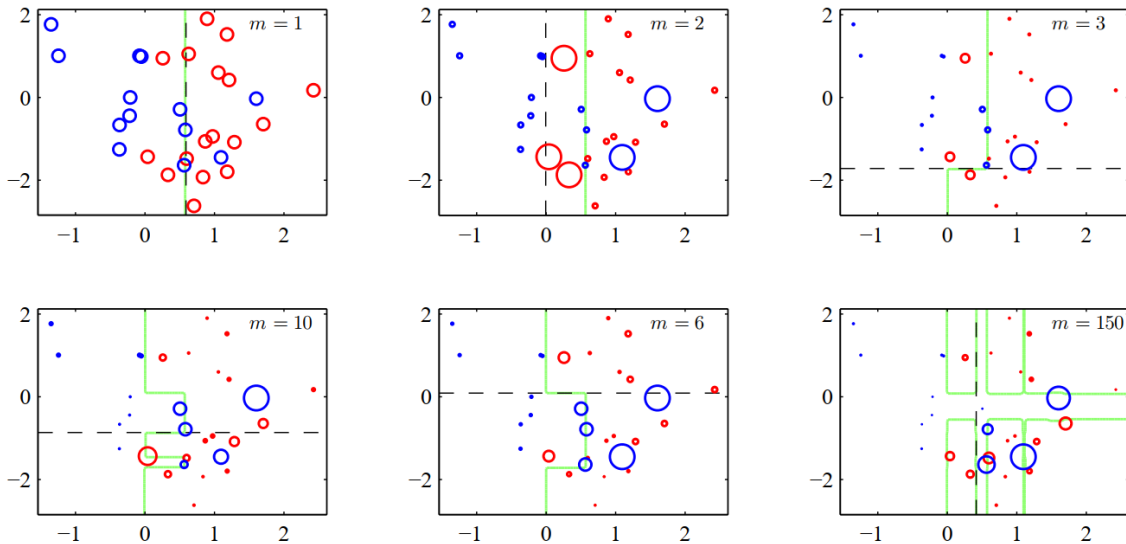


Figure 10.6: Evoluzione di AdaBoost al crescere del numero di iterazioni m . I cerchi più grandi indicano campioni con peso maggiore. I weak learner successivi si concentrano progressivamente sui punti difficili, producendo una frontiera di decisione sempre più articolata.

AdaBoost può essere interpretato come un processo di apprendimento iterativo in cui, a ogni passo, il modello cerca di correggere gli errori commessi in precedenza. I campioni classificati correttamente perdono importanza, mentre quelli sbagliati vengono enfatizzati, costringendo i weak learner successivi a concentrarsi sulle regioni più difficili dello spazio degli input.

Nel corso delle iterazioni, la combinazione pesata dei classificatori produce una frontiera di decisione sempre più complessa, pur partendo da modelli molto semplici.

Remarks

Vantaggi

- È semplice da implementare e computazionalmente efficiente.
- Non richiede una conoscenza specifica del weak learner, purché sia leggermente migliore del caso casuale.
- Non necessita di un'accurata scelta di iperparametri (a eccezione del numero di iterazioni M).
- Fornisce solide garanzie teoriche e ottime prestazioni empiriche in molti problemi reali.
- Trasforma classificatori deboli in un classificatore forte.

Svantaggi

- È sensibile al rumore e agli outlier, che tendono a ricevere pesi sempre più elevati.
- Può degradare le prestazioni in presenza di dati fortemente rumorosi o mal etichettati.
- La performance dipende fortemente dalla qualità dei weak learner.
- Può portare a modelli molto complessi se il numero di iterazioni è elevato.

RIEPILOGO

Multiple Learners – Concetti chiave

L'ensemble learning si basa sull'idea di combinare più modelli di apprendimento (*base learners*) per ottenere prestazioni migliori rispetto a un singolo modello. L'efficacia di questo approccio dipende dalla diversità tra i modelli e dalla strategia di combinazione adottata.

Voting Combina le predizioni di più modelli addestrati sullo stesso dataset mediante una media (regressione) o una maggioranza pesata (classificazione):

$$y(x) = \sum_{m=1}^M w_m y_m(x).$$

I pesi sono fissi e indipendenti dall'input.

Mixture of Experts Generalizza il voting introducendo una funzione di *gating* dipendente dall'input:

$$y(x) = \sum_{l=1}^L g_l(x) d_l(x),$$

consentendo ai modelli di specializzarsi in diverse regioni dello spazio degli input.

Stacking Utilizza un meta-learner per apprendere la funzione di combinazione delle predizioni:

$$y(x) = f(d_1(x), \dots, d_L(x)).$$

La strategia di aggregazione non è imposta, ma appresa dai dati.

Cascading Organizza i modelli in sequenza, utilizzando criteri di confidenza per decidere quale modello fornisce la predizione finale:

$$y(x) = d_l(x) \quad \text{per il primo } l \text{ tale che } c_l(x) > \theta_l.$$

È particolarmente efficace per ridurre il costo computazionale medio.

Bagging Introduce diversità tra i modelli tramite bootstrap del dataset di training. I modelli sono addestrati in parallelo e combinati mediante media o voto:

$$y_{\text{bagging}}(x) = \frac{1}{M} \sum_{m=1}^M y_m(x).$$

Agisce principalmente riducendo la varianza.

Boosting e AdaBoost Addestra i modelli in sequenza, ri-pesando i campioni in base agli errori precedenti:

$$Y_M(x) = \text{sign} \left(\sum_{m=1}^M \alpha_m y_m(x) \right).$$

AdaBoost enfatizza i campioni difficili e mira a ridurre il bias del modello complessivo.

Bias–Variance Tradeoff Le tecniche di ensemble permettono di agire sul compromesso bias–varianza:

- il bagging riduce principalmente la varianza;
- il boosting riduce principalmente il bias;
- la combinazione di più learner può migliorare la robustezza al rumore.

In sintesi, i multiple learners forniscono un potente insieme di strumenti per migliorare accuratezza, stabilità ed efficienza dei modelli di apprendimento, rappresentando una delle strategie fondamentali del machine learning moderno.

Chapter 11

Artificial Neural Networks

Le **Artificial Neural Networks (ANN)** sono modelli di apprendimento automatico parametrici progettati per approssimare funzioni complesse. Esse sono note anche con i seguenti nomi:

- *Neural Networks* (NN)
- *Feedforward Neural Networks* (FNN)
- *Multilayer Perceptrons* (MLP)

Le ANN sono particolarmente adatte a problemi formulabili come associazione tra vettori, ovvero alla stima di una funzione:

$$f : \mathcal{X} \rightarrow \mathcal{Y},$$

dove il codominio può rappresentare:

- un insieme discreto di classi $\mathcal{Y} = \{C_1, \dots, C_k\}$ (classificazione),
- uno spazio continuo $\mathcal{Y} \subseteq \mathbb{R}^d$ (regressione).

Dato un dataset di addestramento

$$\mathcal{D} = \{(x_n, t_n)\}_{n=1}^N,$$

l'obiettivo è apprendere una funzione approssimata

$$\hat{f}(x; \theta)$$

parametrizzata da un insieme di parametri θ , tale che:

$$\hat{f}(x_n; \theta) \approx t_n \quad \forall n.$$

11.1 Feedforward Neural Networks

Le **Feedforward Neural Networks** sono ispirate, in modo astratto, alla struttura del cervello biologico, ma **non costituiscono un modello biologico realistico**. Esse sfruttano solo alcune intuizioni di base, come l'elaborazione distribuita e la composizione di unità semplici.

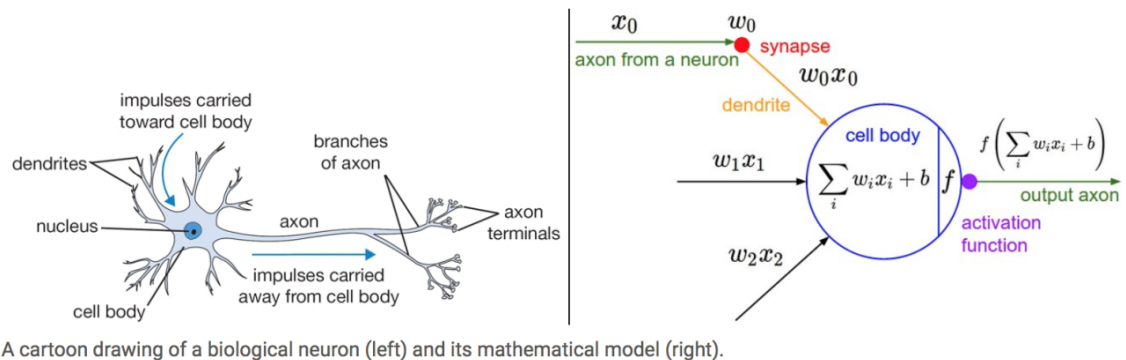


Figure 11.1

In una FNN l'informazione fluisce esclusivamente:

input $\rightarrow$ hidden layers $\rightarrow$ output,

senza la presenza di cicli o retroazioni.

Formalmente, una rete feedforward implementa una funzione ottenuta come composizione di più trasformazioni elementari:

$$f(x; \theta) = f^{(L)}(f^{(L-1)}(\dots f^{(1)}(x; \theta^{(1)}) \dots; \theta^{(L-1)}); \theta^{(L)}),$$

dove:

- $f^{(m)}$ rappresenta il livello m -esimo della rete,
- $\theta^{(m)}$ indica i parametri allenabili del livello m ,
- L è la profondità (*depth*) della rete.

Le FNN hanno quindi una struttura a catena (*chain structure*). L'ultimo livello è detto **output layer**, mentre i livelli intermedi sono detti **hidden layers**.

Il termine **Deep Learning** deriva dall'uso di reti con un numero elevato di livelli, ovvero con grande profondità.

11.1.1 Perché le FNN

Le reti feedforward superano alcune limitazioni fondamentali di altri approcci:

- i **modelli lineari** non sono in grado di rappresentare interazioni non lineari tra le variabili di input;
- i **metodi kernel** richiedono la scelta esplicita di un kernel adeguato:
 - kernel generici (RBF, polinomiali) portano a problemi convessi ma limitati in espressività;
 - kernel progettati manualmente sono fortemente dipendenti dall'applicazione.

Le FNN, invece, apprendono automaticamente:

- combinazioni complesse di molte funzioni parametriche,

- modelli altamente non lineari nei parametri θ ,

al prezzo di un problema di ottimizzazione **non convesso**.

Questo compromesso rende le reti neurali estremamente potenti e flessibili per una vasta gamma di problemi di apprendimento automatico.

11.1.2 Architettura delle Feedforward Neural Networks

La progettazione dell'architettura di una Feedforward Neural Network (FNN) è un aspetto cruciale e comprende le seguenti scelte fondamentali:

- numero di livelli nascosti (*depth*);
- numero di unità per ciascun livello (*width*);
- tipo di unità e funzioni di attivazione;
- funzione di costo (loss function).

Tali decisioni influenzano direttamente la capacità espressiva del modello, la sua addestrabilità e le prestazioni di generalizzazione.

Modello generale di una FNN

Una FNN è composta da più livelli completamente connessi. Dato un livello i , l'uscita del livello successivo è definita come:

$$\mathbf{h}_{i+1} = f(\mathbf{W}_i^\top \mathbf{h}_i + \mathbf{b}_i),$$

dove:

- $\mathbf{h}_i$ è il vettore delle attivazioni del livello i ;
- $\mathbf{W}_i$ è la matrice dei pesi;
- $\mathbf{b}_i$ è il vettore dei bias;
- $f(\cdot)$ è una funzione di attivazione non lineare.

Il numero di parametri cresce con la dimensione dei livelli:

$$|\mathbf{W}_i| = (\text{n. input}) \times (\text{n. output}), \quad |\mathbf{b}_i| = (\text{n. output}).$$

11.2 Profondità e Teorema di Approssimazione Universale

Una domanda fondamentale nella progettazione delle reti neurali è:

Quanti livelli nascosti sono necessari per approssimare una funzione arbitraria?

Il **Teorema di Approssimazione Universale** fornisce una risposta teorica a questo problema.

Il teorema ci dice che una Feedforward Neural Network con:

- un singolo livello nascosto,
- un numero sufficiente di unità nascoste,
- una funzione di attivazione non lineare di tipo “squashing” (ad esempio sigmoide),
- un livello di output lineare,

è in grado di approssimare qualsiasi funzione misurabile di Borel con un errore arbitrariamente piccolo.

Il risultato vale anche per altre funzioni di attivazione non lineari, come la **ReLU**.

Osservazioni importanti

- Il teorema garantisce l'**esistenza** di una rete capace di approssimare una funzione, ma non fornisce indicazioni su:
 - il numero di unità necessarie;
 - la facilità di addestramento;
 - la qualità della generalizzazione.
- Il numero di unità richieste può crescere in modo **esponenziale** con la dimensione dell'input.

Larghezza vs Profondità

Il teorema di approssimazione universale implica che, in teoria:

- una rete **poco profonda ma molto larga** può approssimare qualsiasi funzione;

tuttavia, nella pratica:

- reti **più profonde e più strette** risultano:
 - più efficienti dal punto di vista parametrico;
 - più facili da addestrare;
 - migliori in termini di generalizzazione.

Questo giustifica l'uso di architetture profonde nel **deep learning**, dove la composizione di molte trasformazioni semplici consente di rappresentare strutture complesse in modo gerarchico.

Considerazioni pratiche sull'architettura

Non esistono principi teorici generali per determinare l'architettura ottimale di una FNN. La progettazione è in larga parte **empirica** e dipende da:

- tipo di problema;
- dimensione e struttura dei dati;
- capacità computazionali disponibili.

In sintesi, il teorema di approssimazione universale fornisce il fondamento teorico della potenza espressiva delle reti neurali, mentre le architetture profonde ne rappresentano la realizzazione pratica più efficace.

11.3 Funzioni di attivazione e funzione di costo

Una rete neurale feedforward definisce una funzione parametrica

$$y = f(x; \theta),$$

ottenuta come composizione di trasformazioni lineari e non lineari. Due elementi fondamentali determinano il comportamento del modello:

- le **funzioni di attivazione**, che definiscono la capacità espressiva della rete;
- la **loss function**, che quantifica l'errore e guida l'apprendimento.

Le funzioni di attivazione negli strati nascosti introducono non linearità nel modello. In assenza di tali funzioni, anche una rete profonda collasserebbe in una trasformazione lineare equivalente.

Non esistono principi teorici generali per la scelta ottimale della funzione di attivazione negli strati nascosti: la selezione è prevalentemente **empirica**.

11.3.1 Funzioni di attivazione dell'output layer

La funzione di attivazione dell'output layer dipende direttamente dal tipo di problema da risolvere. La scelta dell'output unit e della funzione di costo è infatti strettamente legata all'interpretazione probabilistica dell'output del modello.

Sia $\mathbf{h} = f(\mathbf{x}; \theta^{(n-1)})$ l'output dell'ultimo hidden layer. L'output della rete è dato da.

$$\mathbf{y} = f^{(n)}(\mathbf{h}; \theta^{(n)}),$$

dove la funzione $f^{(n)}$ viene scelta in funzione del task.

Regressione

Nel caso di problemi di regressione, l'output layer utilizza unità lineari, ovvero la funzione di identità:

$$y = \mathbf{w}^T \mathbf{h} + b.$$

Si assume un modello di rumore Gaussiano:

$$p(t | \mathbf{x}) = \mathcal{N}(t | y, \beta^{-1}),$$

per cui la massimizzazione della log-verosimiglianza equivale alla minimizzazione della **mean squared error (MSE)**.

Le unità lineari non saturano e sono quindi adatte alla predizione di valori reali non limitati, inoltre consentono alla rete di predire valori continui non limitati e di non distorcere il valore stimato. Questa scelta è particolarmente appropriata quando il target appartiene a $\mathbb{R}$ e quando la funzione di perdita adottata è l'errore quadratico medio (Mean Squared Error, MSE).

Classificazione binaria

Per problemi di classificazione binaria, l'output layer utilizza la funzione di attivazione **sigmoid**:

$$y = \sigma(\mathbf{w}^T \mathbf{h} + b).$$

L'output y può essere interpretato come una probabilità. Il modello di verosimiglianza corrisponde a una distribuzione di Bernoulli e la funzione di costo associata è la **binary cross-entropy**, ovvero la negative log-likelihood.

In questo caso, l'unità di output satura solo quando il modello fornisce una predizione corretta con alta confidenza.

Classificazione multi-classe

Nel caso di classificazione multi-classe, l'output layer è costituito da più unità e utilizza la funzione di attivazione **softmax**:

$$y_i = \text{softmax}(\alpha^{(i)}) = \frac{\exp(\alpha^{(i)})}{\sum_j \exp(\alpha^{(j)})}, \quad \alpha^{(i)} = \mathbf{w}_i^T \mathbf{h} + b_i.$$

La softmax produce una distribuzione di probabilità sulle classi. La funzione di costo associata è la **categorical cross-entropy**, che corrisponde alla negative log-likelihood di una distribuzione multinomiale.

Le unità di output saturano solo in presenza di errori minimi, favorendo una maggiore stabilità dell'addestramento.

11.3.2 Funzioni di attivazione nell'hidden layer

Negli **hidden layer**, la funzione di attivazione ha lo scopo principale di introdurre non linearità nel modello, permettendo alla rete di apprendere rappresentazioni complesse e non linearmente separabili dei dati. In questo contesto, la funzione di attivazione non è vincolata all'interpretazione dell'output, ma è scelta per favorire la capacità espressiva e la stabilità dell'addestramento.

Rectified Linear Unit (ReLU)

La funzione di attivazione più utilizzata è la **Rectified Linear Unit**:

$$g(\alpha) = \max(0, \alpha).$$

Proprietà principali:

- semplicità computazionale;
- comportamento lineare per $\alpha > 0$;
- mitigazione del problema del gradiente che svanisce;
- non differenziabilità in $\alpha = 0$, irrilevante nella pratica.

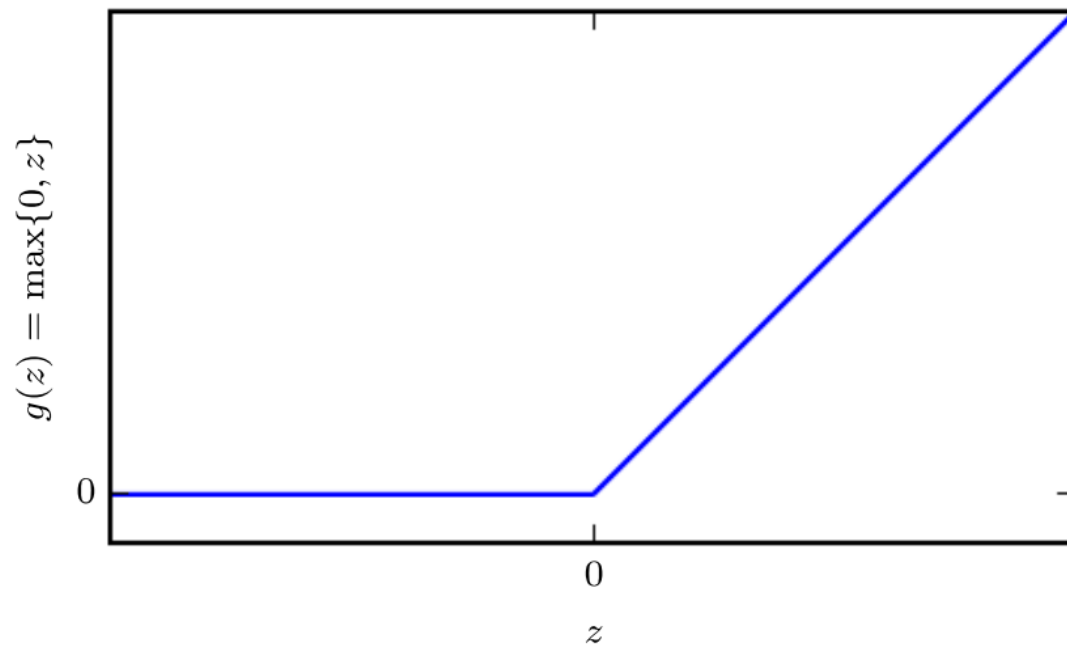


Figure 11.2: Reappresentazione matematica della ReLu

Le ReLU risultano particolarmente adatte a reti profonde.

Sigmoide e tangente iperbolica

Altre funzioni storicamente utilizzate sono:

$$g(\alpha) = \sigma(\alpha), \quad g(\alpha) = \tanh(\alpha),$$

con la relazione:

$$\tanh(\alpha) = 2\sigma(2\alpha) - 1.$$

Entrambe sono funzioni di tipo “squashing” e soffrono di saturazione:

- per grandi valori di $|\alpha|$ la derivata tende a zero;
- il gradiente si attenua attraversando la rete;
- l'apprendimento basato sul gradiente risulta lento.

La funzione $\tanh$ fornisce gradienti leggermente più grandi rispetto alla sigmoide ed è talvolta preferita in contesti specifici.

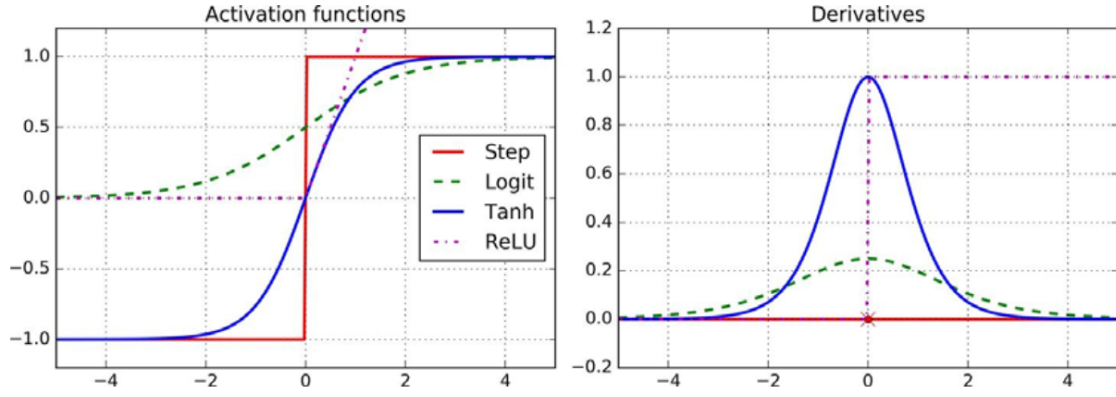


Figure 11.3: Confronto tra le principali funzioni di attivazione e le loro derivate.

Output della rete e interpretazione probabilistica

Sia $\mathbf{h}$ l'output dell'ultimo strato nascosto. La rete neurale definisce implicitamente una distribuzione condizionata:

$$p(t \mid x, \theta),$$

che dipende dalla forma dell'output e dalla funzione di attivazione utilizzata nell'ultimo strato.

La scelta dell'attivazione di output **non è indipendente** dalla funzione di costo, ma ne costituisce parte integrante.

11.3.3 Loss function e principio di massima verosimiglianza

L'addestramento della rete consiste nella minimizzazione della loss function:

$$J(\theta) = \mathbb{E}_{x, t \sim \mathcal{D}} [-\log p(t \mid x, \theta)],$$

che deriva dal principio di **Massima Verosimiglianza (MLE)**.

La forma esplicita della loss dipende dall'ipotesi fatta sulla distribuzione del target.

Regressione Per problemi di regressione si utilizza un output lineare:

$$y = \mathbf{W}^\top \mathbf{h} + \mathbf{b}.$$

Assumendo rumore gaussiano:

$$p(t \mid x, \theta) = \mathcal{N}(t \mid y, \beta^{-1}),$$

la massimizzazione della verosimiglianza equivale alla minimizzazione della **Mean Squared Error (MSE)**:

$$J(\theta) = \mathbb{E}_{x, t \sim \mathcal{D}} \left[\frac{1}{2} \|t - f(x; \theta)\|^2 \right].$$

Le unità di output lineari non saturano, favorendo un apprendimento stabile.

Classificazione binaria Nel caso di classificazione binaria, il target $t \in \{0, 1\}$ è modellato come variabile di Bernoulli:

$$y = \sigma(\mathbf{w}^\top \mathbf{h} + b), \quad p(t \mid x, \theta) = y^t (1 - y)^{1-t}.$$

La loss function associata è la **Binary Cross-Entropy**:

$$J(\theta) = \mathbb{E}_{x,t \sim \mathcal{D}} [-t \log y - (1 - t) \log(1 - y)].$$

Classificazione multiclasse Per problemi di classificazione multiclasse si utilizza la funzione **softmax**:

$$y_i = \frac{\exp(\alpha_i)}{\sum_j \exp(\alpha_j)}, \quad \alpha_i = \mathbf{w}_i^\top \mathbf{h} + b_i.$$

La loss function è la **Categorical Cross-Entropy**:

$$J(\theta) = \mathbb{E}_{x,t \sim \mathcal{D}} \left[- \sum_{i=1}^K \mathbb{I}(t = i) \log y_i \right].$$

11.4 Calcolo del gradiente

L'addestramento di una rete neurale consiste nella minimizzazione della loss function

$$J(\theta),$$

rispetto ai parametri θ del modello. Per questo motivo è necessario calcolare il **gradiente della loss** rispetto a tutti i parametri della rete:

$$\nabla_{\theta} J(\theta).$$

Flusso dell'informazione

Il funzionamento di una rete neurale può essere descritto attraverso due fasi distinte:

- **forward pass**: l'informazione fluisce dall'input all'output per calcolare

$$y = f(x; \theta)$$

e il valore della loss $J(\theta)$;

- **backward pass**: l'informazione fluisce in senso inverso per calcolare i gradienti della loss rispetto ai parametri.

Il calcolo del gradiente è quindi strettamente legato alla struttura composita della funzione implementata dalla rete.

Obiettivo del calcolo del gradiente

Dato un dataset $\mathcal{D}$, l'obiettivo è determinare:

$$\nabla_{\theta} J(\theta) = \left(\frac{\partial J}{\partial \theta_1}, \frac{\partial J}{\partial \theta_2}, \dots \right),$$

dove ciascun parametro θ_i può rappresentare un peso o un bias in uno qualsiasi dei livelli della rete.

Dal punto di vista analitico, il calcolo del gradiente è:

- concettualmente semplice, poiché basato sulla regola della catena;
- potenzialmente costoso dal punto di vista computazionale se eseguito in modo ingenuo.

Regola della catena

La rete neurale è una composizione di funzioni. Nel caso scalare, se:

$$y = g(x), \quad z = f(y),$$

la derivata di z rispetto a x è:

$$\frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx}.$$

Nel caso vettoriale, con:

$$g : \mathbb{R}^m \rightarrow \mathbb{R}^n, \quad f : \mathbb{R}^n \rightarrow \mathbb{R},$$

vale:

$$\frac{\partial z}{\partial x_i} = \sum_j \frac{\partial z}{\partial y_j} \frac{\partial y_j}{\partial x_i},$$

oppure, in notazione compatta:

$$\nabla_x z = \left(\frac{\partial \mathbf{y}}{\partial \mathbf{x}} \right)^\top \nabla_{\mathbf{y}} z,$$

dove $\frac{\partial \mathbf{y}}{\partial \mathbf{x}}$ è la matrice Jacobiana della funzione g .

11.5 Back-propagation

La **back-propagation** è il meccanismo che rende possibile l'addestramento delle reti neurali profonde. Dal punto di vista concettuale, essa risponde a una domanda fondamentale:

in che modo ogni singolo parametro della rete contribuisce all'errore finale?

e serve a:

- calcolare in modo efficiente il gradiente $\nabla_{\theta} J(\theta)$;
- attribuire a ciascun parametro una **responsabilità** rispetto all'errore;
- rendere applicabili algoritmi di ottimizzazione basati sul gradiente.

Senza back-propagation, il costo computazionale del calcolo del gradiente crescerebbe in modo proibitivo con la profondità della rete.

Idea concettuale

La back-propagation si fonda interamente sull'applicazione sistematica della regola della catena a funzioni composte. Una rete neurale può infatti essere vista come una funzione ottenuta dalla composizione di molte trasformazioni elementari, ciascuna delle quali dipende dai parametri di un singolo livello.

Consideriamo una funzione scalare J che dipende da una variabile intermedia y , la quale a sua volta dipende da un'altra variabile x . In questo caso, la regola della catena afferma che la variazione di J rispetto a x può essere espressa come il prodotto della variazione di J rispetto a y e della variazione di y rispetto a x . Questo principio elementare è esattamente ciò che la back-propagation applica in modo ricorsivo lungo l'intera rete.

Nel contesto di una rete neurale, la loss function dipende direttamente dall'output del modello, ma solo indirettamente dai parametri dei livelli più profondi. La back-propagation inizia quindi dal calcolo della derivata della loss rispetto all'output della rete. Questa quantità rappresenta l'errore osservato in uscita e costituisce il punto di partenza per la propagazione all'indietro.

Una volta noto come la loss varia rispetto all'output, la regola della catena permette di determinare come tale variazione dipenda dalle attivazioni dell'ultimo livello nascosto. Questo avviene moltiplicando il gradiente in uscita per la derivata della funzione che mappa le attivazioni nascoste nell'output. Il risultato è una misura di quanto ciascuna attivazione nascosta abbia contribuito all'errore finale.

Lo stesso ragionamento viene poi applicato iterativamente ai livelli precedenti. A ogni passaggio, il gradiente viene trasformato attraverso la derivata della funzione di attivazione del livello corrente e attraverso i pesi che collegano il livello al successivo. In questo modo, l'informazione sull'errore viene trasportata all'indietro lungo la rete, mantenendo una corrispondenza precisa tra ogni parametro e il suo contributo alla loss.

Dal punto di vista matematico, la back-propagation non introduce nuove regole di derivazione, ma organizza l'applicazione della regola della catena in modo efficiente e strutturato. Ogni derivata calcolata in un livello viene riutilizzata per il calcolo delle derivate nei livelli precedenti, evitando computazioni ridondanti.

In sintesi, la back-propagation può essere interpretata come un meccanismo che scompone la derivata globale della loss in una sequenza ordinata di derivate locali, rendendo computazionalmente praticabile il calcolo del gradiente in reti neurali profonde.

11.5.1 Come si applica operativamente

L'applicazione della back-propagation avviene sempre in due fasi distinte:

Forward pass

- si propaga l'input attraverso la rete;
- si calcolano tutte le attivazioni intermedie;
- si ottiene l'output finale y ;
- si valuta la loss $J(\theta)$.

Durante il forward pass vengono memorizzate le quantità necessarie al calcolo del gradiente.

Backward pass

- si calcola il gradiente della loss rispetto all'output;
- si propaga il gradiente all'indietro attraverso la rete;
- si applica la regola della catena a ogni livello;
- si ottengono le derivate della loss rispetto a pesi e bias.

Ogni livello riceve:

- l'informazione sull'errore dal livello successivo;
- la combina con la derivata della propria funzione di attivazione;
- produce i gradienti locali dei parametri.

11.5.2 Perché è efficiente

La back-propagation sfrutta:

- la struttura aciclica del grafo computazionale;
- il riutilizzo delle derivate già calcolate;
- un approccio di programmazione dinamica.

Questo riduce drasticamente il numero di operazioni richieste rispetto a un calcolo diretto delle derivate, rendendo l'addestramento di reti profonde computazionalmente praticabile.

11.6 Stochastic Gradient Descent

Una volta calcolato il gradiente della loss function tramite back-propagation, è necessario un algoritmo che utilizzi tale informazione per aggiornare i parametri della rete. Il metodo di ottimizzazione più semplice e fondamentale è lo **Stochastic Gradient Descent (SGD)**.

11.6.1 Idea di base

L'obiettivo è minimizzare la funzione di costo

$$J(\theta)$$

aggiornando iterativamente i parametri nella direzione opposta al gradiente:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta),$$

dove $\eta > 0$ è il **learning rate**, che controlla l'ampiezza del passo di aggiornamento.

Versione stocastica

Nel contesto delle reti neurali, il gradiente esatto sull'intero dataset è spesso troppo costoso da calcolare. SGD utilizza quindi una stima del gradiente calcolata su un singolo esempio o su un piccolo sottoinsieme di dati (mini-batch).

Dato un mini-batch $\{(x^{(i)}, t^{(i)})\}_{i=1}^m$, l'aggiornamento diventa:

$$g = \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} L(f(x^{(i)}; \theta), t^{(i)}), \quad \theta \leftarrow \theta - \eta g.$$

Questa approssimazione introduce rumore nel gradiente, ma rende l'addestramento computazionalmente efficiente e scalabile.

Learning rate

Il learning rate η è un iperparametro critico:

- valori troppo grandi possono causare instabilità o divergenza;
- valori troppo piccoli rallentano drasticamente la convergenza.

In pratica, η può essere mantenuto costante oppure fatto decrescere nel tempo secondo una regola prefissata.

11.6.2 SGD con momentum

Lo **Stochastic Gradient Descent con momentum** introduce un meccanismo che consente di accelerare la convergenza e di ridurre le oscillazioni tipiche dello SGD standard.

Motivazione

Nel caso dello SGD, il gradiente stimato su mini-batch successivi può variare significativamente, soprattutto in presenza di:

- superfici di costo con valli strette e allungate;
- direzioni con curvature molto diverse.

Questo comportamento può rallentare l'addestramento, causando oscillazioni lungo alcune direzioni e progressi lenti lungo altre.

Idea del momentum

Il momentum introduce una variabile ausiliaria, detta **velocità**, che accumula una media esponenziale dei gradienti passati. L'aggiornamento dei parametri non dipende quindi solo dal gradiente corrente, ma anche dalla direzione seguita nelle iterazioni precedenti.

Aggiornamento dei parametri

Siano $\eta > 0$ il learning rate e $\mu \in [0, 1)$ il coefficiente di momentum. Le equazioni di aggiornamento sono:

$$\mathbf{v}^{(k+1)} = \mu \mathbf{v}^{(k)} - \eta \mathbf{g}^{(k)}, \quad \theta^{(k+1)} = \theta^{(k)} + \mathbf{v}^{(k+1)},$$

dove $\mathbf{g}^{(k)}$ è la stima del gradiente al passo k .

11.7 Metodi di regolarizzazione

Le reti neurali profonde hanno un'elevata capacità espressiva, ma questa caratteristica le rende particolarmente soggette a **overfitting**, ovvero a un'eccessiva aderenza ai dati di addestramento a scapito della capacità di generalizzazione.

La **regolarizzazione** comprende un insieme di tecniche volte a ridurre l'errore di generalizzazione, introducendo vincoli o perturbazioni controllate durante l'addestramento.

11.7.1 Regolarizzazione dei parametri

Una forma comune di regolarizzazione consiste nell'aggiungere un termine di penalizzazione alla funzione di costo:

$$\bar{J}(\theta) = J(\theta) + \lambda E_{\text{reg}}(\theta),$$

dove $E_{\text{reg}}(\theta)$ è una funzione che penalizza valori elevati dei parametri e $\lambda > 0$ controlla l'intensità della regolarizzazione.

Un esempio tipico è la penalizzazione della norma dei pesi:

$$E_{\text{reg}}(\theta) = \sum_j |\theta_j|^q,$$

che incoraggia soluzioni più semplici e stabili.

11.7.2 Dataset augmentation

La **dataset augmentation** è una tecnica di regolarizzazione che consiste nel generare nuovi esempi di addestramento a partire da quelli esistenti, applicando trasformazioni che non alterano l'etichetta semantica dei dati.

Nel caso di immagini, esempi tipici di trasformazioni includono rotazioni, traslazioni, ridimensionamenti e variazioni di illuminazione. L'aggiunta di rumore è un'altra forma di augmentation utilizzata in diversi contesti.

Dal punto di vista concettuale, la dataset augmentation:

- aumenta artificialmente la dimensione del dataset;
- riduce l'overfitting;
- induce invarianze desiderate nel modello.

A differenza di altre tecniche di regolarizzazione, la dataset augmentation agisce direttamente sui dati, non sui parametri o sull'algoritmo di ottimizzazione.

11.7.3 Parameter sharing

Il **parameter sharing** è una tecnica di regolarizzazione che impone vincoli sui parametri del modello, forzando diversi componenti della rete a condividere gli stessi pesi.

Questo riduce il numero totale di parametri liberi e limita la complessità effettiva del modello, migliorandone la capacità di generalizzazione.

Il parameter sharing è un elemento centrale nelle **Convolutional Neural Networks (CNN)**, dove lo stesso filtro convoluzionale viene applicato a diverse regioni dell'input. In questo contesto, la condivisione dei parametri:

- riduce drasticamente il numero di parametri;
- introduce invarianza rispetto a traslazioni locali;
- rende l'apprendimento più efficiente e stabile.

Il parameter sharing rappresenta un esempio di come vincoli strutturali, coerenti con la natura dei dati, possano migliorare significativamente le prestazioni di una rete neurale.

11.7.4 Early stopping

L'**early stopping** è una tecnica di regolarizzazione che consiste nell'interrompere l'addestramento prima della convergenza sul training set, al fine di evitare l'overfitting.

Durante l'addestramento, la loss sul training set tende a diminuire monotonamente, mentre la loss sul validation set inizialmente diminuisce ma può successivamente aumentare. Questo comportamento indica che il modello sta iniziando ad adattarsi eccessivamente ai dati di addestramento, perdendo capacità di generalizzazione.

L'early stopping arresta l'ottimizzazione nel punto in cui la loss di validazione raggiunge il minimo, utilizzando quindi un criterio basato su **cross-validation**.

Dal punto di vista concettuale, l'early stopping agisce come una forma di regolarizzazione implicita: limita la complessità effettiva del modello senza modificare esplicitamente la funzione di costo o l'architettura della rete.

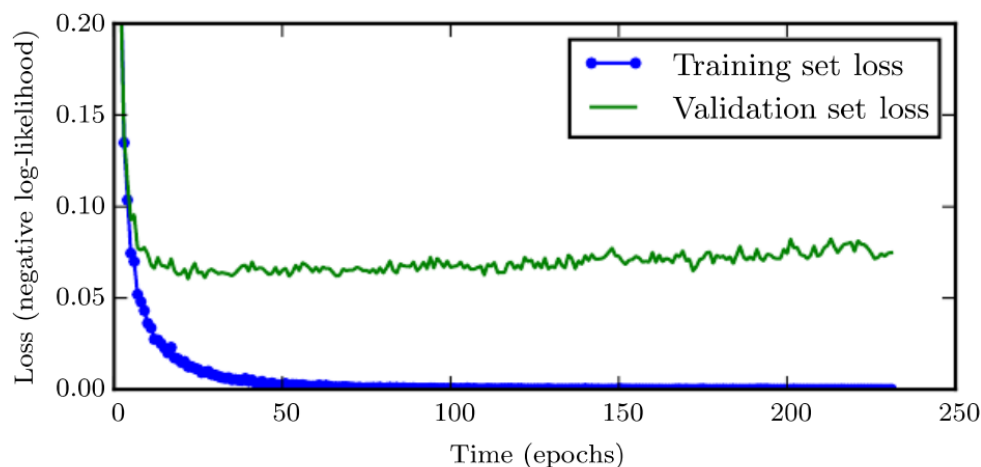


Figure 11.4: Andamento della loss sul training set e sul validation set durante l'addestramento. La training loss decresce monotonamente, mentre la validation loss, dopo una fase iniziale di miglioramento, smette di diminuire e inizia ad aumentare, indicando overfitting. L'early stopping consiste nell'interrompere l'addestramento nel punto di minimo della validation loss.

11.7.5 Dropout

Il **dropout** è una tecnica di regolarizzazione specifica per le reti neurali, che consiste nel disattivare casualmente una frazione delle unità della rete durante l'addestramento.

Ad ogni iterazione, ogni unità viene ignorata con una certa probabilità α . In questo modo, la rete non può fare affidamento su percorsi specifici e viene costretta a distribuire l'informazione su più unità.

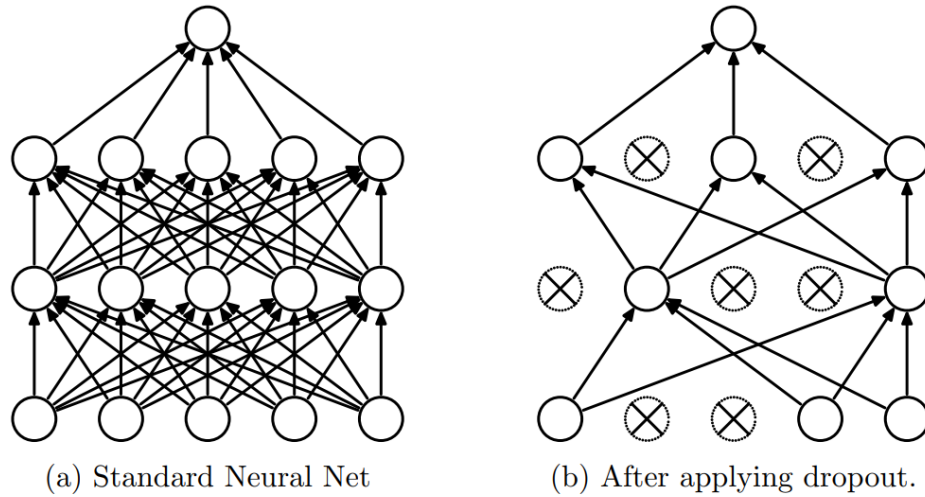


Figure 11.5: Effetto del Dropout sui neuroni

Dal punto di vista concettuale, il dropout:

- riduce la co-adattazione tra neuroni;
- agisce come una forma di ensemble implicito di reti più piccole;
- migliora la robustezza e la generalizzazione del modello.

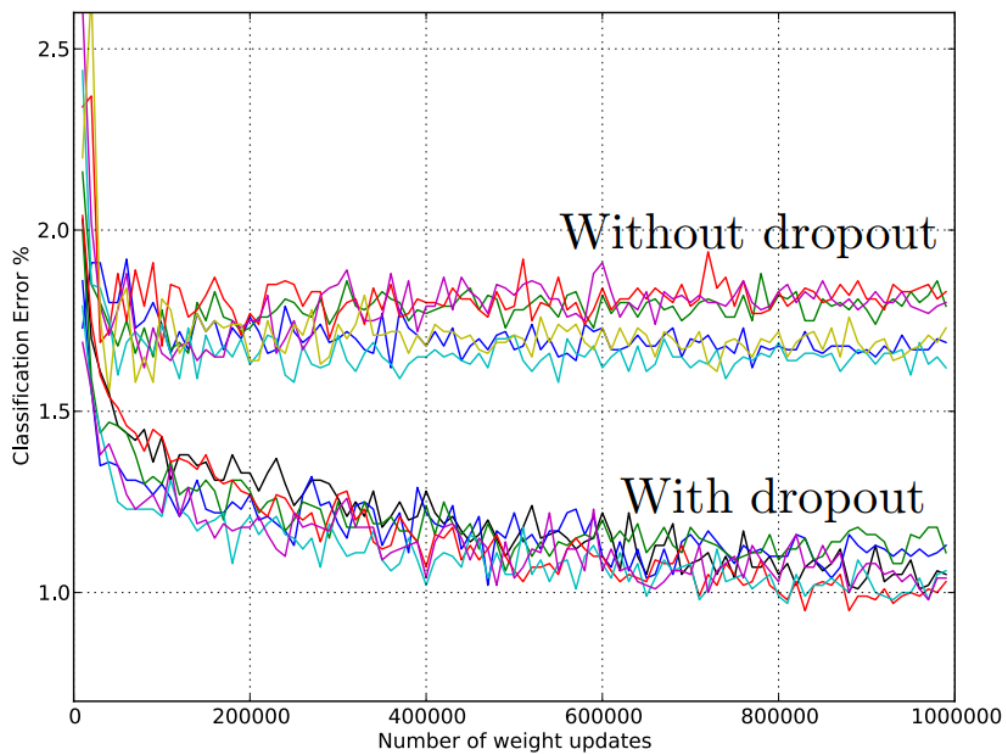


Figure 11.6: Errore di classificazione in funzione del numero di aggiornamenti dei pesi con e senza dropout. Le curve con dropout mostrano un errore finale più basso e una maggiore stabilità rispetto al caso senza dropout.

RIEPILOGO

Artificial Neural Networks

Le **Artificial Neural Networks (ANN)** sono modelli parametrici che approssimano funzioni complesse tramite la composizione di trasformazioni lineari e non lineari. In particolare, le **Feedforward Neural Networks (FNN)** implementano funzioni ottenute come catene di livelli senza cicli, caratterizzate da profondità (depth) e larghezza (width).

Il **Teorema di Approssimazione Universale** garantisce che una rete con almeno un livello nascosto e un numero sufficiente di unità possa approssimare qualsiasi funzione, ma non fornisce indicazioni sull'efficienza, sull'addestrabilità o sulla generalizzazione del modello. In pratica, architetture più **profonde** risultano più efficienti e generalizzano meglio rispetto a reti molto larghe e poco profonde.

Le **funzioni di attivazione** negli strati nascosti introducono non linearità e determinano la capacità espressiva della rete. Funzioni come ReLU sono particolarmente adatte a reti profonde poiché riducono i problemi di saturazione del gradiente.

La rete definisce implicitamente una distribuzione condizionata $p(t | x, \theta)$. La **loss function** deriva dal principio di massima verosimiglianza ed è strettamente legata all'attivazione dello strato di output:

- regressione: output lineare e Mean Squared Error;
- classificazione binaria: sigmoide e Binary Cross-Entropy;
- classificazione multiclasse: softmax e Categorical Cross-Entropy.

Il **calcolo del gradiente** è reso efficiente dall'algoritmo di **back-propagation**, che applica in modo sistematico la regola della catena per propagare l'errore dall'output verso i parametri dei livelli precedenti. La back-propagation non aggiorna i parametri, ma fornisce le informazioni necessarie agli algoritmi di ottimizzazione.

L'ottimizzazione avviene tipicamente tramite **Stochastic Gradient Descent (SGD)** e sue varianti, come **SGD con momentum**, che accelerano la convergenza e riducono le oscillazioni.

A causa dell'elevata capacità dei modelli, le reti neurali sono soggette a overfitting. Per questo motivo vengono utilizzate diverse tecniche di **regolarizzazione**, tra cui:

- penalizzazione dei parametri;
- dropout;
- early stopping;
- dataset augmentation;
- parameter sharing.

Queste tecniche non aumentano la capacità di approssimazione del modello, ma ne controllano la complessità effettiva, migliorando la generalizzazione.

Chapter 12

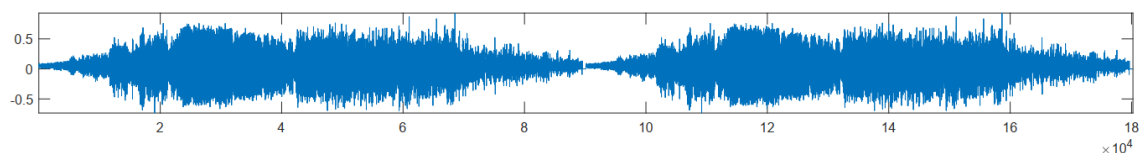
Convolutional Neural Networks

12.1 Motivazioni

Finora, nei modelli di apprendimento automatico e nelle Artificial Neural Networks, gli input sono stati trattati come **vettori di feature generici**. Questa rappresentazione è adeguata in molti contesti, ma risulta limitante quando i dati possiedono una **struttura interna** rilevante.

In numerose applicazioni reali, infatti, gli input sono segnali strutturati, come:

- segnali audio, rappresentabili come vettori monodimensionali di lunghezza variabile;
- immagini, rappresentabili come matrici bidimensionali multi-canale;
- video, rappresentabili come sequenze temporali di immagini multi-canale;
- segnali fisici, ottenuti come campionamenti di grandezze continue.

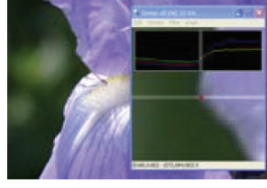


...	0.0468	0.0468	0.0468	0.0390	0.0390	0.0390	0.0546	0.0625	0.0625	0.0390	0.0312	0.0468	0.0625	...
-----	--------	--------	--------	--------	--------	--------	--------	--------	--------	--------	--------	--------	--------	-----

Figure 12.1: Un audio equivale ad avere un tensore 1D

Questi dati non sono semplici vettori, ma **tensori** di diversa dimensionalità, nei quali:

- esiste una nozione di località;
- le relazioni spaziali o temporali tra le componenti sono semanticamente rilevanti;
- le stesse strutture possono comparire in posizioni diverse dell'input.



45	60	98	127	132	133	137	133
46	65	98	123	126	128	131	133
47	65	96	115	119	123	135	137
47	63	91	107	113	122	138	134
50	59	80	97	110	123	133	134
49	53	68	83	97	113	128	133
50	50	58	70	84	102	116	126
50	50	52	58	69	86	101	120

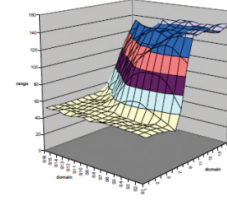


Figure 12.2: Un'immagine equivale ad avere una matrice 2D ed un tensore 3D

L'utilizzo di reti completamente connesse in questi casi presenta due limiti principali:

- un numero di parametri molto elevato;
- l'incapacità di sfruttare esplicitamente la struttura locale dei dati.

Le **Convolutional Neural Networks (CNN)** nascono per superare questi limiti.

12.2 Richiamo alla convoluzione

L'operatore fondamentale alla base delle CNN è la **convoluzione**, un'operazione matematica che combina due funzioni producendo una terza funzione che misura il grado di sovrapposizione tra una funzione e una versione traslata dell'altra.

Convoluzione continua

Date due funzioni continue $x(t)$ e $w(t)$, la loro convoluzione è definita come:

$$(x * w)(t) \equiv \int_{-\infty}^{+\infty} x(a) w(t - a) da.$$

Convoluzione discreta

Nel caso discreto, la convoluzione assume la forma:

$$(x * w)(t) \equiv \sum_{a=-\infty}^{+\infty} x(a) w(t - a).$$

Questa formulazione è particolarmente rilevante per il trattamento di segnali digitali e costituisce la base delle operazioni utilizzate nelle CNN.

Convoluzione bidimensionale

Per segnali bidimensionali, come le immagini, la convoluzione discreta 2D tra un input I e un kernel K è definita come:

$$(I * K)(i, j) \equiv \sum_{m \in S_1} \sum_{n \in S_2} I(m, n) K(i - m, j - n),$$

dove S_1 e S_2 sono insiemi finiti che definiscono il supporto del kernel.

La convoluzione è un'operazione **commutativa**, a patto di effettuare il ribaltamento del kernel:

$$(I * K)(i, j) = \sum_{m \in S_1} \sum_{n \in S_2} I(i - m, j - n) K(m, n).$$

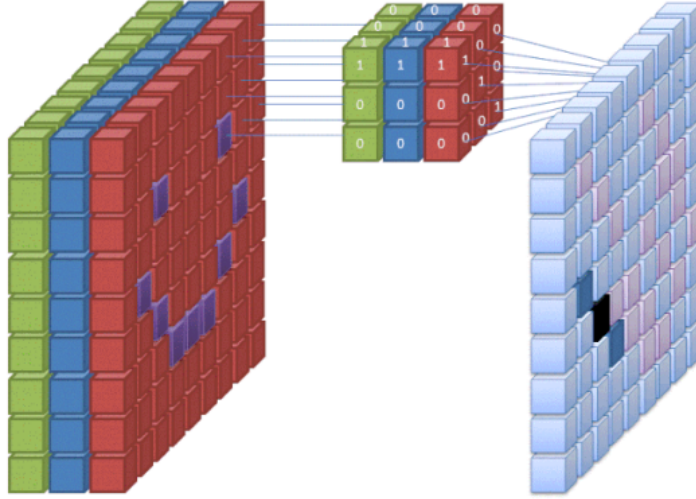


Figure 12.3: Schema del funzionamento di una convoluzione 2D in una Convolutional Neural Network. Un kernel tridimensionale, esteso su tutta la profondità dell'input, viene applicato localmente a diverse regioni dell'immagine. La stessa configurazione di pesi è condivisa tra tutte le posizioni spaziali, producendo una feature map che evidenzia la presenza di uno specifico pattern indipendentemente dalla sua posizione nell'input.

Convoluzione tridimensionale

La convoluzione può essere estesa a segnali di dimensionalità superiore. Nel caso tridimensionale, la convoluzione discreta tra un input I e un kernel K è definita come:

$$(I * K)(i, j, k) \equiv \sum_{m \in S_1} \sum_{n \in S_2} \sum_{u \in S_3} I(i + m, j + n, k + u) K(m, n, u).$$

La convoluzione 3D è utilizzata quando:

- l'input possiede una struttura tridimensionale intrinseca;
- è necessario catturare correlazioni lungo tre dimensioni, ad esempio spazio e tempo.

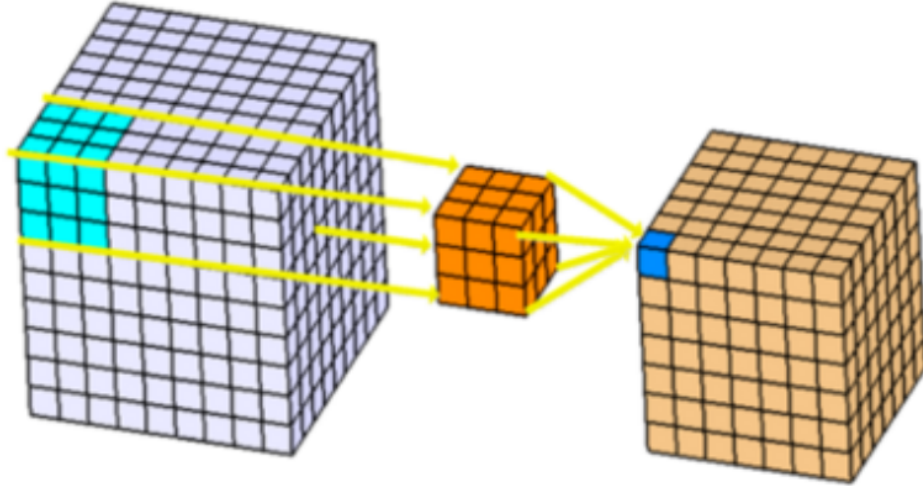


Figure 12.4: Un kernel tridimensionale opera su una regione locale dell'input estendendosi su tutta la profondità, combinando le informazioni provenienti dai diversi canali. Il risultato di questa operazione è un singolo valore della feature map di output, ottenuto come somma delle risposte del kernel sui vari canali.

Esempi tipici includono:

- video, rappresentati come sequenze temporali di immagini;
- dati volumetrici, come immagini medicali (CT, MRI);
- segnali multi-canale con dipendenza spaziale e temporale.

Nel contesto delle CNN per immagini a colori, è comune applicare una convoluzione **2D** su input **3D** (altezza, larghezza, canali), utilizzando kernel tridimensionali che si estendono su tutta la profondità dell'input.

12.2.1 Cross-correlazione

Nella pratica, molte librerie di machine learning implementano una variante detta **cross-correlazione**, definita come:

$$(I * K)(i, j) = \sum_{m \in S_1} \sum_{n \in S_2} I(i + m, j + n) K(m, n).$$

Dal punto di vista computazionale, il fatto che non sia necessario il ribaltamento del Kernel rende l'operazione:

- più semplice da implementare;
- più efficiente in termini di accesso alla memoria;
- meglio ottimizzabile su architetture parallele (GPU/TPU).

Dal punto di vista dell'apprendimento, non vi è alcuna perdita di generalità: poiché i valori del kernel sono parametri allenabili, la rete può apprendere automaticamente anche versioni ribaltate dei filtri, se necessario.

Per questo motivo, nel contesto delle CNN, il termine *convoluzione* viene comunemente utilizzato per indicare l'operazione di cross-correlazione.

12.2.2 Il kernel

Il **kernel** è l'elemento fondamentale alla base delle Convolutional Neural Networks. Esso può essere visto come una piccola matrice di pesi che viene applicata localmente all'input per rilevare specifici pattern.

Nel caso di immagini bidimensionali, un kernel ha tipicamente dimensioni ridotte, ad esempio 3×3 o 5×5 . Quando l'input presenta più canali (ad esempio immagini RGB), il kernel si estende anche lungo la dimensione dei canali, diventando un tensore tridimensionale.

I valori del kernel non sono fissati a priori, ma costituiscono **parametri allenabili** della rete. Durante l'addestramento, il kernel viene ottimizzato affinché risponda in modo elevato alla presenza di uno specifico pattern locale nell'input, come un bordo, un angolo o una texture.

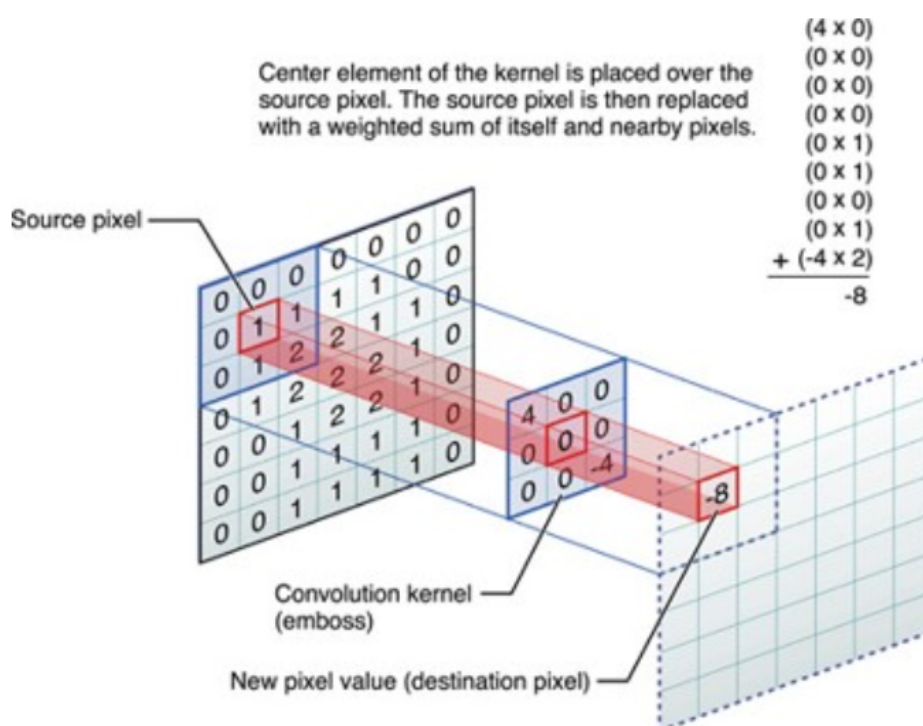


Figure 12.5

12.2.3 La feature map

Una **feature map** è il risultato dell'applicazione di un kernel sull'input tramite l'operazione di convoluzione. Essa può essere vista come una nuova immagine (o matrice) in cui ogni valore rappresenta la risposta del kernel in una specifica posizione dell'input.

Operativamente, mentre il kernel scorre sull'input, in ciascuna posizione viene prodotto un singolo valore di output. L'insieme di tutti questi valori, organizzati secondo la struttura spaziale dell'input, costituisce la feature map.

Ogni feature map è quindi associata a un singolo kernel e indica *dove* il pattern cercato dal kernel è presente nell'input e con quale intensità. Utilizzando più kernel nello stesso convolutional layer, si ottengono più feature map, ciascuna sensibile a un pattern diverso.

12.3 Convolutional Neural Networks

Le **Convolutional Neural Networks (CNN)** sono una classe di reti neurali profonde progettate per elaborare dati strutturati, come segnali temporali, immagini e video. Esse estendono le Feedforward Neural Networks introducendo livelli convoluzionali, che sfruttano operatori locali e parametri condivisi per modellare in modo efficiente le dipendenze spaziali e temporali presenti nei dati.

Formalmente, una CNN è una rete feedforward composta da una sequenza di livelli specializzati, in cui almeno uno è un **convolutional layer**. Ogni livello convoluzionale applica un insieme di kernel all'input, producendo una collezione di **feature map** che rappresentano la presenza di pattern locali nell'input.

A differenza delle reti completamente connesse, nelle CNN:

- le connessioni tra neuroni sono **locali** (sparse connectivity);
- gli stessi parametri sono riutilizzati su diverse regioni dell'input (parameter sharing);
- la struttura spaziale dell'input viene preservata attraverso i livelli.

Queste caratteristiche permettono di ridurre drasticamente il numero di parametri allenabili, migliorando sia l'efficienza computazionale sia la capacità di generalizzazione del modello.

Nel contesto delle immagini, l'input di una CNN è tipicamente rappresentato come un tensore tridimensionale, in cui due dimensioni descrivono la struttura spaziale e la terza rappresenta i canali (ad esempio RGB). Le convoluzioni vengono applicate localmente sulle dimensioni spaziali e globalmente sulla profondità, consentendo alla rete di apprendere filtri sensibili a pattern visivi elementari, come bordi, angoli e texture.

Una CNN è generalmente organizzata come una successione di **convolutional layers**, ciascuno seguito da una funzione di attivazione non lineare e, opzionalmente, da un livello di pooling. I livelli convoluzionali iniziali apprendono feature locali semplici, mentre i livelli più profondi combinano tali feature per rappresentare strutture sempre più astratte.

In sintesi, le Convolutional Neural Networks rappresentano un'evoluzione delle reti neurali classiche, in cui vincoli strutturali coerenti con la natura dei dati permettono di ottenere modelli più efficienti, robusti e scalabili per l'elaborazione di segnali strutturati.

12.4 Convolutional layer

Una **Convolutional Neural Network** può essere vista come una Feedforward Neural Network in cui uno o più livelli sono **convolutional layers**. Un convolutional layer non è una singola operazione, ma una trasformazione composta, articolata in più stadi applicati in sequenza allo stesso input.

In particolare, un convolutional layer è costituito da tre stadi principali:

1. uno stadio di convoluzione (convolution stage);
2. uno stadio di attivazione non lineare;
3. uno stadio di pooling (opzionale).

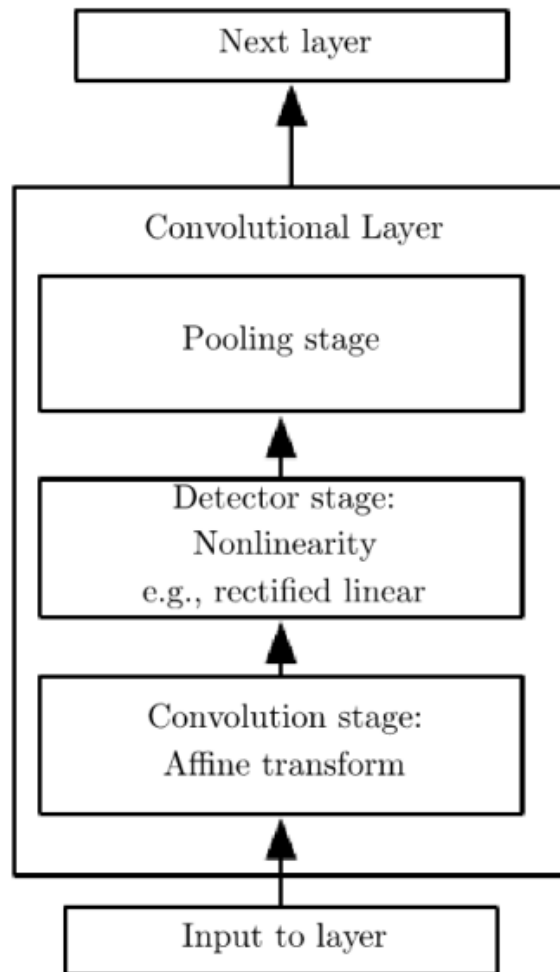


Figure 12.6

12.4.1 Convolution stage

Il **convolution stage** è il primo stadio di un convolutional layer e rappresenta il momento in cui viene applicata l'operazione di convoluzione (o, in pratica, di cross-correlazione) tra l'input e un insieme di kernel.

In questo stadio, ogni kernel viene fatto scorrere sull'input e, in ciascuna posizione, produce un valore di attivazione tramite un prodotto scalare tra i pesi del kernel e una regione locale dell'input. Il risultato è una **feature map**, che indica la risposta del kernel nelle diverse posizioni spaziali.

Una caratteristica fondamentale del convolution stage è la **sparse connectivity**. A differenza delle reti completamente connesse, ogni unità di output dipende solo da un sottoinsieme limitato delle unità di input, definito dal **receptive field** del kernel. Questo significa che ciascun neurone è connesso soltanto a una regione locale dell'input e non all'intero volume di ingresso.

La sparse connectivity è una conseguenza diretta della struttura dell'operazione di convoluzione e non dipende né dalla funzione di attivazione né dallo stadio di pooling. Essa consente di ridurre significativamente il numero di connessioni e di parametri, rendendo il modello più efficiente e meglio adattato a dati con struttura spaziale o temporale.

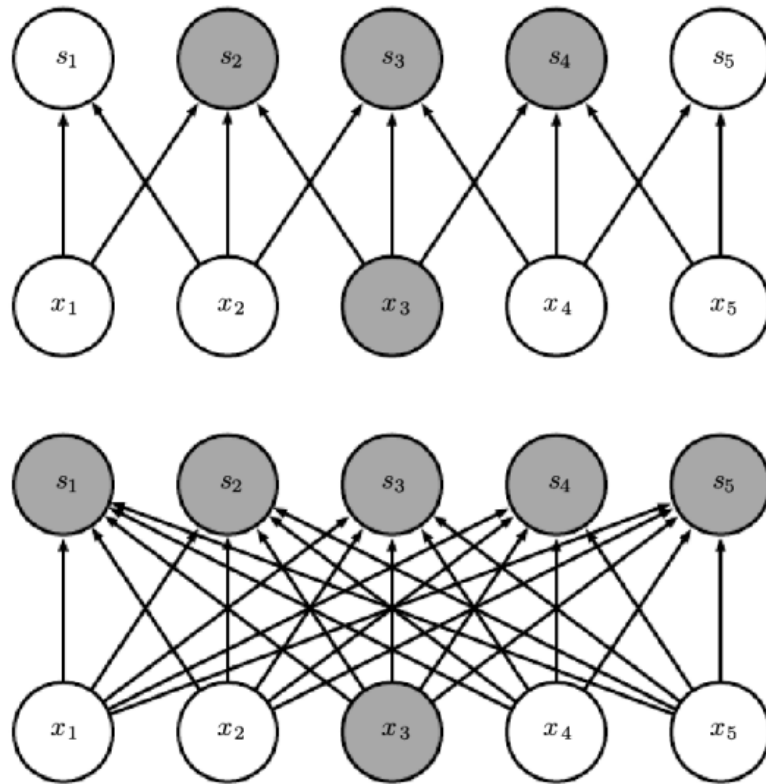


Figure 12.7: Confronto tra la connettività di un livello convoluzionale con quella di un livello completamente connesso. Nella parte superiore è mostrata la **sparse connectivity**: ogni unità di output è connessa solo a un sottoinsieme locale delle unità di input, definito dal receptive field. Nella parte inferiore è invece rappresentata una connessione completamente densa, in cui ogni unità di output dipende da tutte le unità di input. Questo confronto evidenzia come la convoluzione riduca il numero di connessioni e parametri, sfruttando la struttura locale dei dati.

NB: Il receptive field è la porzione dell'input che può influenzare il valore di un neurone nella feature map.

Parameter sharing

Il **parameter sharing** è una proprietà fondamentale del convolution stage e consiste nell'utilizzo degli stessi parametri del kernel in tutte le posizioni dell'input.

Quando un kernel viene fatto scorrere sull'input, i suoi pesi non cambiano da una posizione all'altra. Lo stesso insieme di parametri viene applicato ripetutamente a regioni locali diverse, producendo una feature map. Di conseguenza, tutti i neuroni appartenenti alla stessa feature map condividono gli stessi pesi.

Dal punto di vista concettuale, il parameter sharing implica che la rete non apprende un pattern specifico per ogni posizione dell'input, ma apprende *che tipo di pattern* cercare, indipendentemente da dove esso compaia. Questo rende le CNN naturalmente adatte a dati in cui le stesse strutture possono apparire in posizioni diverse, come nel caso delle immagini.

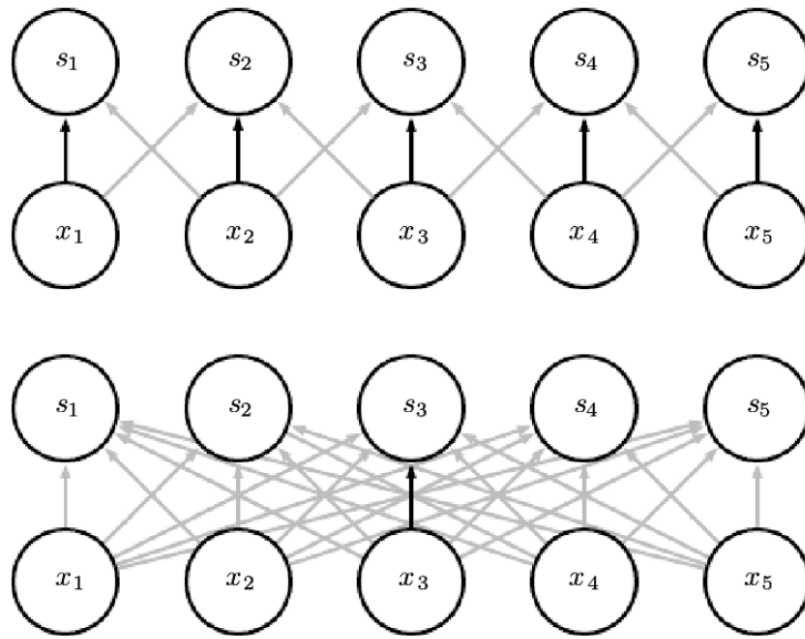


Figure 12.8

Il parameter sharing comporta una drastica riduzione del numero di parametri rispetto alle reti completamente connesse e contribuisce in modo significativo al miglioramento della capacità di generalizzazione. Insieme alla sparse connectivity, esso rappresenta uno dei principali motivi per cui le Convolutional Neural Networks risultano più efficienti ed efficaci nell'elaborazione di dati strutturati.

Padding

Il **padding** è una tecnica utilizzata nel convolution stage che consiste nell'aggiungere valori artificiali ai bordi dell'input prima di applicare la convoluzione. Nella pratica, i valori aggiunti sono tipicamente pari a zero (*zero-padding*).

L'introduzione del padding ha un effetto diretto sulla dimensione della feature map di output. Senza padding, il kernel può essere applicato solo alle posizioni in cui esso è completamente contenuto nell'input, causando una riduzione progressiva delle dimensioni spaziali man mano che si procede nei layer.

Il padding consente invece di:

- controllare la dimensione dell'output;
- preservare l'informazione ai bordi dell'input;
- mantenere costante la risoluzione spaziale attraverso più layer convoluzionali.

In particolare, si distinguono due casi comuni:

- **valid convolution:** nessun padding, con riduzione della dimensione dell'output;
- **same convolution:** padding scelto in modo da ottenere un output con la stessa dimensione spaziale dell'input.

Dal punto di vista concettuale, il padding permette ai neuroni convoluzionali di trattare in modo simmetrico anche le regioni vicine ai bordi dell'input, evitando che tali regioni contribuiscano meno all'apprendimento rispetto alle zone centrali.

Il padding influisce inoltre sul receptive field effettivo dei neuroni, poiché determina quante volte un pixel dell'input può essere incluso nelle operazioni di convoluzione.

NB : Il padding avviene prima della convoluzione vera e propria.

12.4.2 Detector stage

Il **detector stage** è il secondo stadio di un convolutional layer e consiste nell'applicazione di una funzione di attivazione non lineare alle feature map prodotte dal convolution stage.

Dal punto di vista matematico, a ogni valore della feature map viene applicata una funzione

$$g(\cdot),$$

ottenendo una nuova mappa di attivazioni. Una scelta comune per il detector stage è la funzione **ReLU**:

$$g(x) = \max(0, x).$$

Il ruolo principale del detector stage è introdurre **non linearità** nel modello. In assenza di questo stadio, una sequenza di convoluzioni si ridurrebbe a un'unica trasformazione lineare equivalente, limitando drasticamente la capacità espressiva della rete.

Dal punto di vista concettuale, il detector stage può essere interpretato come un meccanismo di rilevazione: un kernel produce una risposta elevata solo quando il pattern che ha appreso è presente nella regione dell'input, e la funzione di attivazione enfatizza tali risposte, attenuando o annullando le attivazioni non rilevanti.

Il detector stage non modifica la struttura spaziale delle feature map, ma ne trasforma i valori, rendendo la rappresentazione più selettiva e adatta alla composizione di più layer convoluzionali.

12.4.3 Pooling stage

Il **pooling stage** è lo stadio finale di un convolutional layer e ha lo scopo di ridurre la dimensionalità spaziale delle feature map, mantenendo al contempo l'informazione più rilevante.

Il pooling opera localmente su ciascuna feature map, aggregando i valori all'interno di regioni spaziali limitate. A differenza del convolution stage, il pooling non introduce nuovi parametri allenabili, ma applica una trasformazione deterministica.

Le due operazioni di pooling più comuni sono:

- **max pooling**, che seleziona il valore massimo all'interno della regione considerata;
- **average pooling**, che calcola la media dei valori nella regione.

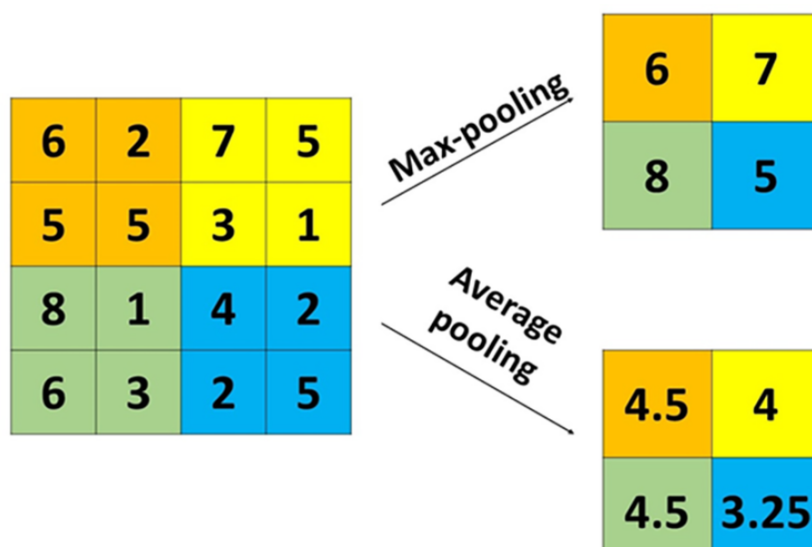


Figure 12.9: Il max pooling seleziona la risposta più forte in una regione, enfatizzando la presenza di un pattern, mentre l'average pooling ne fornisce una misura media dell'intensità.

Dal punto di vista concettuale, il pooling:

- riduce la risoluzione spaziale delle feature map;
- diminuisce il costo computazionale dei layer successivi;
- introduce una certa invarianza a piccole traslazioni dell'input.

Riducendo la dimensionalità delle rappresentazioni intermedie, il pooling contribuisce anche a limitare l'overfitting e a rendere la rete più robusta rispetto a variazioni locali dell'input.

È importante notare che il pooling agisce solo sulle dimensioni spaziali delle feature map e non sulla loro profondità: il numero di feature map rimane invariato.

12.4.4 Dimensione delle feature map e numero di parametri

Si consideri un input tridimensionale di dimensioni

$$w_{in} \times h_{in} \times d_{in},$$

dove w_{in} e h_{in} rappresentano le dimensioni spaziali e d_{in} il numero di canali. Si considerino inoltre d_{out} kernel convoluzionali di dimensioni

$$w_k \times h_k \times d_{in},$$

uno per ciascuna feature map di output, con stride s e padding p .

12.5 Dimensione della feature map di output

Le dimensioni spaziali delle feature map di output sono date da:

$$w_{out} = \frac{w_{in} - w_k + 2p}{s} + 1, \quad h_{out} = \frac{h_{in} - h_k + 2p}{s} + 1.$$

Il numero di feature map in output è pari al numero di kernel utilizzati, ovvero d_{out} . Di conseguenza, l'output del convolutional layer è un tensore di dimensioni:

$$w_{out} \times h_{out} \times d_{out}.$$

Numero di parametri allenabili

Il numero di parametri allenabili di un convolutional layer dipende esclusivamente dalle dimensioni dei kernel e dal loro numero, e non dalla dimensione spaziale dell'input.

Ogni kernel contiene:

$$w_k \cdot h_k \cdot d_{in}$$

pesi, e a ciascun kernel è associato un termine di bias.

Poiché i kernel sono d_{out} , il numero totale di parametri allenabili è:

$$|\theta| = w_k \cdot h_k \cdot d_{in} \cdot d_{out} + d_{out}.$$

Questo risultato evidenzia uno dei principali vantaggi delle Convolutional Neural Networks: grazie al **parameter sharing**, il numero di parametri cresce linearmente con il numero di kernel e non con la dimensione dell'input, a differenza delle reti completamente connesse.

RIEPILOGO

Convolutional Neural Networks

Le **Convolutional Neural Networks (CNN)** sono una classe di reti neurali feed-forward progettate per elaborare dati strutturati, come immagini, segnali e video. Esse sfruttano vincoli architetturali coerenti con la struttura dei dati, ottenendo modelli più efficienti e con migliore capacità di generalizzazione rispetto alle reti completamente connesse.

L'operatore fondamentale delle CNN è la **convoluzione**, che applica un kernel locale all'input producendo una **feature map**. Nella pratica, viene utilizzata la cross-correlazione, equivalente dal punto di vista dell'apprendimento e più efficiente dal punto di vista computazionale.

Il **kernel** è un tensore di parametri allenabili che definisce un operatore lineare locale. Applicando lo stesso kernel in tutte le posizioni dell'input si realizza il principio di **parameter sharing**, che consente di rilevare lo stesso pattern indipendentemente dalla sua posizione e riduce drasticamente il numero di parametri.

Ogni valore di una feature map dipende solo da una regione locale dell'input, detta **receptive field**. Questa proprietà, nota come **sparse connectivity**, permette alle CNN di sfruttare le relazioni locali presenti nei dati e contribuisce all'efficienza del modello.

Un **convolutional layer** è composto da tre stadi principali:

- **convolution stage**, in cui i kernel vengono applicati all'input;

- **detector stage**, che introduce non linearità tramite funzioni di attivazione (tipicamente ReLU);
- **pooling stage**, che riduce la dimensionalità spaziale delle feature map e introduce invarianza a piccole traslazioni.

Il **padding** consente di controllare la dimensione delle feature map e di preservare l'informazione ai bordi dell'input, mentre lo **stride** determina di quanto il kernel si sposta sull'input durante la convoluzione.

La dimensione delle feature map di output dipende da input, kernel, padding e stride, mentre il numero di parametri allenabili di un convolutional layer dipende esclusivamente dalle dimensioni dei kernel e dal loro numero, grazie al parameter sharing.

Infine, il **transfer learning** permette di riutilizzare CNN pre-addestrate su grandi dataset, sfruttando le feature generiche apprese nei layer convoluzionali iniziali per risolvere nuovi compiti con meno dati e minore costo computazionale.

Le CNN rappresentano quindi un'estensione strutturata delle reti neurali classiche, particolarmente efficace per l'elaborazione di dati con struttura spaziale o temporale.

Chapter 13

Unsupervised Learning

Nel contesto del *Machine Learning*, si parla di *Unsupervised Learning* quando il processo di apprendimento avviene in assenza di etichette associate ai dati. Formalmente, è disponibile un insieme di dati di input

$$\mathcal{D} = \{\mathbf{x}_n\}_{n=1}^N,$$

ma non sono noti i corrispondenti valori target.

L'obiettivo principale dell'apprendimento non supervisionato non è la predizione di una variabile di uscita, bensì la scoperta di *strutture latenti* nei dati. Tra i problemi tipici rientrano:

- il **clustering**, ovvero l'individuazione di gruppi omogenei di dati;
- la **modellazione della distribuzione dei dati**, utile anche come fase preliminare a metodi supervisionati;
- la **riduzione della dimensionalità** e l'analisi di variabili latenti.

13.0.1 Cluster

Nel contesto dell'apprendimento non supervisionato, un *cluster* è un insieme di dati che presentano caratteristiche simili tra loro e differenti rispetto ai dati appartenenti ad altri gruppi. L'idea di clustering nasce dall'osservazione che i dati non sono distribuiti in modo uniforme nello spazio delle feature, ma tendono a concentrarsi in regioni ben definite.

Dal punto di vista geometrico, un cluster può essere interpretato come una regione dello spazio in cui la densità dei dati è elevata. Nel caso dei Gaussian Mixture Models, ciascun cluster è modellato tramite una distribuzione gaussiana, che ne descrive la posizione (media) e la forma (covarianza).

13.0.2 Variabili latenti

Durante il processo di generazione dei dati, ogni osservazione è prodotta da una specifica componente del modello. Tuttavia, nell'apprendimento non supervisionato, questa informazione non è osservabile direttamente.

Per rappresentare tale informazione nascosta si introducono le *variabili latenti*. Una variabile latente è una variabile non osservata che influenza la generazione dei dati osservabili.

L'introduzione delle variabili latenti consente di:

- modellare esplicitamente l'appartenenza dei dati ai cluster;
- passare da un clustering puramente geometrico a un clustering probabilistico;
- formulare il problema dell'apprendimento come un problema di inferenza statistica.

13.1 Gaussian Mixture Models

Un *Gaussian Mixture Model* (GMM) è un modello probabilistico che assume che i dati osservati siano generati da una combinazione di più distribuzioni gaussiane. La distribuzione di probabilità complessiva è definita come una combinazione convessa di K componenti gaussiane:

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k),$$

dove:

- π_k è la probabilità a priori della componente k -esima, con $\pi_k \geq 0$ e $\sum_{k=1}^K \pi_k = 1$;
- $\boldsymbol{\mu}_k$ è il vettore media della k -esima gaussiana;
- $\boldsymbol{\Sigma}_k$ è la matrice di covarianza associata.

Il GMM è un *modello generativo*: descrive esplicitamente il processo attraverso cui i dati vengono prodotti. In particolare, la generazione di un singolo campione $\mathbf{x}_n$ può essere descritta in due fasi:

1. selezione di una componente k secondo la distribuzione discreta definita dai pesi $\{\pi_1, \dots, \pi_K\}$;
2. generazione del dato $\mathbf{x}_n$ campionando dalla distribuzione gaussiana

$$\mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k).$$

Nel contesto dell'apprendimento non supervisionato, il problema consiste nello stimare i parametri del modello

$$\{\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\}_{k=1}^K$$

a partire dai soli dati osservati $\mathcal{D}$, senza conoscere quale componente gaussiana abbia generato ciascun campione.

13.1.1 Generazione dei dati con un Gaussian Mixture Model

Un *Gaussian Mixture Model* definisce un processo generativo esplicito per i dati osservati. L'ipotesi fondamentale è che ogni osservazione $\mathbf{x}_n$ sia generata da una tra K distribuzioni gaussiane, selezionata in modo casuale secondo una distribuzione discreta sui componenti.

Il processo generativo può essere descritto come segue:

1. si estrae una variabile discreta $k \in \{1, \dots, K\}$ secondo la distribuzione

$$P(k) = \pi_k;$$

2. dato il valore di k , si genera il campione $\mathbf{x}_n$ dalla distribuzione gaussiana associata:

$$\mathbf{x}_n \sim \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k).$$

Ripetendo questo processo N volte si ottiene un dataset

$$\mathcal{D} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\},$$

che risulta composto da più gruppi di dati, ciascuno caratterizzato da una propria media, covarianza e peso probabilistico.

Dal punto di vista geometrico, ciascuna componente gaussiana genera un *cluster* con forma ellittica nello spazio delle feature, la cui orientazione e dispersione sono determinate dalla matrice di covarianza $\boldsymbol{\Sigma}_k$. Un esempio bidimensionale di dati generati da un GMM con due componenti è mostrato in Figura 13.1.

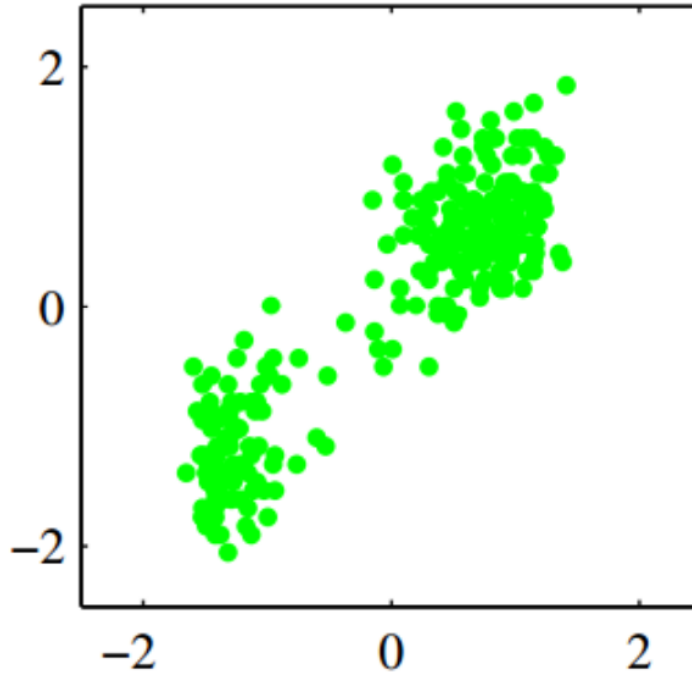


Figure 13.1: Esempio di dati generati da un Gaussian Mixture Model con due componenti. Ogni cluster corrisponde a una diversa distribuzione gaussiana con media e covarianza proprie.

È importante sottolineare che, nel contesto dell'apprendimento non supervisionato, il dataset osservato contiene esclusivamente i campioni $\mathbf{x}_n$, mentre l'informazione relativa a quale componente gaussiana abbia generato ciascun dato non è osservabile. Questa informazione nascosta verrà modellata introducendo opportune *variabili latenti*.

13.1.2 Introduzione formale delle variabili latenti nel GMM

Per rendere esplicita l'appartenenza di ciascun dato a un cluster, nel Gaussian Mixture Model si introducono delle variabili latenti discrete. Per ogni osservazione $\mathbf{x}_n$ si definisce una variabile latente

$$\mathbf{z}_n = (z_{n1}, \dots, z_{nK})^T,$$

dove ciascuna componente z_{nk} è una variabile binaria tale che:

$$z_{nk} \in \{0, 1\}, \quad \sum_{k=1}^K z_{nk} = 1.$$

Questa rappresentazione è nota come *codifica 1-out-of-K* e garantisce che ogni dato sia associato a una e una sola componente del modello.

La variabile latente $z_{nk} = 1$ indica che il campione $\mathbf{x}_n$ è stato generato dalla k -esima distribuzione gaussiana.

13.1.3 Distribuzione a priori delle variabili latenti

Si assume che le variabili latenti siano distribuite secondo una distribuzione categorica parametrizzata dai pesi del modello:

$$P(z_{nk} = 1) = \pi_k,$$

con $\pi_k \geq 0$ e $\sum_{k=1}^K \pi_k = 1$. La distribuzione congiunta della variabile latente $\mathbf{z}_n$ può quindi essere scritta come:

$$P(\mathbf{z}_n) = \prod_{k=1}^K \pi_k^{z_{nk}}.$$

13.1.4 Distribuzione condizionata dei dati

Dato il valore della variabile latente, la distribuzione del dato osservato è gaussiana:

$$P(\mathbf{x}_n \mid z_{nk} = 1) = \mathcal{N}(\mathbf{x}_n; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k).$$

Utilizzando la codifica 1-out-of-K, la distribuzione condizionata può essere espressa in forma compatta come:

$$P(\mathbf{x}_n \mid \mathbf{z}_n) = \prod_{k=1}^K \mathcal{N}(\mathbf{x}_n; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)^{z_{nk}}.$$

13.1.5 Distribuzione congiunta e marginale

Applicando la regola della catena, la distribuzione congiunta delle variabili osservate e latenti è:

$$P(\mathbf{x}_n, \mathbf{z}_n) = P(\mathbf{x}_n \mid \mathbf{z}_n)P(\mathbf{z}_n).$$

La distribuzione marginale dei dati osservati si ottiene marginalizzando rispetto alle variabili latenti:

$$P(\mathbf{x}_n) = \sum_{\mathbf{z}_n} P(\mathbf{x}_n, \mathbf{z}_n) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_n; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k),$$

che coincide con l'espressione del Gaussian Mixture Model.

13.1.6 Distribuzione a posteriori delle variabili latenti

Dato un campione osservato $\mathbf{x}_n$, è possibile calcolare la distribuzione a posteriori delle variabili latenti tramite il teorema di Bayes:

$$\gamma(z_{nk}) \equiv P(z_{nk} = 1 \mid \mathbf{x}_n) = \frac{\pi_k \mathcal{N}(\mathbf{x}_n; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}_n; \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}.$$

La quantità $\gamma(z_{nk})$ rappresenta la *responsabilità* della k -esima componente nel generare il dato $\mathbf{x}_n$ ed esprime un'assegnazione probabilistica del campione al cluster.

Interpretazione grafica di un Gaussian Mixture Model Un Gaussian Mixture Model descrive i dati come generati da una combinazione (lineare) di k distribuzioni gaussiane. Ogni componente gaussiana rappresenta una possibile sorgente dei dati, caratterizzata da una media e da una matrice di covarianza.

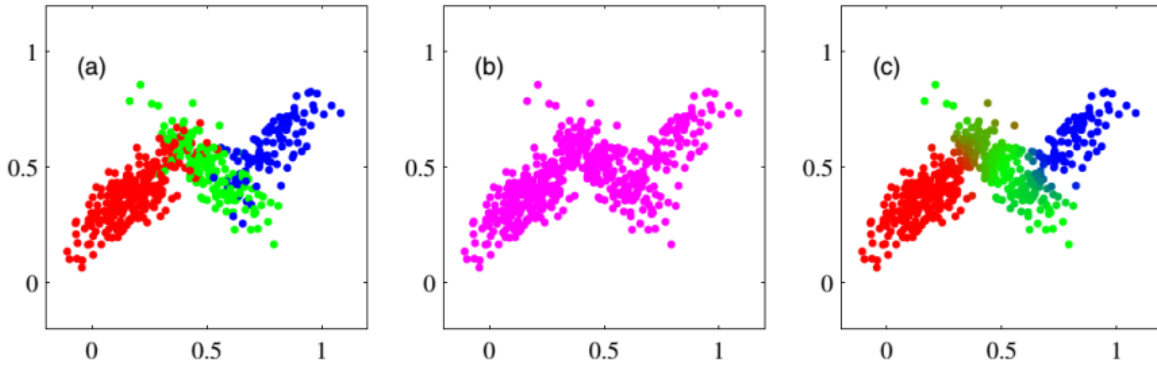


Figure 13.2: Rappresentazione grafica del funzionamento di un Gaussian Mixture Model. (a) Inizializzazione delle componenti gaussiane. (b) Distribuzione dei dati senza assegnazione esplicita ai cluster. (c) Assegnazione probabilistica delle osservazioni alle componenti del modello.

La Figura illustra il comportamento di un GMM in tre fasi concettuali.

Nel pannello (a) sono mostrate più componenti gaussiane inizializzate nello spazio dei dati. Ogni colore rappresenta una diversa gaussiana, definita da una media e da una covarianza. In questa fase le componenti sono solo ipotesi sulla struttura dei dati.

Nel pannello (b) si osservano i dati nel loro insieme, senza una suddivisione esplicita in cluster. A differenza di metodi come il K-means, nel GMM non esiste un'assegnazione rigida dei punti a un singolo cluster.

Nel pannello (c) è mostrato il risultato del modello appreso: ogni osservazione è associata probabilisticamente alle diverse componenti gaussiane. I colori rappresentano la componente con probabilità a posteriori maggiore per ciascun punto, ma ogni osservazione contribuisce in realtà a tutte le componenti in misura diversa.

Ruolo del parametro k

Nel Gaussian Mixture Model, il parametro k indica il numero di componenti gaussiane che costituiscono il modello. A differenza del K-means, in cui k rappresenta il numero di cluster rigidi, nel GMM k rappresenta il numero di distribuzioni probabilistiche utilizzate per modellare i dati.

Ogni componente gaussiana non corrisponde necessariamente a un cluster ben separato, ma contribuisce alla distribuzione complessiva dei dati. Valori diversi di k permettono di modellare strutture più o meno complesse:

- un valore di k piccolo produce un modello più semplice;
- un valore di k grande consente di approssimare distribuzioni più complesse, ma aumenta il rischio di overfitting.

Il valore di k è quindi un iperparametro del modello e deve essere scelto in base al compromesso tra capacità di rappresentazione e generalizzazione.

13.2 K-means Clustering

L'algoritmo *K-means* è uno dei metodi più semplici e diffusi per il clustering non supervisionato. Dato un insieme di dati

$$\mathcal{D} = \{\mathbf{x}_n\}_{n=1}^N$$

e un numero fissato di cluster K , l'obiettivo di K-means è suddividere i dati in K gruppi minimizzando la distanza tra ciascun punto e il centro del cluster a cui è assegnato.

13.2.1 Obiettivo dell'algoritmo

K-means mira a minimizzare la seguente funzione obiettivo:

$$J = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2,$$

dove:

- $\boldsymbol{\mu}_k$ è il centroide del cluster k ;
- $r_{nk} \in \{0, 1\}$ è una variabile di assegnazione tale che $r_{nk} = 1$ se il punto $\mathbf{x}_n$ appartiene al cluster k e 0 altrimenti;
- per ogni n , vale il vincolo $\sum_{k=1}^K r_{nk} = 1$.

Questo formalizza un *hard clustering*: ogni dato appartiene a un solo cluster.

13.2.2 Algoritmo K-means

L'algoritmo procede in modo iterativo alternando due fasi:

Fase di assegnazione Dato un insieme corrente di centroidi $\{\boldsymbol{\mu}_k\}$, ogni punto viene assegnato al cluster il cui centroide è più vicino:

$$r_{nk} = \begin{cases} 1 & \text{se } k = \arg \min_j \|\mathbf{x}_n - \boldsymbol{\mu}_j\|^2, \\ 0 & \text{altrimenti.} \end{cases}$$

Fase di aggiornamento Una volta fissate le assegnazioni, i centroidi vengono ricalcolati come media dei punti appartenenti al cluster:

$$\boldsymbol{\mu}_k = \frac{1}{N_k} \sum_{n=1}^N r_{nk} \mathbf{x}_n, \quad N_k = \sum_{n=1}^N r_{nk}.$$

Queste due fasi vengono ripetute fino a convergenza, ovvero fino a quando le assegnazioni non cambiano più oppure la funzione obiettivo non diminuisce ulteriormente.

13.2.3 Interpretazione geometrica

La Figura 13.3 mostra l'evoluzione dell'algoritmo K-means nel caso bidimensionale con $K = 2$: a partire da centroidi iniziali arbitrari, le assegnazioni e i centroidi vengono aggiornati iterativamente fino alla convergenza.

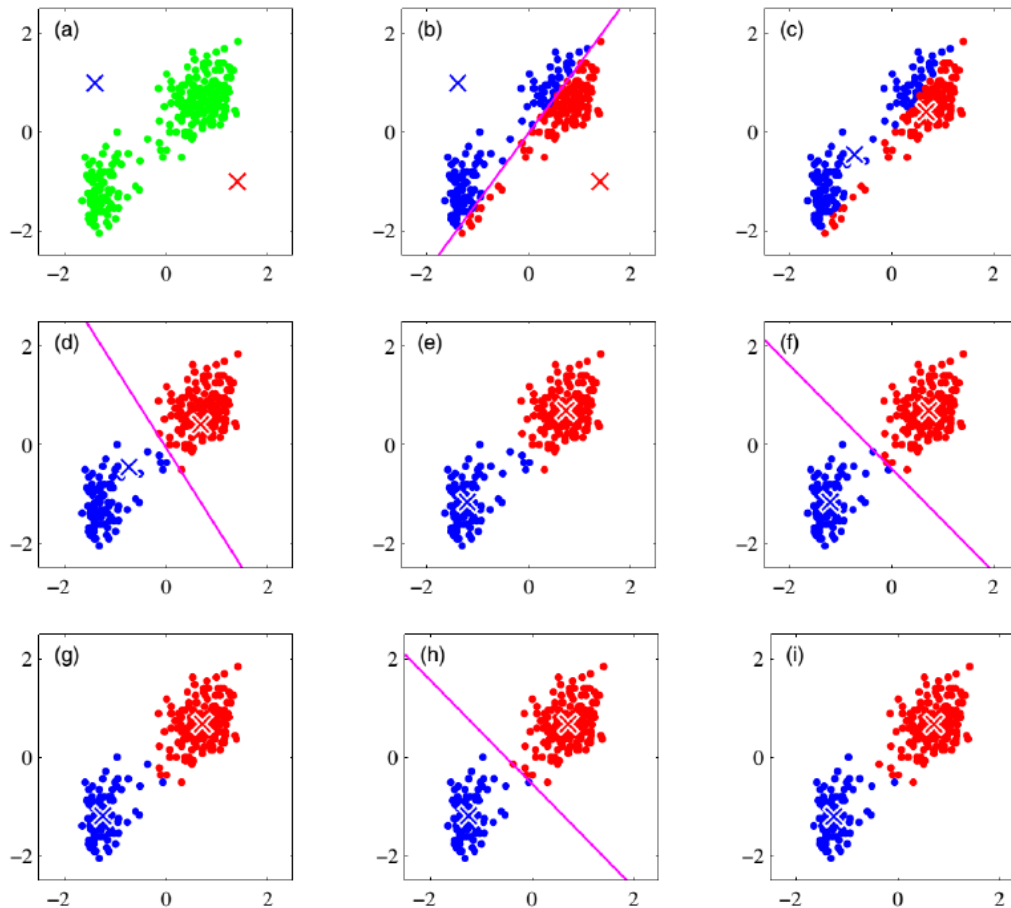


Figure 13.3: Evoluzione dell'algoritmo K-means. I punti sono assegnati ai cluster in base alla distanza dal centroide corrente; le linee indicano i confini di decisione. Le croci rappresentano i centroidi, che vengono aggiornati a ogni iterazione fino alla convergenza.

Dal punto di vista geometrico, K-means costruisce una partizione dello spazio in regioni di Voronoi, ciascuna associata a un centroide. I confini tra i cluster sono iperpiani determinati dalla distanza euclidea dai centroidi.

Osservazioni

K-means è un algoritmo semplice ed efficiente, ma presenta alcune limitazioni:

- il numero di cluster K deve essere fissato a priori;
- il risultato dipende fortemente dall'inizializzazione dei centroidi;
- il clustering è basato esclusivamente sulla distanza euclidea;
- i cluster ottenuti hanno forma approssimativamente sferica.

Queste limitazioni motivano l'introduzione di modelli probabilistici più generali, come i Gaussian Mixture Models, e di algoritmi di stima più flessibili, come l'Expectation Maximization.

13.3 Expectation Maximization (EM)

Nel caso dei Gaussian Mixture Models, il problema dell'apprendimento consiste nello stimare i parametri

$$\Theta = \{\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\}_{k=1}^K$$

a partire da un insieme di dati osservati

$$\mathcal{D} = \{\mathbf{x}_n\}_{n=1}^N,$$

in presenza di variabili latenti non osservabili $\mathbf{z}_n$.

13.3.1 Massima verosimiglianza con variabili latenti

L'obiettivo è massimizzare la log-verosimiglianza dei dati osservati:

$$\ln p(\mathcal{D} \mid \Theta) = \sum_{n=1}^N \ln \left(\sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_n; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right).$$

La presenza della somma all'interno del logaritmo rende la massimizzazione diretta di questa funzione un problema non banale. Questa difficoltà è dovuta al fatto che le variabili latenti $\mathbf{z}_n$, che indicano quale componente abbia generato ciascun dato, non sono osservabili.

13.3.2 Idea dell'algoritmo EM

L'algoritmo *Expectation Maximization* affronta il problema alternando due fasi:

- una fase di **stima** delle variabili latenti, data una stima corrente dei parametri;
- una fase di **aggiornamento** dei parametri, data la distribuzione stimata delle variabili latenti.

L'algoritmo procede quindi per ottimizzazione iterativa, migliorando la log-verosimiglianza dei dati a ogni iterazione.

13.3.3 E-step (Expectation step)

Dato un insieme corrente di parametri $\Theta^{(t)}$, si calcola la distribuzione a posteriori delle variabili latenti:

$$\gamma(z_{nk})^{(t+1)} \equiv P(z_{nk} = 1 \mid \mathbf{x}_n, \Theta^{(t)}) = \frac{\pi_k^{(t)} \mathcal{N}(\mathbf{x}_n; \boldsymbol{\mu}_k^{(t)}, \boldsymbol{\Sigma}_k^{(t)})}{\sum_{j=1}^K \pi_j^{(t)} \mathcal{N}(\mathbf{x}_n; \boldsymbol{\mu}_j^{(t)}, \boldsymbol{\Sigma}_j^{(t)})}.$$

Questa quantità rappresenta la *responsabilità* della k -esima componente nella generazione del dato $\mathbf{x}_n$. A differenza di K-means, l'assegnazione ai cluster è ora *soft* e probabilistica.

13.3.4 M-step (Maximization step)

Utilizzando le responsabilità $\gamma(z_{nk})^{(t+1)}$, si aggiornano i parametri del modello:

Aggiornamento delle medie

$$\boldsymbol{\mu}_k^{(t+1)} = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk})^{(t+1)} \mathbf{x}_n, \quad N_k = \sum_{n=1}^N \gamma(z_{nk})^{(t+1)}.$$

Aggiornamento delle covarianze

$$\boldsymbol{\Sigma}_k^{(t+1)} = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk})^{(t+1)} (\mathbf{x}_n - \boldsymbol{\mu}_k^{(t+1)})(\mathbf{x}_n - \boldsymbol{\mu}_k^{(t+1)})^T.$$

Aggiornamento dei pesi

$$\pi_k^{(t+1)} = \frac{N_k}{N}.$$

13.3.5 Algoritmo EM

L'algoritmo EM può essere riassunto come segue:

1. inizializzazione dei parametri $\Theta^{(0)}$;
2. ripetizione alternata di E-step e M-step;
3. arresto quando la log-verosimiglianza converge o dopo un numero prefissato di iterazioni.

L'algoritmo converge a un massimo locale della log-verosimiglianza dei dati osservati.

13.3.6 Relazione con K-means

L'algoritmo EM per i Gaussian Mixture Models può essere visto come una generalizzazione probabilistica di K-means. In particolare:

- K-means utilizza assegnazioni hard, mentre EM utilizza assegnazioni soft;
- K-means assume cluster sferici e varianze identiche;
- EM permette cluster ellittici e di diversa dispersione.

13.3.7 Interpretazione dell'algoritmo EM

Il comportamento dell'algoritmo EM può essere compreso osservando l'evoluzione iterativa delle distribuzioni gaussiane stimate. La Figura 13.4 mostra un esempio bidimensionale di apprendimento di un GMM con due componenti.

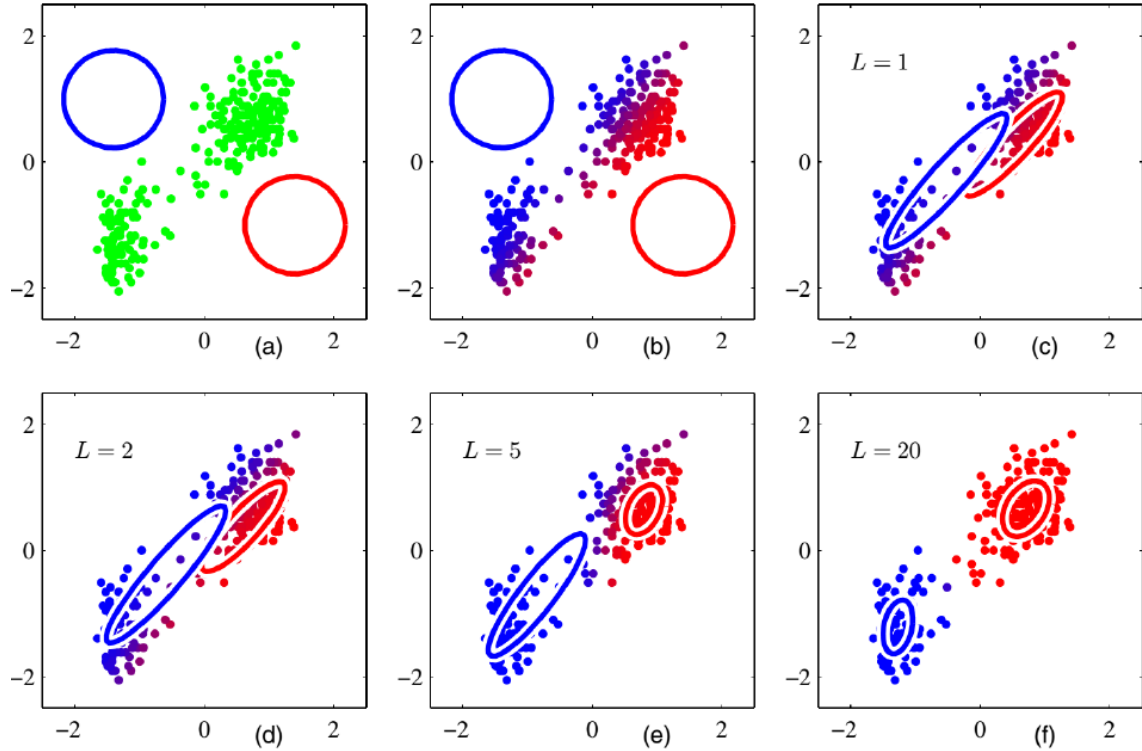


Figure 13.4: Evoluzione dell'algoritmo EM per un Gaussian Mixture Model con due componenti. Le ellissi rappresentano le curve di livello delle distribuzioni gaussiane stimate. Il numero L indica l'iterazione dell'algoritmo.

Nella fase iniziale (Figura (a)), i parametri del modello sono inizializzati in modo approssimativo: le gaussiane non riflettono ancora la reale struttura dei dati. Durante le prime iterazioni (Figure (b) e (c)), l'algoritmo EM alterna:

- una stima probabilistica dell'appartenenza dei punti ai cluster (E-step);
- un aggiornamento dei parametri delle gaussiane (M-step).

A ogni iterazione L , le ellissi si deformano e si spostano per adattarsi progressivamente alla distribuzione dei dati. In particolare:

- le medie si muovono verso le regioni di maggiore densità;
- le covarianze catturano orientazione e dispersione dei cluster;
- le assegnazioni diventano via via più nette.

Dopo un numero sufficiente di iterazioni (Figure (d), (e) e (f)), il modello converge a una configurazione stabile in cui ciascuna componente gaussiana descrive correttamente uno dei cluster presenti nei dati.

Questa rappresentazione evidenzia una differenza fondamentale rispetto a K-means: mentre K-means aggiorna solo i centroidi e produce cluster sferici, EM apprende anche la forma, l'orientazione e la dispersione dei cluster attraverso le matrici di covarianza.

13.4 General Expectation Maximization Problem

L'algoritmo Expectation Maximization non è limitato ai Gaussian Mixture Models, ma rappresenta un metodo generale per la stima dei parametri di modelli probabilistici in presenza di variabili latenti.

13.4.1 Definizione del problema

Si consideri:

- un insieme di dati osservati

$$X = \{\mathbf{x}_1, \dots, \mathbf{x}_N\};$$

- un insieme di variabili latenti non osservate

$$Z = \{\mathbf{z}_1, \dots, \mathbf{z}_N\};$$

- un modello probabilistico parametrizzato

$$p(\mathbf{y} \mid \boldsymbol{\theta}), \quad \text{dove } \mathbf{y}_n = (\mathbf{x}_n, \mathbf{z}_n)$$

rappresenta il *dato completo*.

L'obiettivo è determinare il valore dei parametri

$$\boldsymbol{\theta}^*$$

che massimizza la log-verosimiglianza dei dati osservati:

$$\boldsymbol{\theta}^* = \arg \max_{\boldsymbol{\theta}} \ln p(X \mid \boldsymbol{\theta}).$$

Poiché le variabili latenti Z non sono osservabili, la massimizzazione diretta della verosimiglianza risulta in generale complessa o intrattabile.

13.4.2 Funzione ausiliaria Q

L'algoritmo EM introduce una funzione ausiliaria

$$Q(\boldsymbol{\theta}' \mid \boldsymbol{\theta}) = \mathbb{E}_{Z|X, \boldsymbol{\theta}} [\ln p(X, Z \mid \boldsymbol{\theta}')],$$

che rappresenta il valore atteso della log-verosimiglianza del dato completo, calcolato rispetto alla distribuzione a posteriori delle variabili latenti, data la stima corrente dei parametri $\boldsymbol{\theta}$.

13.4.3 Algoritmo EM generale

L'algoritmo EM procede iterativamente secondo i seguenti passi:

E-step (Expectation) Dato il valore corrente dei parametri $\boldsymbol{\theta}^{(t)}$, si calcola la distribuzione a posteriori delle variabili latenti e la funzione:

$$Q(\boldsymbol{\theta}' \mid \boldsymbol{\theta}^{(t)}) = \mathbb{E}_{Z|X, \boldsymbol{\theta}^{(t)}} [\ln p(X, Z \mid \boldsymbol{\theta}')].$$

M-step (Maximization) Si aggiornano i parametri massimizzando la funzione Q :

$$\boldsymbol{\theta}^{(t+1)} = \arg \max_{\boldsymbol{\theta}'} Q(\boldsymbol{\theta}' \mid \boldsymbol{\theta}^{(t)}).$$

Questi due passi vengono ripetuti fino a convergenza.

13.4.4 Proprietà dell'algoritmo EM

L'algoritmo EM presenta le seguenti proprietà generali:

- la log-verosimiglianza dei dati osservati non diminuisce a ogni iterazione;
- la convergenza è garantita verso un massimo locale;
- la qualità della soluzione dipende dall'inizializzazione dei parametri;
- il metodo è applicabile a un'ampia classe di modelli con variabili latenti.

13.4.5 Applicazioni

Il framework generale di EM trova applicazione in numerosi contesti, tra cui:

- clustering non supervisionato;
- Gaussian Mixture Models;
- Hidden Markov Models;
- Bayesian Networks;
- modelli con variabili latenti continue o discrete.

RIEPILOGO

Unsupervised learning

L'**apprendimento non supervisionato** si applica a contesti in cui sono disponibili solo i dati di input, senza etichette associate. L'obiettivo è individuare strutture latenti nei dati, come cluster o pattern statistici, direttamente a partire dalle osservazioni.

I **Gaussian Mixture Models (GMM)** sono modelli probabilistici generativi che descrivono i dati come generati da una combinazione di distribuzioni gaussiane. Ogni componente del modello rappresenta un cluster ed è caratterizzata da una media, una matrice di covarianza e una probabilità a priori.

Per modellare l'appartenenza dei dati ai cluster si introducono **variabili latenti**, che indicano quale componente gaussiana ha generato ciascun campione. Queste variabili non sono osservabili e devono essere inferite insieme ai parametri del modello.

L'algoritmo **K-means** fornisce una soluzione semplice al problema del clustering basata su assegnazioni hard e distanza euclidea. Sebbene efficiente, è limitato a cluster sferici, è sensibile all'inizializzazione e non fornisce una modellazione probabilistica.

L'**Expectation Maximization (EM)** generalizza K-means introducendo assegnazioni soft e una formulazione probabilistica. L'algoritmo alterna una fase di stima delle variabili latenti (E-step) e una fase di aggiornamento dei parametri del modello (M-step), garantendo la non diminuzione della log-verosimiglianza e la convergenza a un massimo locale.

Infine, EM rappresenta un **metodo generale di inferenza** applicabile a un'ampia classe di modelli con variabili latenti, come Gaussian Mixture Models, Hidden Markov Models e Bayesian Networks.

Chapter 14

Dimensionally reduction

14.1 Variabili latenti

Nei problemi di riduzione della dimensionalità, i dati osservati sono spesso rappresentati in spazi ad alta dimensionalità. Tuttavia, la dimensionalità dello spazio di rappresentazione non coincide necessariamente con il numero di gradi di libertà che descrivono le variazioni significative dei dati.

Le *variabili latenti* rappresentano tali gradi di libertà nascosti, non direttamente osservabili, che generano i dati osservati. Sebbene ogni campione sia descritto da un vettore ad alta dimensionalità

$$\mathbf{x}_n \in \mathbb{R}^d,$$

la variabilità effettiva dei dati può essere spiegata da un numero molto inferiore di parametri.

Un esempio semplice è costituito da un insieme di immagini ottenute ruotando un singolo prototipo. In questo caso, ogni immagine è rappresentata da un vettore ad alta dimensionalità, ma tutte le variazioni osservate dipendono da un unico parametro, l'angolo di rotazione. Il dataset presenta quindi un solo grado di libertà.

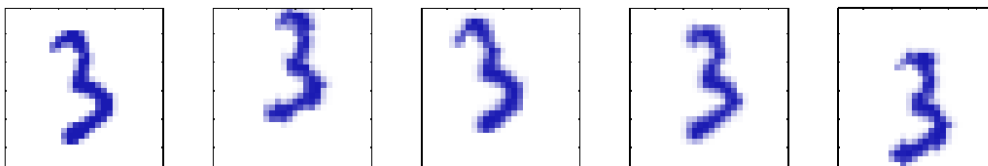


Figure 14.1: Esempio di immagini della stessa cifra scritta a mano. Sebbene ogni immagine sia rappresentata in uno spazio ad alta dimensionalità, le variazioni osservate sono riconducibili a un numero ridotto di gradi di libertà (variabili latenti), come rotazioni, traslazioni e differenze di stile.

Un ulteriore esempio è dato da un insieme di immagini RGB di dimensione 100×100 , ottenute mediante trasformazioni geometriche di una singola immagine. Sebbene ogni immagine appartenga a uno spazio di dimensione 30,000, le trasformazioni applicate (traslazioni bidimensionali e rotazione) introducono soltanto tre gradi di libertà. Questo evidenzia come lo spazio dei dati sia molto più grande dello spazio delle variazioni effettive.

14.2 Manifold dei dati

Quando i dati presentano una struttura interna, essi non occupano uniformemente l'intero spazio di rappresentazione, ma si distribuiscono su una regione a dimensionalità inferiore. Questa regione viene detta *manifold*.

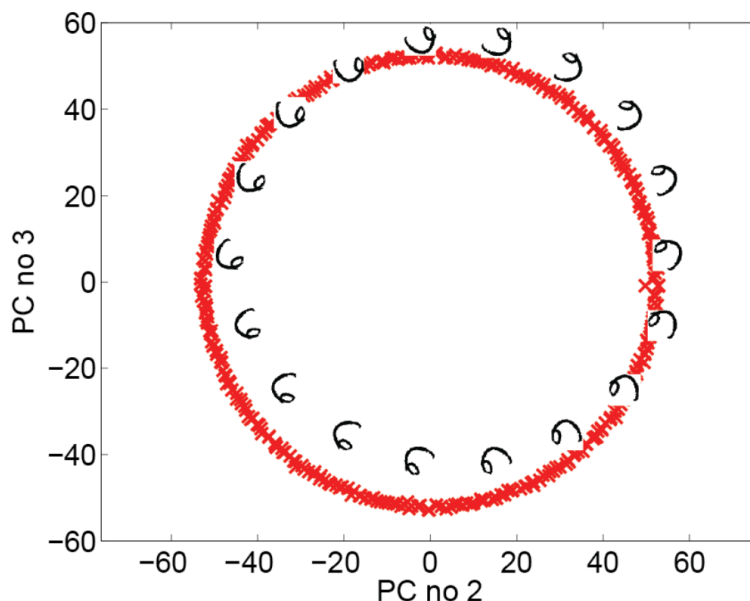


Figure 14.2: Rappresentazione del manifold dei dati ottenuta proiettando immagini della stessa cifra sulle componenti principali PC2 e PC3.

La nozione di manifold può essere visualizzata proiettando i dati in uno spazio a dimensionalità ridotta. In Figura 14.2 sono riportate le proiezioni di immagini della stessa cifra scritta a mano sulle componenti principali seconda e terza. Sebbene lo spazio di rappresentazione sia bidimensionale, i dati non occupano l'intero piano, ma si distribuiscono lungo una struttura continua e chiusa. Questa struttura rappresenta il manifold su cui giacciono i dati: una varietà a dimensionalità inferiore che riflette i gradi di libertà effettivi del dataset.

In questo caso, la disposizione circolare dei punti è coerente con una variazione continua di un singolo fattore latente, come ad esempio la rotazione del prototipo. La figura mostra quindi come dati ad alta dimensionalità possano essere descritti efficacemente come campioni di un manifold a bassa dimensionalità immerso in uno spazio di dimensione maggiore.

Per dati dotati di struttura, il numero di distorsioni o trasformazioni rilevanti è inferiore al numero di dimensioni dello spazio dei dati. Di conseguenza, i campioni appartengono a un manifold a dimensionalità ridotta.

14.3 Principal Component Analysis

La *Principal Component Analysis* (PCA) è una delle tecniche più utilizzate per la riduzione della dimensionalità. L'obiettivo della PCA è individuare una rappresentazione a dimensione ridotta dei dati che preservi il più possibile la loro struttura, mantenendo le variazioni più significative.

La PCA viene comunemente impiegata per:

- riduzione della dimensionalità;
- compressione dei dati (con perdita di informazione controllata);
- visualizzazione di dati ad alta dimensionalità;
- estrazione di caratteristiche rilevanti.

Dal punto di vista geometrico, la PCA ricerca nuove direzioni nello spazio dei dati lungo le quali la variabilità dei campioni è massima. Proiettando i dati su tali direzioni, è possibile ottenere una rappresentazione compatta che approssima il manifold su cui i dati giacciono.

14.3.1 Centratura dei dati e matrice di covarianza

Per poter misurare correttamente la varianza dei dati proiettati, è necessario considerare dati centrati rispetto alla loro media.

La media del dataset è definita come

$$\bar{\mathbf{x}} = \frac{1}{N} \sum_{n=1}^N \mathbf{x}_n.$$

A partire da essa si costruisce la matrice dei dati centrati

$$\mathbf{X} = \begin{bmatrix} (\mathbf{x}_1 - \bar{\mathbf{x}})^T \\ \vdots \\ (\mathbf{x}_n - \bar{\mathbf{x}})^T \\ \vdots \\ (\mathbf{x}_N - \bar{\mathbf{x}})^T \end{bmatrix} \in \mathbb{R}^{N \times d}.$$

La matrice di covarianza del dataset è quindi definita come

$$\mathbf{S} = \frac{1}{N} \sum_{n=1}^N (\mathbf{x}_n - \bar{\mathbf{x}})(\mathbf{x}_n - \bar{\mathbf{x}})^T = \frac{1}{N} \mathbf{X}^T \mathbf{X}.$$

La matrice di covarianza $\mathbf{S}$ descrive come le diverse componenti dei dati variano congiuntamente ed è l'oggetto centrale su cui si basa la PCA.

14.3.2 Massimizzazione della varianza

Si consideri la proiezione dei dati centrati lungo una direzione unitaria

$$\mathbf{u}_1 \in \mathbb{R}^d, \quad \mathbf{u}_1^T \mathbf{u}_1 = 1.$$

La proiezione del campione $\mathbf{x}_n$ è data da $\mathbf{u}_1^T \mathbf{x}_n$, mentre la proiezione della media è $\mathbf{u}_1^T \bar{\mathbf{x}}$.

La varianza dei dati proiettati può essere scritta come

$$\frac{1}{N} \sum_{n=1}^N (\mathbf{u}_1^T \mathbf{x}_n - \mathbf{u}_1^T \bar{\mathbf{x}})^2 = \mathbf{u}_1^T \mathbf{S} \mathbf{u}_1,$$

dove $\mathbf{S}$ è la matrice di covarianza del dataset.

Il problema della PCA consiste quindi nel massimizzare la varianza dei dati proiettati, imponendo che la direzione di proiezione abbia norma unitaria:

$$\max_{\mathbf{u}_1} \mathbf{u}_1^T \mathbf{S} \mathbf{u}_1 \quad \text{soggetto a} \quad \mathbf{u}_1^T \mathbf{u}_1 = 1.$$

Questo problema vincolato può essere risolto introducendo un moltiplicatore di Lagrange λ_1 e considerando la funzione

$$\mathcal{L}(\mathbf{u}_1, \lambda_1) = \mathbf{u}_1^T \mathbf{S} \mathbf{u}_1 + \lambda_1 (1 - \mathbf{u}_1^T \mathbf{u}_1).$$

Imponendo l'annullamento della derivata rispetto a $\mathbf{u}_1$ si ottiene

$$\mathbf{S} \mathbf{u}_1 = \lambda_1 \mathbf{u}_1.$$

Pertanto, la direzione che massimizza la varianza è un autovettore della matrice di covarianza.

Moltiplicando a sinistra per $\mathbf{u}_1^T$ e usando il vincolo di normalizzazione, si ha

$$\mathbf{u}_1^T \mathbf{S} \mathbf{u}_1 = \lambda_1,$$

che rappresenta la varianza dei dati proiettati lungo la direzione $\mathbf{u}_1$.

La varianza è massima quando $\mathbf{u}_1$ è l'autovettore associato al massimo autovalore λ_1 della matrice di covarianza. Questa direzione prende il nome di *prima componente principale*.

14.4 PCA come minimizzazione dell'errore di ricostruzione

Si consideri una base ortonormale completa dello spazio dei dati

$$\{\mathbf{u}_i\}_{i=1}^d, \quad \mathbf{u}_i^T \mathbf{u}_j = \delta_{ij},$$

dove δ_{ij} è il delta di Kronecker. Ogni campione $\mathbf{x}_n$ può essere espresso come combinazione lineare di tali vettori:

$$\mathbf{x}_n = \sum_{i=1}^d \alpha_{ni} \mathbf{u}_i.$$

Grazie all'ortonormalità della base, i coefficienti sono dati da

$$\alpha_{ni} = \mathbf{x}_n^T \mathbf{u}_i,$$

e quindi

$$\mathbf{x}_n = \sum_{i=1}^d (\mathbf{x}_n^T \mathbf{u}_i) \mathbf{u}_i.$$

L'obiettivo della riduzione della dimensionalità è approssimare ogni campione $\mathbf{x}_n$ utilizzando solo $m < d$ componenti. L'approssimazione può essere scritta come

$$\tilde{\mathbf{x}}_n = \sum_{i=1}^m z_{ni} \mathbf{u}_i + \sum_{i=m+1}^d b_i \mathbf{u}_i,$$

dove z_{ni} sono coefficienti dipendenti dal campione, mentre b_i sono termini costanti, indipendenti da n .

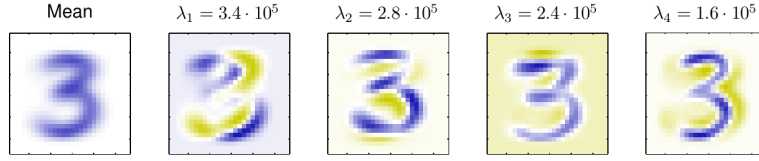


Figure 14.3: Media del dataset e prime componenti principali ottenute tramite PCA. Ogni componente principale corrisponde a un autovettore della matrice di covarianza ed evidenzia una direzione di variazione significativa nei dati. Le componenti sono ordinate in base al valore decrescente dell'autovalore associato, che misura la varianza spiegata lungo ciascuna direzione.

L'errore di approssimazione viene misurato tramite l'errore quadratico medio (MSE):

$$J = \frac{1}{N} \sum_{n=1}^N \|\mathbf{x}_n - \tilde{\mathbf{x}}_n\|^2.$$

Minimizzando J rispetto ai coefficienti z_{ni} e ai termini b_i , si ottiene

$$z_{ni} = \mathbf{x}_n^T \mathbf{u}_i, \quad i = 1, \dots, m,$$

$$b_i = \bar{\mathbf{x}}^T \mathbf{u}_i, \quad i = m + 1, \dots, d.$$

Sostituendo tali espressioni, l'errore di ricostruzione diventa

$$\mathbf{x}_n - \tilde{\mathbf{x}}_n = \sum_{i=m+1}^d [(\mathbf{x}_n - \bar{\mathbf{x}})^T \mathbf{u}_i] \mathbf{u}_i,$$

e l'errore quadratico medio complessivo risulta

$$J = \sum_{i=m+1}^d \mathbf{u}_i^T \mathbf{S} \mathbf{u}_i,$$

dove $\mathbf{S}$ è la matrice di covarianza del dataset.

La minimizzazione dell'errore J sotto il vincolo di normalizzazione $\mathbf{u}_i^T \mathbf{u}_i = 1$ porta all'equazione agli autovalori

$$\mathbf{S} \mathbf{u}_i = \lambda_i \mathbf{u}_i.$$

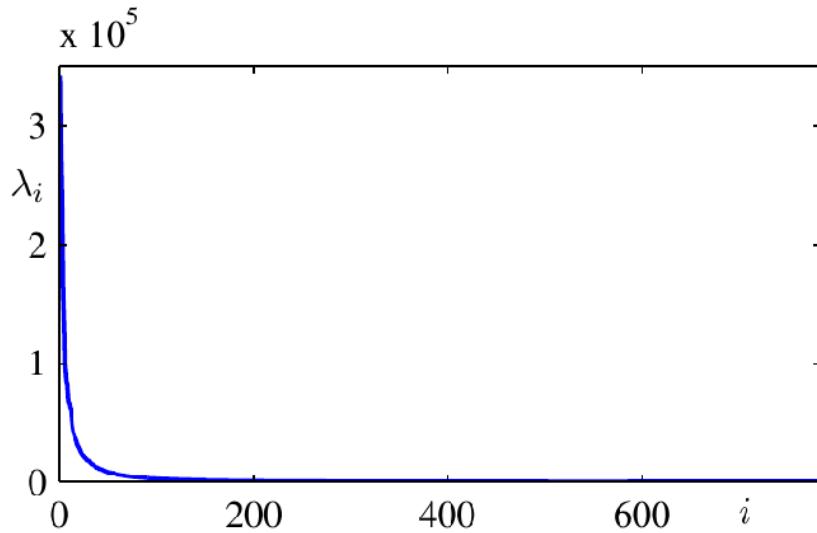


Figure 14.4: Spettro degli autovalori della matrice di covarianza. Ogni autovalore λ_i rappresenta la varianza dei dati lungo la corrispondente componente principale. La rapida decrescita degli autovalori indica che gran parte della variabilità del dataset è concentrata nelle prime componenti.

L'errore di ricostruzione minimo è dato dalla somma degli autovalori associati alle componenti scartate:

$$J = \sum_{i=m+1}^d \lambda_i.$$

Pertanto, l'errore è minimizzato scegliendo come componenti principali gli autovettori associati ai m autovalori maggiori della matrice di covarianza. Questo dimostra che la massimizzazione della varianza e la minimizzazione dell'errore di ricostruzione sono formulazioni equivalenti della PCA.

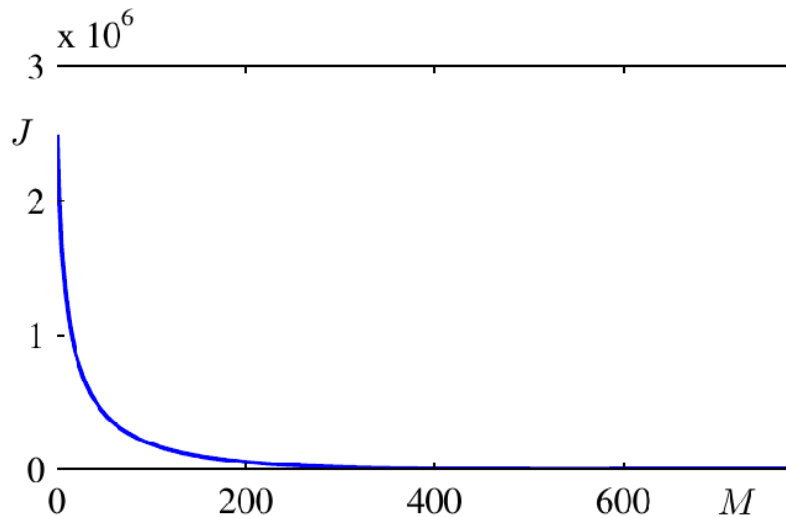


Figure 14.5: Andamento dell'errore di ricostruzione in funzione del numero di componenti principali mantenute. L'errore è pari alla somma degli autovalori scartati e diminuisce rapidamente all'aumentare delle componenti incluse, mostrando l'equivalenza tra massimizzazione della varianza e minimizzazione dell'errore di ricostruzione nella PCA.

14.5 Algoritmi per la PCA

Una volta formulato il problema della PCA, le componenti principali possono essere calcolate in diversi modi equivalenti, che differiscono principalmente per efficienza computazionale.

1. **Autodecomposizione completa della matrice di covarianza.** Si calcola la matrice di covarianza

$$\mathbf{S} = \frac{1}{N} \mathbf{X}^T \mathbf{X},$$

e se ne determinano tutti gli autovalori e gli autovettori. Questo approccio è concettualmente semplice, ma computazionalmente costoso quando la dimensionalità d è elevata.

2. **Autodecomposizione parziale.** In molti casi è sufficiente calcolare solo gli m autovettori associati ai maggiori autovalori, ottenendo così una riduzione significativa del costo computazionale.
3. **Decomposizione ai valori singolari (SVD).** In alternativa, la PCA può essere calcolata mediante la decomposizione ai valori singolari della matrice dei dati centrati $\mathbf{X}$. Questo approccio è numericamente più stabile ed è particolarmente efficace per dataset ad alta dimensionalità.

14.5.1 PCA per dati ad alta dimensionalità

Nei problemi reali è frequente che la dimensionalità dei dati sia molto elevata, mentre il numero di campioni disponibili sia relativamente ridotto. In particolare, può verificarsi il caso

$$N \ll d,$$

ad esempio quando si lavora con un piccolo insieme di immagini ad alta risoluzione.

In questa situazione, la matrice di covarianza

$$\mathbf{S} = \frac{1}{N} \mathbf{X}^T \mathbf{X} \in \mathbb{R}^{d \times d}$$

presenta almeno $d - N + 1$ autovalori nulli. Il calcolo diretto degli autovettori di $\mathbf{S}$ risulta quindi inefficiente dal punto di vista computazionale.

Per superare questo problema, si osserva che l'equazione agli autovalori

$$\frac{1}{N} \mathbf{X}^T \mathbf{X} \mathbf{u}_i = \lambda_i \mathbf{u}_i$$

può essere trasformata moltiplicando entrambi i membri a sinistra per $\mathbf{X}$:

$$\frac{1}{N} \mathbf{X} \mathbf{X}^T (\mathbf{X} \mathbf{u}_i) = \lambda_i (\mathbf{X} \mathbf{u}_i).$$

Definendo

$$\mathbf{v}_i = \mathbf{X} \mathbf{u}_i,$$

si ottiene il problema agli autovalori

$$\frac{1}{N} \mathbf{X} \mathbf{X}^T \mathbf{v}_i = \lambda_i \mathbf{v}_i,$$

dove la matrice $\mathbf{X} \mathbf{X}^T \in \mathbb{R}^{N \times N}$ ha la stessa parte non nulla dello spettro di $\mathbf{X}^T \mathbf{X}$, ma dimensione molto inferiore.

Una volta calcolati gli autovalori λ_i e gli autovettori $\mathbf{v}_i$ della matrice $\mathbf{X} \mathbf{X}^T$, gli autovettori della matrice di covarianza possono essere ricostruiti come

$$\mathbf{u}_i = \frac{1}{\sqrt{N \lambda_i}} \mathbf{X}^T \mathbf{v}_i,$$

dove il fattore di normalizzazione garantisce che $\mathbf{u}_i^T \mathbf{u}_i = 1$.

In conclusione, quando il numero di campioni è molto inferiore alla dimensionalità dei dati, la PCA può essere calcolata in modo efficiente lavorando nello spazio dei campioni anziché nello spazio delle caratteristiche.

14.6 Probabilistic Principal Component Analysis

La *Probabilistic Principal Component Analysis* (PPCA) fornisce un'interpretazione probabilistica della PCA classica, formulandola come un modello generativo con variabili latenti continue.

Si assume che ogni osservazione $\mathbf{x} \in \mathbb{R}^d$ sia generata a partire da un vettore latente

$$\mathbf{z} \in \mathbb{R}^k, \quad k < d,$$

tramite una trasformazione lineare affetta da rumore:

$$\mathbf{x} = \mathbf{W} \mathbf{z} + \boldsymbol{\mu} + \boldsymbol{\epsilon}.$$

Nel modello PPCA si assumono le seguenti ipotesi:

- le variabili latenti seguono una distribuzione gaussiana standard

$$p(\mathbf{z}) = \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I});$$

- il rumore è gaussiano isotropo

$$\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I});$$

- il modello condizionato dei dati, dato lo stato latente, è quindi

$$p(\mathbf{x} | \mathbf{z}) = \mathcal{N}(\mathbf{x}; \mathbf{W}\mathbf{z} + \boldsymbol{\mu}, \sigma^2 \mathbf{I}).$$

I parametri del modello sono la matrice di proiezione $\mathbf{W} \in \mathbb{R}^{d \times k}$, il vettore di media $\boldsymbol{\mu}$ e la varianza del rumore σ^2 .

14.6.1 Maximum Likelihood PCA

Nel contesto della Probabilistic PCA, i parametri del modello possono essere stimati tramite il principio di *massima verosimiglianza*. Dato un insieme di dati osservati

$$\{\mathbf{x}_n\}_{n=1}^N,$$

l'obiettivo è determinare i parametri $\mathbf{W}$, $\boldsymbol{\mu}$ e σ^2 che massimizzano la log-verosimiglianza dei dati:

$$\arg \max_{\mathbf{W}, \boldsymbol{\mu}, \sigma^2} \sum_{n=1}^N \log p(\mathbf{x}_n | \mathbf{W}, \boldsymbol{\mu}, \sigma^2).$$

Poiché la distribuzione marginale dei dati è gaussiana,

$$p(\mathbf{x}) = \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \mathbf{C}), \quad \mathbf{C} = \mathbf{W}\mathbf{W}^T + \sigma^2 \mathbf{I},$$

la funzione di verosimiglianza dipende unicamente dalla matrice di covarianza indotta dal modello.

Imponendo l'annullamento delle derivate della log-verosimiglianza rispetto ai parametri, si ottiene una soluzione in forma chiusa. In particolare, la stima di massima verosimiglianza della media coincide con la media campionaria:

$$\boldsymbol{\mu}_{\text{ML}} = \bar{\mathbf{x}} = \frac{1}{N} \sum_{n=1}^N \mathbf{x}_n.$$

La soluzione di massima verosimiglianza per la matrice $\mathbf{W}$ risulta essere strettamente legata agli autovalori e agli autovettori della matrice di covarianza dei dati. In particolare, le direzioni individuate dal modello coincidono con le componenti principali della PCA classica, mentre il parametro σ^2 rappresenta la varianza del rumore isotropo.

Questo risultato mostra che la PCA classica può essere interpretata come una soluzione di massima verosimiglianza di un modello generativo lineare con rumore gaussiano.

14.6.2 Stima dei parametri tramite algoritmo EM

La soluzione di massima verosimiglianza per il modello di Probabilistic PCA può essere ottenuta anche mediante l'algoritmo EM (*Expectation-Maximization*). Questo approccio risulta particolarmente utile quando si interpretano le variabili latenti come dati mancanti.

Nel contesto della PPCA, l'algoritmo EM alterna due fasi:

- **E-step (Expectation):** dato un insieme corrente di parametri del modello, si calcola la distribuzione a posteriori delle variabili latenti

$$p(\mathbf{z}_n \mid \mathbf{x}_n),$$

insieme ai momenti necessari per l'aggiornamento dei parametri.

- **M-step (Maximization):** si aggiornano i parametri $\mathbf{W}$, $\boldsymbol{\mu}$ e σ^2 massimizzando il valore atteso della log-verosimiglianza completa, utilizzando le stime ottenute nell'E-step.

Iterando le due fasi, l'algoritmo EM converge a una soluzione di massima verosimiglianza. Nel caso della PPCA, tale soluzione coincide con quella ottenuta tramite l'autodecomposizione della matrice di covarianza, mostrando ancora una volta l'equivalenza tra la formulazione probabilistica e la PCA classica.

L'uso dell'algoritmo EM evidenzia come la PCA possa essere interpretata come un modello generativo con variabili latenti, aprendo la strada a generalizzazioni non lineari e a modelli più complessi.

14.7 Autoencoder

Gli *autoencoder* sono reti neurali progettate per apprendere una rappresentazione compatta dei dati attraverso un processo di ricostruzione. L'idea di base consiste nell'addestrare una rete neurale a riprodurre in uscita lo stesso dato fornito in ingresso, imponendo però una restrizione sulla dimensionalità della rappresentazione interna.

Un autoencoder è composto da due parti principali:

- un **encoder**, che mappa il dato di ingresso $\mathbf{x} \in \mathbb{R}^d$ in una rappresentazione latente a dimensionalità ridotta $\mathbf{z} \in \mathbb{R}^k$, con $k < d$;
- un **decoder**, che ricostruisce il dato originale a partire dalla rappresentazione latente.

Indicando con $f(\cdot)$ la funzione di codifica e con $g(\cdot)$ la funzione di decodifica, il processo può essere scritto come

$$\mathbf{z} = f(\mathbf{x}), \quad \tilde{\mathbf{x}} = g(\mathbf{z}).$$

L'addestramento dell'autoencoder avviene minimizzando una funzione di perdita di ricostruzione che misura la differenza tra l'ingresso e l'uscita della rete:

$$\mathcal{L}(\mathbf{x}, \tilde{\mathbf{x}}) = \|\mathbf{x} - \tilde{\mathbf{x}}\|^2,$$

oppure, più in generale, una funzione di perdita adatta al tipo di dati considerati.

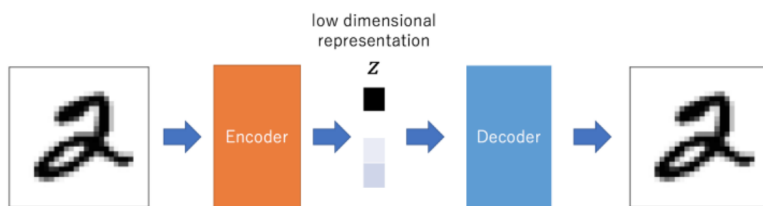


Figure 14.6: Schema generale di un autoencoder. L'encoder comprime il dato di ingresso in una rappresentazione latente a bassa dimensionalità $\mathbf{z}$, mentre il decoder utilizza tale rappresentazione per ricostruire l'input originale. L'addestramento avviene minimizzando l'errore di ricostruzione tra input e output, forzando la rete ad apprendere una rappresentazione compatta e informativa dei dati.

14.7.1 Bottleneck e rappresentazione latente

La caratteristica fondamentale di un autoencoder è la presenza di uno *strato centrale* a dimensionalità ridotta, detto *bottleneck*. Questo strato impone una compressione dell'informazione, costringendo la rete ad apprendere una rappresentazione latente compatta dei dati.

Dal punto di vista architetturale, l'encoder implementa una funzione

$$\mathbf{z} = f_1(\mathbf{x}),$$

che mappa l'input in uno spazio latente a bassa dimensionalità, mentre il decoder implementa una funzione

$$\tilde{\mathbf{x}} = f_2(\mathbf{z}),$$

che ricostruisce l'input a partire dalla rappresentazione latente.

La presenza di funzioni di attivazione non lineari negli strati nascosti consente all'autoencoder di apprendere trasformazioni non lineari, permettendo di approssimare manifolds complessi che non possono essere catturati da tecniche lineari come la PCA.

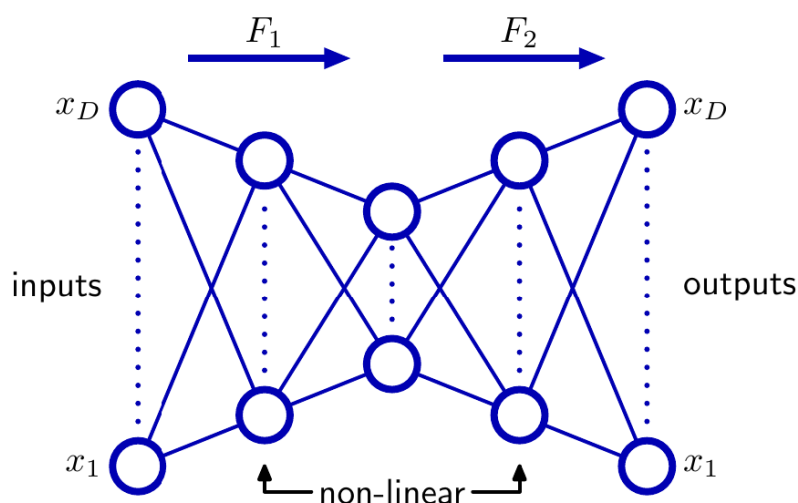


Figure 14.7: Schema di un autoencoder con bottleneck.

14.7.2 Convolutional Autoencoders

Quando i dati di ingresso sono immagini, l'utilizzo di autoencoder completamente connessi risulta inefficiente, poiché non sfrutta la struttura spaziale dei dati. I *Convolutional Autoencoders* (CAE) estendono il concetto di autoencoder introducendo strati convoluzionali sia nell'encoder sia nel decoder.

Nell'encoder, strati convoluzionali e operazioni di sottocampionamento permettono di estrarre caratteristiche locali e di ridurre progressivamente la dimensionalità spaziale dell'immagine, producendo una rappresentazione latente compatta. Il decoder opera in modo simmetrico, utilizzando strati di convoluzione trasposta (o upsampling) per ricostruire l'immagine originale a partire dallo spazio latente.

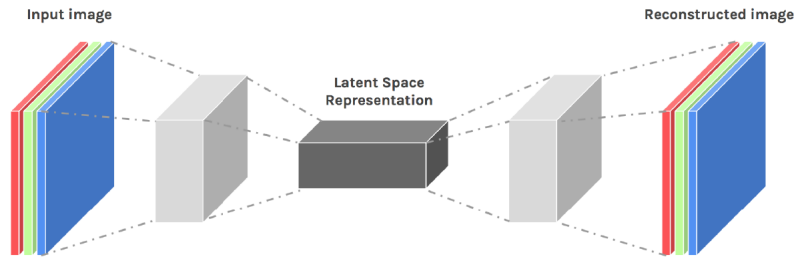


Figure 14.8: Schema di un Convolutional Autoencoder per immagini.

14.7.3 Autoencoder per Anomaly Detection

Gli autoencoder sono particolarmente adatti a problemi di *anomaly detection*, ovvero al riconoscimento di campioni anomali rispetto a un insieme di dati considerati normali. Questo tipo di problema può essere visto come una forma di classificazione a una sola classe (*one-class classification*).

Si consideri un dataset

$$\mathcal{D} = \{(\mathbf{x}_n, n)\},$$

composto esclusivamente da campioni appartenenti alla classe normale. L'obiettivo è apprendere una funzione

$$f : \mathcal{X} \rightarrow \{n, a\},$$

che, dato un nuovo campione $\mathbf{x}_0$, stabilisca se esso sia normale (n) o anomalo (a).

L'approccio basato su autoencoder prevede i seguenti passi:

1. addestrare un autoencoder utilizzando esclusivamente campioni normali, in modo che la rete apprenda una rappresentazione latente e una ricostruzione accurata dei dati nominali;
2. calcolare l'errore di ricostruzione sui dati di training e determinare una soglia δ , ad esempio

$$\delta = \text{mean}(\mathcal{L}) + \text{std}(\mathcal{L}),$$

dove $\mathcal{L}$ indica la loss di ricostruzione;

3. dato un nuovo campione $\mathbf{x}_0$, ricostruirlo tramite l'autoencoder e calcolare l'errore di ricostruzione $\mathcal{L}_0$;

4. se $\mathcal{L}_0 < \delta$, il campione viene classificato come normale, altrimenti come anomalo.

L'idea alla base di questo metodo è che l'autoencoder, essendo stato addestrato solo su dati normali, sia in grado di ricostruire accuratamente tali campioni, mentre produca un errore elevato su dati che si discostano significativamente dalla distribuzione appresa.

14.8 Variational Autoencoders

I *Variational Autoencoders* (VAE) sono una classe di autoencoder progettati per apprendere non solo una rappresentazione latente compatta dei dati, ma anche una struttura probabilistica dello spazio latente. A differenza degli autoencoder standard, i VAE sono modelli *generativi*, in grado di produrre nuovi campioni realistici a partire dallo spazio latente.

Negli autoencoder classici, l'encoder mappa un dato di ingresso in un singolo punto dello spazio latente. Nei VAE, invece, l'encoder produce una *distribuzione* nello spazio latente, tipicamente una distribuzione gaussiana, da cui vengono estratti i campioni utilizzati dal decoder.

In particolare, dato un input $\mathbf{x}$, l'encoder stima i parametri di una distribuzione

$$q(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}(\mathbf{x}), \boldsymbol{\Sigma}(\mathbf{x})),$$

mentre il decoder definisce una distribuzione

$$p(\mathbf{x} \mid \mathbf{z}),$$

che modella il processo di generazione dei dati a partire dalle variabili latenti.

Una delle proprietà fondamentali dei Variational Autoencoders è la possibilità di interpretare e manipolare lo spazio latente in modo continuo e semanticamente significativo. Questo consente di modificare specifiche caratteristiche dei dati generati agendo lungo particolari direzioni dello spazio latente.

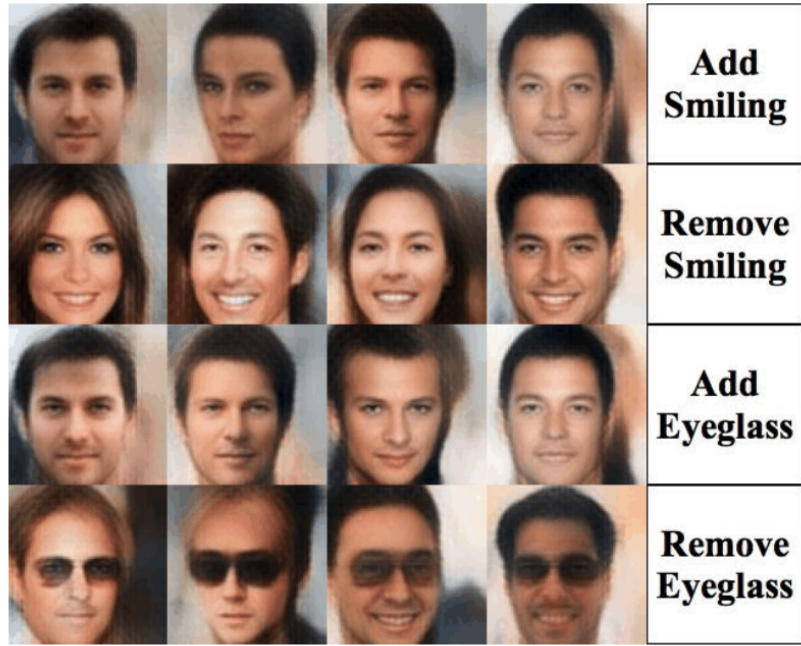


Figure 14.9: Esempio di manipolazione dello spazio latente in un Variational Autoencoder. Variazioni controllate delle variabili latenti permettono di modificare attributi semantici delle immagini generate, come l'aggiunta o la rimozione del sorriso o degli occhiali.

14.8.1 Problemi dei VAE e funzione obiettivo

La formulazione dei Variational Autoencoders introduce alcune problematiche fondamentali che richiedono soluzioni specifiche.

Produzione di una distribuzione latente. A differenza degli autoencoder standard, l'encoder di un VAE non produce un singolo punto nello spazio latente, ma i parametri di una distribuzione, tipicamente una gaussiana:

$$q(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}(\mathbf{x}), \boldsymbol{\Sigma}(\mathbf{x})).$$

Questo consente di modellare l'incertezza sulla rappresentazione latente associata a ciascun dato.

Regolarizzazione dello spazio latente. Senza ulteriori vincoli, le distribuzioni latenti apprese potrebbero essere disorganizzate o degeneri, rendendo impossibile una generazione coerente di nuovi campioni. Per evitare questo problema, nei VAE si impone che la distribuzione latente appresa sia vicina a una distribuzione prior fissata, generalmente

$$p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

Questa regolarizzazione è ottenuta introducendo un termine di divergenza di Kullback-Leibler tra la distribuzione latente appresa e il prior:

$$\text{KL}(q(\mathbf{z} \mid \mathbf{x}) \parallel p(\mathbf{z})).$$

Evidence Lower Bound (ELBO). La funzione obiettivo dei VAE è data dalla *Evidence Lower Bound* (ELBO), che bilancia accuratezza di ricostruzione e regolarizzazione dello spazio latente:

$$\mathcal{L}_{\text{ELBO}} = \mathbb{E}_{q(\mathbf{z}|\mathbf{x})} [\log p(\mathbf{x} | \mathbf{z})] - \text{KL}(q(\mathbf{z} | \mathbf{x}) \parallel p(\mathbf{z})).$$

Il primo termine incoraggia una buona ricostruzione dei dati, mentre il secondo impone una struttura continua e ben organizzata nello spazio latente.

14.8.2 Problemi dei VAE e funzione obiettivo

La formulazione dei Variational Autoencoders introduce alcune problematiche fondamentali che richiedono soluzioni specifiche.

Produzione di una distribuzione latente. A differenza degli autoencoder standard, l'encoder di un VAE non produce un singolo punto nello spazio latente, ma i parametri di una distribuzione, tipicamente una gaussiana:

$$q(\mathbf{z} | \mathbf{x}) = \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}(\mathbf{x}), \boldsymbol{\Sigma}(\mathbf{x})).$$

Questo consente di modellare l'incertezza sulla rappresentazione latente associata a ciascun dato.

Regolarizzazione dello spazio latente. Senza ulteriori vincoli, le distribuzioni latenti apprese potrebbero essere disorganizzate o degeneri, rendendo impossibile una generazione coerente di nuovi campioni. Per evitare questo problema, nei VAE si impone che la distribuzione latente appresa sia vicina a una distribuzione prior fissata, generalmente

$$p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

Questa regolarizzazione è ottenuta introducendo un termine di divergenza di Kullback–Leibler tra la distribuzione latente appresa e il prior:

$$\text{KL}(q(\mathbf{z} | \mathbf{x}) \parallel p(\mathbf{z})).$$

Evidence Lower Bound (ELBO). La funzione obiettivo dei VAE è data dalla *Evidence Lower Bound* (ELBO), che bilancia accuratezza di ricostruzione e regolarizzazione dello spazio latente:

$$\mathcal{L}_{\text{ELBO}} = \mathbb{E}_{q(\mathbf{z}|\mathbf{x})} [\log p(\mathbf{x} | \mathbf{z})] - \text{KL}(q(\mathbf{z} | \mathbf{x}) \parallel p(\mathbf{z})).$$

Il primo termine incoraggia una buona ricostruzione dei dati, mentre il secondo impone una struttura continua e ben organizzata nello spazio latente.

14.8.3 Reparameterization Trick

Nei Variational Autoencoders, il decoder opera su campioni estratti dalla distribuzione latente

$$q(\mathbf{z} | \mathbf{x}) = \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}(\mathbf{x}), \boldsymbol{\Sigma}(\mathbf{x})).$$

Tuttavia, l'operazione di campionamento non è differenziabile e impedirebbe l'uso diretto dell'algoritmo di backpropagation per l'addestramento della rete.

Per risolvere questo problema, nei VAE si introduce il *reparameterization trick*, che consente di separare la componente stocastica dai parametri della distribuzione. In particolare, un campione $\mathbf{z}$ può essere riscritto come

$$\mathbf{z} = \boldsymbol{\mu}(\mathbf{x}) + \boldsymbol{\sigma}(\mathbf{x}) \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}),$$

dove $\boldsymbol{\sigma}$ è la deviazione standard e $\odot$ denota il prodotto elemento per elemento.

In questo modo, la variabilità stocastica è interamente contenuta nella variabile casuale $\boldsymbol{\epsilon}$, mentre $\boldsymbol{\mu}$ e $\boldsymbol{\sigma}$ rimangono funzioni deterministiche e differenziabili dell'input. Ciò permette di propagare il gradiente attraverso il campionamento e di addestrare il modello tramite discesa del gradiente.

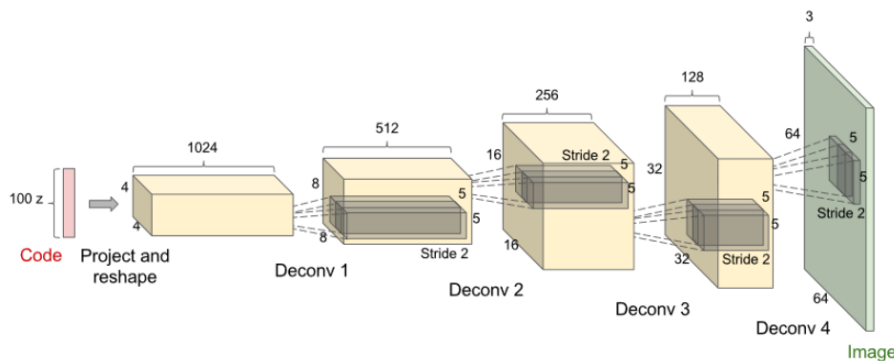
Il reparameterization trick rappresenta un elemento chiave dei VAE e rende possibile l'ottimizzazione della funzione obiettivo ELBO mediante metodi di apprendimento standard.

14.9 Generative Adversarial Networks

I *Generative Adversarial Networks* (GAN) sono modelli generativi che apprendono la distribuzione dei dati attraverso un processo di addestramento avversario. A differenza dei Variational Autoencoders, i GAN non modellano esplicitamente una distribuzione di probabilità dei dati, ma imparano a generare campioni realistici tramite la competizione tra due reti neurali.

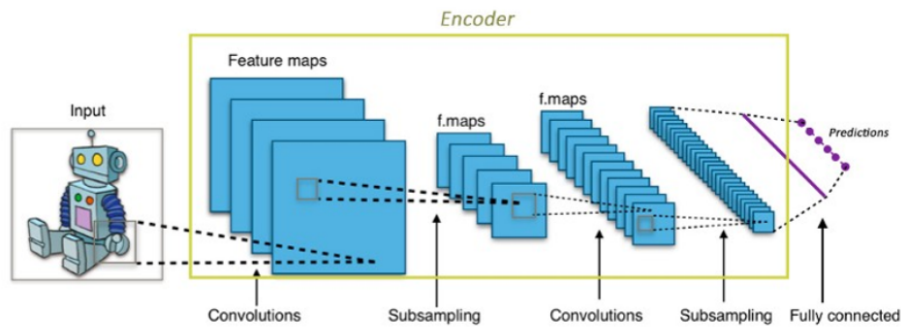
Un GAN è composto da due modelli:

- un **generatore** (G), che riceve in input un vettore latente casuale $\mathbf{z}$ e produce un campione sintetico $\tilde{\mathbf{x}} = G(\mathbf{z})$;



Il generatore riceve in input un vettore latente casuale, campionato da una distribuzione semplice, e lo trasforma progressivamente in un'immagine attraverso una serie di convoluzioni trasposte e operazioni di upsampling. A partire da una rappresentazione compatta e priva di struttura spaziale, la rete costruisce l'immagine aumentando gradualmente la risoluzione e introducendo dettagli sempre più complessi. Il ruolo del generatore è quello di trasformare il rumore in campioni che approssimino la distribuzione dei dati reali.

- un **discriminatore** (D), che riceve in input un campione e restituisce la probabilità che esso provenga dal dataset reale piuttosto che dal generatore.



Nei GAN, il discriminatore è tipicamente implementato come una rete convoluzionale profonda che estrae feature gerarchiche dalle immagini per distinguere campioni reali da campioni generati. Il generatore, invece, utilizza una struttura convoluzionale inversa per produrre immagini a partire da uno spazio latente.

14.9.1 Addestramento avversario

L'addestramento dei GAN si basa su un gioco competitivo tra generatore e discriminatore. Il discriminatore viene addestrato per distinguere correttamente tra campioni reali e campioni generati, mentre il generatore viene addestrato per produrre campioni in grado di ingannare il discriminatore.

Il processo di addestramento procede iterativamente secondo i seguenti passi:

1. il discriminatore viene addestrato utilizzando un insieme di campioni reali e un insieme di campioni generati;
2. il generatore viene aggiornato mantenendo fisso il discriminatore, con l'obiettivo di massimizzare la probabilità che i campioni generati vengano classificati come reali.

Questo meccanismo competitivo porta idealmente a una situazione di equilibrio in cui il discriminatore non è più in grado di distinguere tra dati reali e dati generati, e i campioni prodotti dal generatore seguono la stessa distribuzione dei dati osservati.

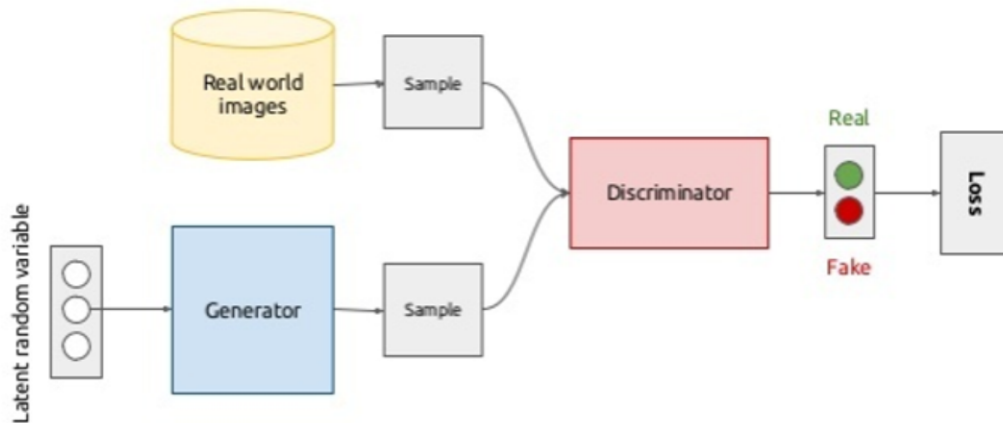


Figure 14.10: Schema di addestramento di un GAN. Il generatore trasforma rumore latente in immagini, mentre il discriminatore distingue tra dati reali e generati.

14.9.2 Spazio latente nei GAN

Anche nei GAN è presente uno spazio latente, rappresentato dal vettore $\mathbf{z}$. Tuttavia, a differenza dei Variational Autoencoders, lo spazio latente dei GAN non è regolarizzato esplicitamente e non è garantito che possieda una struttura continua o semanticamente interpretabile.

Lo spazio latente nei GAN funge principalmente da sorgente di variabilità per la generazione dei dati, consentendo di produrre campioni diversi a partire da input casuali differenti.

14.9.3 Vantaggi e limiti

I GAN sono noti per la loro capacità di generare campioni di elevata qualità percettiva, in particolare nel dominio delle immagini. Tuttavia, l'addestramento avversario può risultare instabile e non fornisce una misura esplicita della verosimiglianza dei dati.

Queste caratteristiche rendono i GAN complementari ai Variational Autoencoders: mentre i VAE privilegiano la stabilità e l'interpretabilità dello spazio latente, i GAN puntano alla qualità visiva dei campioni generati.

RIEPILOGO

Convolutional Neural Networks

La riduzione della dimensionalità si basa sull'osservazione che dati ad alta dimensionalità presentano spesso una struttura intrinseca a bassa dimensionalità, descrivibile tramite variabili latenti e manifolds. L'obiettivo è individuare rappresentazioni compatte che preservino l'informazione rilevante dei dati.

La *Principal Component Analysis* (PCA) realizza una riduzione dimensionale lineare proiettando i dati lungo le direzioni di massima varianza. Essa è equivalente alla minimizzazione dell'errore di ricostruzione, che risulta pari alla somma degli autovalori scartati della matrice di covarianza.

La *Probabilistic PCA* fornisce un'interpretazione statistica della PCA introducendo un modello generativo lineare con variabili latenti gaussiane. In questo contesto, la PCA emerge come soluzione di massima verosimiglianza e può essere stimata tramite l'algoritmo EM.

Gli *autoencoder* generalizzano la PCA introducendo modelli non lineari basati su reti neurali, composti da un encoder e un decoder separati da un bottleneck. Essi apprendono rappresentazioni latenti compatte ottimizzando una loss di ricostruzione e trovano applicazione, tra l'altro, nell'anomaly detection.

I *Variational Autoencoders* (VAE) estendono gli autoencoder introducendo una formulazione probabilistica dello spazio latente. L'addestramento avviene massimizzando l'ELBO, che bilancia accuratezza di ricostruzione e regolarizzazione tramite la KL divergence. Il reparameterization trick consente l'ottimizzazione tramite back-propagation.

I *Generative Adversarial Networks* (GAN) sono modelli generativi basati su un addestramento avversario tra un generatore e un discriminatore. Essi non modellano esplicitamente una likelihood, ma apprendono a generare campioni realistici tramite un gioco competitivo, privilegiando la qualità percettiva dei dati generati.

Nel complesso, queste tecniche mostrano come la modellazione dello spazio latente costituisca un elemento centrale per l'analisi, la compressione e la generazione dei dati complessi.

Chapter 15

Reinforcement learning

15.1 Dynamic Systems e Proprietà di Markov

Un sistema dinamico è un sistema che evolve nel tempo secondo una legge di transizione. Indichiamo con x_k lo stato del sistema al tempo k e con z_k l'osservazione disponibile all'agente. L'evoluzione del sistema è descritta da:

- una funzione di transizione dello stato f ;
- una funzione di osservazione h ;
- termini di rumore w_k e v_k .

Si distinguono due problemi fondamentali:

- **Reasoning**: dato il modello (f, h) e lo stato corrente, prevedere gli stati futuri;
- **Learning**: determinare il modello (f, h) a partire dall'esperienza passata.

Lo **stato** di un sistema dinamico deve codificare tutta l'informazione necessaria per prevedere l'evoluzione futura del sistema, rappresentare la conoscenza acquisita e perseguire l'obiettivo dell'agente.

15.1.1 Proprietà di Markov

Un processo soddisfa la proprietà di Markov se, una volta noto lo stato corrente, l'evoluzione futura del sistema è indipendente dalla storia passata. In altre parole, lo stato corrente contiene tutta l'informazione necessaria per predire il futuro.

15.2 Markov Decision Processes

Un Markov Decision Process (MDP) è un modello matematico per il decision making sequenziale in sistemi dinamici completamente osservabili. Negli MDP non è necessario introdurre osservazioni esplicite, poiché lo stato del sistema è interamente accessibile all'agente.

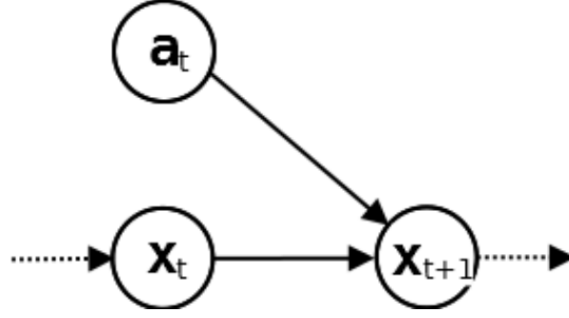


Figure 15.1: Modello grafico di un Markov Decision Process: lo stato successivo dipende dallo stato corrente e dall'azione scelta.

15.2.1 Modello grafico

La Figura 15.1 mostra il modello grafico di un MDP. Lo stato successivo x_{t+1} dipende esclusivamente dallo stato corrente x_t e dall'azione a_t eseguita dall'agente.

Questo modello soddisfa la proprietà di Markov:

$$P(x_{t+1} \mid x_t, a_t, x_{t-1}, a_{t-1}, \dots) = P(x_{t+1} \mid x_t, a_t)$$

15.2.2 Definizione formale

Un MDP deterministico è definito come una tupla:

$$\mathcal{M} = \langle X, A, \delta, r \rangle$$

dove:

- X è l'insieme finito degli stati;
- A è l'insieme finito delle azioni;
- $\delta : X \times A \rightarrow X$ è la funzione di transizione;
- $r : X \times A \rightarrow \mathbb{R}$ è la funzione di ricompensa.

L'evoluzione del sistema è descritta dalle equazioni:

$$x_{t+1} = \delta(x_t, a_t), \quad r_t = r(x_t, a_t)$$

Learning rate Il learning rate $\alpha \in (0, 1]$ controlla quanto le nuove osservazioni influenzano l'aggiornamento dei valori Q . Un valore elevato di α permette un apprendimento più rapido ma meno stabile, mentre un valore più piccolo rende l'apprendimento più lento ma più stabile. Se non specificato, il learning rate si assume costante.

15.3 MDP

Nei Markov Decision Processes non deterministici, l'esecuzione di una stessa azione in uno stesso stato può portare a esiti diversi. Questo modello è più realistico rispetto al caso deterministico e consente di rappresentare ambienti incerti.

15.3.1 MDP non deterministico

Un MDP non deterministico è definito come una tupla:

$$\mathcal{M} = \langle X, A, \delta, r \rangle$$

dove:

- X è l'insieme finito degli stati;
- A è l'insieme finito delle azioni;
- $\delta : X \times A \rightarrow 2^X$ è la funzione di transizione;
- $r : X \times A \times X \rightarrow \mathbb{R}$ è la funzione di ricompensa.

L'evoluzione del sistema è descritta da:

$$x_{t+1} \in \delta(x_t, a_t)$$

15.3.2 MDP stocastico

Nel caso stocastico, le transizioni sono modellate tramite una distribuzione di probabilità. Un MDP stocastico è definito come:

$$\mathcal{M} = \langle X, A, P, r \rangle$$

dove $P(x' \mid x, a)$ rappresenta la probabilità di transizione dallo stato x allo stato x' eseguendo l'azione a .

La dinamica del sistema è:

$$x_{t+1} \sim P(\cdot \mid x_t, a_t)$$

In entrambi i casi, si assume che lo stato risultante dall'esecuzione di un'azione sia completamente osservabile.

NOTA BENE: Un problema si dice stocastico quando l'esito di un'azione non è deterministico. Questo significa che, a parità di stato e di azione, il sistema può evolvere in modi diversi a causa della presenza di fenomeni casuali. In questi casi, il comportamento dell'ambiente non è completamente prevedibile e deve essere descritto in termini probabilistici. Formalmente: l'evoluzione del sistema o la ricompensa sono governate da variabili aleatorie.

15.3.3 Esempio di MDP non deterministico: Grid World

Consideriamo un ambiente a griglia in cui l'obiettivo dell'agente è raggiungere la colonna più a destra partendo da qualunque stato iniziale. Il problema è modellato come un Markov Decision Process non deterministico

$$\mathcal{M} = \langle X, A, \delta, r \rangle.$$

L'insieme degli stati è definito come:

$$X = \{(r, c) \mid (r, c) \text{ coordinate nella griglia}\}.$$

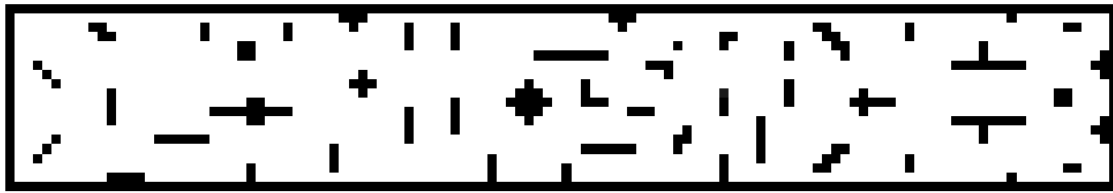


Figure 15.2: Esempio di ambiente non deterministico: l'agente deve raggiungere il lato destro della griglia evitando gli ostacoli.

L'insieme delle azioni disponibili è:

$$A = \{\text{Left}, \text{Right}\}.$$

La funzione di transizione δ è non deterministica: l'esecuzione di un'azione produce un movimento cardinale, ma con una probabilità pari a 0.1 l'agente si muove diagonalmente. Inoltre, se lo stato di destinazione corrisponde a una cella occupata da un ostacolo, l'agente rimane nello stato corrente.

La funzione di ricompensa è definita come segue:

- +1000 per il raggiungimento della colonna più a destra;
- -10 per l'urto contro un ostacolo;
- +1 per ogni azione *Right*;
- -1 per ogni azione *Left*.

Questo esempio evidenzia come, in presenza di transizioni non deterministiche, l'ottimalità di una policy sia definita in termini di valore atteso della ricompensa cumulativa e non in base all'esito deterministico delle singole azioni.

15.4 Policy

In un Markov Decision Process, una policy descrive il comportamento dell'agente specificando quale azione eseguire in ogni stato. Formalmente, una policy è una funzione:

$$\pi : X \rightarrow A$$

che associa a ogni stato $x \in X$ un'azione $\pi(x) \in A$.

Nel seguito si considerano policy stazionarie e deterministiche. Per MDP a orizzonte infinito, esiste sempre almeno una policy stazionaria ottima.

15.4.1 Funzione valore

Data una policy π , la funzione valore $V^\pi(x)$ è definita come il valore atteso della ricompensa cumulativa scontata:

$$V^\pi(x) \triangleq \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid x_0 = x, \pi \right],$$

dove $r_t = r(x_t, a_t, x_{t+1})$, $a_t = \pi(x_t)$ e $\gamma \in [0, 1]$ è il fattore di sconto.

15.4.2 Policy ottima

Una policy π^* è detta ottima se massimizza la funzione valore per ogni stato:

$$V^{\pi^*}(x) \geq V^\pi(x), \quad \forall x \in X, \forall \pi.$$

Equivalentemente, la policy ottima può essere definita come:

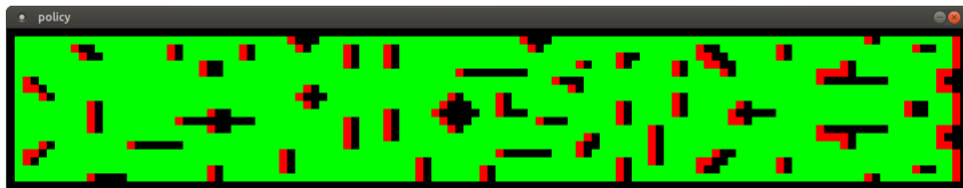
$$\pi^* = \arg \max_{\pi} V^\pi(x), \quad \forall x \in X.$$

Nel caso non deterministico o stocastico, l'ottimalità è definita in termini di valore atteso della ricompensa cumulativa e non in base all'esito deterministico delle singole azioni.

Esempio-Optimal Policy

La Figura 15.3 mostra una politica ottima per un MDP non deterministico. Ogni cella rappresenta uno stato del sistema, mentre il colore indica l'azione scelta dalla policy ottima: verde per l'azione *Right*, rosso per l'azione *Left*. La presenza di azioni *Left* in alcune regioni evidenzia come l'ottimalità sia definita in termini di valore atteso e non di effetto deterministico dell'azione

Optimal policy



green: action *Right*

red: action *Left*

Figure 15.3: Esempio di politica ottima in un controller non deterministico.

Anche se la policy è deterministica, il sistema è non deterministico: l'esecuzione di una stessa azione nello stesso stato può portare a stati successivi diversi. La politica ottima massimizza quindi il valore atteso della ricompensa cumulativa e non il risultato immediato.

15.5 Equazione di Bellman

L'idea alla base dell'equazione di Bellman è che il valore di uno stato dipende dalla ricompensa immediata ottenuta e dal valore atteso degli stati futuri. Questa osservazione introduce una relazione ricorsiva tra i valori degli stati.

15.5.1 Equazione di Bellman per una policy

Data una policy π , la funzione valore $V^\pi(x)$ soddisfa la seguente equazione di Bellman:

$$V^\pi(x) = \mathbb{E}[r(x, \pi(x), x') + \gamma V^\pi(x')].$$

Nel caso stocastico, l'equazione può essere esplicitata come:

$$V^\pi(x) = \sum_{x' \in X} P(x' | x, \pi(x)) [r(x, \pi(x), x') + \gamma V^\pi(x')].$$

15.5.2 Equazione di Bellman per il valore ottimo

La funzione valore ottima è definita come:

$$V^*(x) \triangleq \max_{\pi} V^\pi(x).$$

Essa soddisfa la seguente equazione di Bellman di ottimalità:

$$V^*(x) = \max_{a \in A} \mathbb{E}[r(x, a, x') + \gamma V^*(x')].$$

Nel caso stocastico, l'equazione assume la forma:

$$V^*(x) = \max_{a \in A} \sum_{x' \in X} P(x' | x, a) [r(x, a, x') + \gamma V^*(x')].$$

Questa equazione non definisce un algoritmo, ma una condizione di consistenza che il valore ottimo deve soddisfare. Gli algoritmi di reinforcement learning mirano ad approssimare una soluzione di tale equazione anche in assenza della conoscenza esplicita delle dinamiche dell'ambiente.

15.6 One-State Markov Decision Process

Il One-State Markov Decision Process rappresenta il caso più semplice di MDP e costituisce una base concettuale fondamentale per il reinforcement learning.

Un One-State MDP è definito come:

$$\mathcal{M} = \langle \{x_0\}, A, \delta, r \rangle$$

dove $\{x_0\}$ è l'unico stato del sistema, $A = \{a_1, \dots, a_n\}$ è l'insieme delle azioni disponibili, $\delta(x_0, a_i) = x_0$ per ogni $a_i \in A$ e $r(x_0, a_i, x_0) = r(a_i)$ è la funzione di ricompensa.

Poiché lo stato non cambia mai, il problema decisionale consiste unicamente nella selezione dell'azione che massimizza la ricompensa.

15.6.1 Policy ottima

Una policy $\pi : \{x_0\} \rightarrow A$ seleziona un'unica azione. Nel caso deterministico e noto, la policy ottima è:

$$\pi^*(x_0) = \arg \max_{a_i \in A} r(a_i).$$

Nel caso in cui il reward sia non deterministico, la policy ottima massimizza il valore atteso:

$$\pi^*(x_0) = \arg \max_{a_i \in A} \mathbb{E}[r(a_i)].$$

Questo scenario introduce il problema del compromesso tra esplorazione e sfruttamento, che costituisce il fondamento dei metodi di reinforcement learning.

15.7 Q-function

La funzione valore $V^\pi(x)$ misura quanto sia conveniente trovarsi in uno stato x seguendo una policy π . Tuttavia, nel reinforcement learning è spesso necessario valutare direttamente le azioni.

La Q-function associata a una policy π è definita come:

$$Q^\pi(x, a) \triangleq \mathbb{E}[r(x, a, x') + \gamma V^\pi(x')].$$

Essa rappresenta il valore atteso dell'esecuzione dell'azione a nello stato x , seguita dall'applicazione della policy π .

Nel caso ottimo, la funzione valore e la Q-function sono legate da:

$$V^*(x) = \max_{a \in A} Q^*(x, a).$$

La Q-function ottima è definita come:

$$Q^*(x, a) = \max_{\pi} Q^\pi(x, a),$$

e consente di determinare direttamente la policy ottima:

$$\pi^*(x) = \arg \max_{a \in A} Q^*(x, a).$$

La Q-function ottima soddisfa la seguente equazione di Bellman:

$$Q^*(x, a) = \mathbb{E} \left[r(x, a, x') + \gamma \max_{a' \in A} Q^*(x', a') \right].$$

15.8 Q-learning e apprendimento senza modello

Il Q-learning è un algoritmo di reinforcement learning che consente di apprendere la Q-function ottima $Q^*(x, a)$ senza conoscere esplicitamente il modello dell'ambiente, ovvero senza conoscere né la funzione di transizione δ né la funzione di ricompensa r .

L'apprendimento avviene attraverso l'interazione diretta con l'ambiente: l'agente esegue un'azione, osserva lo stato successivo e la ricompensa, e aggiorna iterativamente la stima della Q-function.

15.8.1 Regola di aggiornamento

Nel caso generale non deterministico, la regola di aggiornamento del Q-learning è:

$$Q_n(x, a) \leftarrow Q_{n-1}(x, a) + \alpha \left[r + \gamma \max_{a' \in A} Q_{n-1}(x', a') - Q_{n-1}(x, a) \right],$$

dove x è lo stato corrente, a l'azione eseguita, x' lo stato osservato dopo l'azione, r la ricompensa immediata, $\alpha \in (0, 1]$ è il learning rate e $\gamma \in [0, 1]$ è il fattore di sconto.

Questa regola deriva direttamente dall'equazione di Bellman per la Q-function ottima.

La regola di aggiornamento del Q-learning si applica quando il problema è modellato come un MDP con dinamica di transizione ignota e stocastica e l'agente osserva direttamente sequenze di interazione del tipo (x, a, r, x') . In questi casi non è possibile calcolare la funzione valore tramite le equazioni di Bellman e i valori $Q(x, a)$ devono essere appresi dall'esperienza.

In altre parole, la regola di aggiornamento del Q-learning si usa quando l'agente non conosce il modello dell'ambiente ma può interagire con esso osservando cosa succede dopo ogni azione. In pratica, ogni volta che l'agente si trova in uno stato x , sceglie un'azione a , riceve una ricompensa r e osserva il nuovo stato x' , si applica la formula di aggiornamento per correggere il valore $Q(x, a)$. Questa operazione viene ripetuta dopo ogni passo e permette di migliorare progressivamente la stima della funzione Q a partire dall'esperienza.

15.8.2 Action selection: exploration vs exploitation

Negli algoritmi di Reinforcement Learning, oltre alla regola di aggiornamento dei valori Q , è necessario stabilire come l'agente seleziona le azioni durante l'apprendimento. Questo porta al problema del bilanciamento tra *exploration* ed *exploitation*.

Per *exploitation* si intende la scelta dell'azione che, sulla base delle stime correnti di $Q(x, a)$, appare come la migliore. L'*exploration*, invece, consiste nel provare azioni diverse per raccogliere nuove informazioni sull'ambiente ed evitare di convergere prematuramente a soluzioni subottimali. In ambienti stocastici, una strategia puramente greedy non è sufficiente.

Una soluzione semplice e ampiamente utilizzata è la ε -greedy policy. Secondo questa strategia, in ogni stato x :

- con probabilità $1 - \varepsilon$ viene selezionata l'azione che massimizza $Q(x, a)$;
- con probabilità ε viene selezionata un'azione casuale tra quelle disponibili.

Il parametro ε controlla il livello di esplorazione. All'inizio dell'apprendimento è utile scegliere un valore relativamente alto di ε per favorire l'esplorazione, mentre nel tempo ε può essere ridotto per privilegiare l'*exploitation* delle conoscenze acquisite.

Negli esercizi, la strategia ε -greedy viene utilizzata quando l'ambiente è stocastico e il modello non è noto, in quanto permette di apprendere efficacemente la funzione Q garantendo al tempo stesso una sufficiente esplorazione dello spazio delle azioni.

Il Q-learning converge alla Q-function ottima Q^* se sono soddisfatte le seguenti condizioni:

- tutte le coppie stato-azione sono visitate infinite volte;
- il learning rate α decresce in modo appropriato;

- il processo di apprendimento continua sufficientemente a lungo.

In tali condizioni, la policy derivata dalla Q-function approssimata converge alla policy ottima:

$$\pi^*(x) = \arg \max_{a \in A} Q^*(x, a).$$

15.8.3 Esempio: Slot Machine e k-Armed Bandit

Un esempio classico di One-State Markov Decision Process è il problema della slot machine a più leve, noto come *k-armed bandit*. In questo scenario, l'agente si trova sempre nello stesso stato x_0 e può scegliere una tra n azioni $A = \{a_1, \dots, a_n\}$, ciascuna corrispondente a una leva della slot machine.

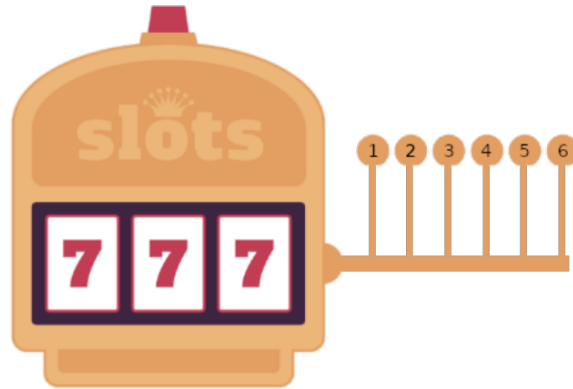


Figure 15.4: Esempio di One-State MDP: slot machine con n leve. Ogni azione produce una ricompensa casuale, mentre lo stato rimane invariato.

Il problema è modellato come un MDP:

$$\mathcal{M} = \langle \{x_0\}, \{a_1, \dots, a_n\}, \delta, r \rangle$$

dove la funzione di transizione è definita come:

$$\delta(x_0, a_i) = x_0, \quad \forall a_i \in A,$$

e la funzione di ricompensa dipende esclusivamente dall'azione:

$$r(x_0, a_i, x_0) = r(a_i).$$

Le ricompense sono in generale non deterministiche e associate a distribuzioni di probabilità ignote. L'obiettivo dell'agente è determinare l'azione che massimizza il valore atteso della ricompensa, bilanciando esplorazione e sfruttamento.

Q-function nel k-armed bandit

Nel caso del k-armed bandit, essendo presente un solo stato s , la funzione valore azione (Q-function) si riduce a una funzione dipendente esclusivamente dall'azione:

$$Q(s, a) \equiv Q(a)$$

La Q-function rappresenta il valore atteso della ricompensa ottenuta se viene selezionata l'azione a :

$$Q(a) = \mathbb{E}[r \mid a]$$

Poiché le distribuzioni di ricompensa associate alle azioni sono sconosciute, la Q-function non è nota e deve essere stimata a partire dall'esperienza.

15.8.4 Problema e concetto di soluzione

Il problema del k-armed bandit consiste nello stimare la Q-function associata a ciascuna azione e nel selezionare una policy che massimizzi la ricompensa attesa cumulativa.

Una policy π associa a ogni istante temporale un'azione da eseguire. Nel caso ideale, la policy ottima seleziona sempre l'azione con valore atteso massimo:

$$\pi^* = \arg \max_{a \in \mathcal{A}} Q(a)$$

Tuttavia, poiché i valori $Q(a)$ sono inizialmente ignoti, l'agente deve bilanciare:

- *exploration*: selezionare azioni poco esplorate per migliorare la stima di $Q(a)$;
- *exploitation*: selezionare l'azione che massimizza la stima corrente di $Q(a)$.

Apprendimento della Q-function

La stima della Q-function può essere aggiornata incrementalmente utilizzando una regola di apprendimento di tipo reinforcement learning. Dopo aver selezionato l'azione a_t e osservato la ricompensa r_t , la stima viene aggiornata come:

$$Q_{t+1}(a_t) = Q_t(a_t) + \alpha(r_t - Q_t(a_t))$$

dove:

- $\alpha \in (0, 1]$ è il learning rate;
- r_t è la ricompensa osservata al tempo t .

Questa regola rappresenta un caso particolare del Q-learning, semplificato dall'assenza di dinamica di stato e di ricompense future.

15.9 Altri algoritmi di reinforcement learning

Oltre al Q-learning, esistono altri algoritmi per l'apprendimento in Markov Decision Processes non deterministici. Tra questi:

- **SARSA**, un metodo *on-policy* che valuta la policy effettivamente seguita dall'agente;
- **Temporal Difference (TD)**, che combina idee di Monte Carlo e programmazione dinamica;
- approcci *model-based*, che stimano esplicitamente il modello dell'ambiente per pianificare le azioni.

RIEPILOGO

Reinforcement Learning – Concetti chiave

Il *Reinforcement Learning* studia problemi decisionali sequenziali in cui un agente apprende interagendo con un ambiente, senza disporre di esempi supervisionati, ma solo di segnali di ricompensa.

Markov Decision Process (MDP)

Un problema di reinforcement learning è formalizzato tramite un *Markov Decision Process*:

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, p, r, \gamma \rangle$$

dove:

- $\mathcal{S}$ è l'insieme degli stati;
- $\mathcal{A}$ è l'insieme delle azioni;
- $p(s' | s, a)$ è la probabilità di transizione;
- $r(s, a)$ è la funzione di ricompensa;
- $\gamma \in [0, 1)$ è il fattore di sconto.

La proprietà di Markov implica che l'evoluzione futura del sistema dipende solo dallo stato corrente e dall'azione selezionata.

Policy e funzioni di valore

Una *policy* π definisce il comportamento dell'agente, specificando l'azione da eseguire in ciascuno stato. Le prestazioni di una policy sono valutate tramite:

- la funzione valore di stato $V^\pi(s)$;
- la funzione valore azione $Q^\pi(s, a)$.

La policy ottima π^* massimizza il valore atteso cumulativo delle ricompense.

Equazioni di Bellman

Le equazioni di Bellman forniscono una caratterizzazione ricorsiva delle funzioni di valore. Nel caso ottimo, esse esprimono il principio di ottimalità e costituiscono la base per gli algoritmi di apprendimento e pianificazione.

Reinforcement Learning senza modello

Nel reinforcement learning l'agente non conosce né le probabilità di transizione né la funzione di ricompensa. Gli algoritmi *model-free* apprendono direttamente le funzioni di valore a partire dall'esperienza, senza stimare esplicitamente il modello dell'ambiente.

Q-learning

Il Q-learning è un algoritmo model-free che apprende la Q-function ottima tramite aggiornamenti incrementali basati sull'errore di Bellman. La policy è ottenuta selezionando l'azione che massimizza il valore stimato della Q-function.

k-Armed Bandit

Il k-armed bandit è un caso particolare di reinforcement learning modellabile come MDP a singolo stato. In questo caso la Q-function dipende solo dall'azione:

$$Q(a) = \mathbb{E}[r \mid a]$$

Il problema consiste nello stimare i valori attesi delle azioni e nel bilanciare *exploration* ed *exploitation* per massimizzare la ricompensa cumulativa.

Action selection

Poiché i valori delle azioni sono inizialmente ignoti, è necessario adottare strategie di selezione dell'azione che consentano l'esplorazione dell'ambiente, come ad esempio policy ε -greedy.

Obiettivo del Reinforcement Learning

L'obiettivo finale è apprendere una policy ottima che massimizzi la somma attesa delle ricompense nel tempo, attraverso l'interazione iterativa con l'ambiente e l'aggiornamento delle stime di valore.

GLOSSARIO

Dataset (insieme di dati) Un *dataset* è un insieme strutturato di istanze utilizzato per l'addestramento, la validazione o la valutazione di un modello di Machine Learning. Può essere visto come una tabella in cui ogni riga rappresenta un'istanza e ogni colonna una feature.

Instance (istanza, esempio) Un *instance* è un singolo esempio contenuto nel dataset. Rappresenta un caso specifico del problema e costituisce l'unità di base su cui il modello effettua una predizione.

Feature (attributo, caratteristica) Una *feature* è una caratteristica numerica o categoriale che descrive un'istanza. Ogni istanza è rappresentata da un vettore di feature che ne codifica le proprietà rilevanti per il problema di apprendimento.

Label / Target (etichetta, valore di uscita) La *label* (o *target*) è il valore di output corretto associato a un'istanza in un problema di *supervised learning*. Rappresenta ciò che il modello deve imparare a predire a partire dalle feature.

Nel caso di:

- *classificazione*, la label appartiene a un insieme finito di classi;
- *regressione*, la label è un valore numerico continuo.

Model (modello) Un *model* è una funzione, parametrica o non parametrica, che associa le feature di un'istanza a una predizione dell'output. Il modello rappresenta ciò che viene appreso dai dati e viene utilizzato per effettuare predizioni su nuove istanze.

Learning Algorithm (algoritmo di apprendimento) Un *learning algorithm* è la procedura che stima i parametri del modello a partire dal dataset di training, ottimizzando una funzione di costo. L'algoritmo di apprendimento non effettua predizioni, ma serve esclusivamente a costruire o aggiornare il modello.

Prediction (predizione) Una *prediction* è l'output prodotto dal modello quando viene applicato a un'istanza. La predizione rappresenta la stima del valore di output e non coincide necessariamente con la label corretta.

Loss Function (funzione di perdita, funzione di costo) Una *loss function* misura l'errore commesso dal modello su una singola istanza, confrontando la predizione con la label corretta. La loss function fornisce un segnale numerico che guida l'algoritmo di apprendimento nel migliorare il modello durante la fase di training.

Metric (metrica di valutazione) Una *metric* è una misura utilizzata per valutare le prestazioni complessive di un modello su un insieme di dati. A differenza della loss function, la metrica non viene utilizzata per l'apprendimento del modello, ma solo per confrontare modelli o stimarne la capacità di generalizzazione.

Esempi comuni di metriche sono l'accuracy, la precision, la recall e l'RMSE.

Training Set (insieme di addestramento) Il *training set* è la parte del dataset utilizzata per addestrare il modello, cioè per stimarne i parametri tramite l'algoritmo di apprendimento. Durante il training, il modello minimizza la loss function sui dati di training.

Test Set (insieme di test) Il *test set* è un insieme di dati separato dal training set e utilizzato esclusivamente per valutare le prestazioni del modello su istanze non viste durante l'addestramento. Il test set fornisce una stima della capacità di generalizzazione del modello.

Validation Set (insieme di validazione) Il *validation set* è un insieme di dati utilizzato durante lo sviluppo del modello per confrontare diverse scelte progettuali, come iperparametri o architetture. I dati di validazione non vengono usati per addestrare il modello, ma per guidarne la selezione.

Mean (media) La *mean* è il valore medio di un insieme di dati. Rappresenta il “centro” dei dati, cioè il valore attorno al quale le osservazioni tendono a concentrarsi.

Variance (varianza) La *variance* misura quanto i dati sono dispersi attorno alla media. Una varianza piccola indica dati molto concentrati, una varianza grande indica dati molto sparsi.

Covariance (covarianza) La *covariance* misura come due variabili variano insieme. Se è positiva, le due variabili tendono ad aumentare o diminuire insieme; se è negativa, una tende ad aumentare quando l'altra diminuisce; se è circa zero, non c'è una relazione lineare evidente.

Covariance Matrix (matrice di covarianza) La *covariance matrix* raccoglie tutte le covarianze tra le feature di un dataset. Sulla diagonale contiene le varianze delle singole feature, mentre fuori diagonale contiene le covarianze tra coppie di feature. Descrive la forma e l'orientamento della distribuzione dei dati.

Distribution (distribuzione di probabilità) Una *distribution* descrive come i valori di una variabile sono distribuiti, cioè quali valori sono più probabili e quali meno. In altre parole, una distribuzione assegna una probabilità (o una densità) ai possibili valori che una variabile può assumere.

Nel Machine Learning, una distribuzione viene usata per modellare i dati, le osservazioni o le incertezze associate a un fenomeno reale.

Gaussian Distribution / Normal Distribution (distribuzione gaussiana / normale)

La *Gaussian distribution* è una distribuzione continua a forma di campana, completamente descritta da media e varianza. La media indica il centro della distribuzione, la varianza ne controlla la larghezza. I termini “gaussiana” e “normale” indicano la stessa distribuzione.

Discrete Distribution (distribuzione discreta) Una *discrete distribution* è una distribuzione di probabilità definita su un insieme finito o numerabile di valori. A ciascun valore è associata una probabilità, e la somma di tutte le probabilità è pari a uno.

Observations (osservazioni) Le *observations* sono i dati osservati, cioè i valori effettivamente misurati o raccolti dal mondo reale. Nel Machine Learning, le osservazioni costituiscono il dataset di training.

Weights (pesi) I *weights* sono parametri numerici di un modello che determinano l'importanza delle feature. Pesi grandi indicano feature più influenti, pesi piccoli indicano feature meno rilevanti.

Likelihood (verosimiglianza) La *likelihood* misura quanto è plausibile osservare i dati assumendo fissati i parametri di un modello. Non è una probabilità dei parametri, ma una misura di quanto bene il modello spiega i dati osservati.

Log-Likelihood (log-verosimiglianza) La *log-likelihood* è il logaritmo della likelihood. Viene utilizzata perché semplifica i calcoli, trasforma prodotti in somme e rende più stabile l'ottimizzazione numerica. Massimizzare la log-likelihood equivale a massimizzare la likelihood.

Simbolo	Nome	Significato / uso
$\wedge$	AND (congiunzione)	vero se entrambi i valori sono veri
$\vee$	OR (disgiunzione)	vero se almeno uno dei valori è vero
$\neg$ / $\neg$	NOT (negazione)	inverte il valore di verità
$\rightarrow$	Implication (implicazione)	$A \rightarrow B$: se A allora B
$\leftrightarrow$	Iff / Biconditional (doppia implicazione)	vero se A e B hanno lo stesso valore
$\oplus$	XOR (exclusive or)	vero se esattamente uno è vero
$\forall$	For all (per ogni)	quantificatore universale
$\exists$	Exists (esiste)	quantificatore esistenziale
$\models$	Entails (implica logicamente)	$A \models B$: B è vero in ogni modello in cui A lo è
$\vdash$	Derives (derivabile)	$A \vdash B$: B è deducibile da A

Table 15.1: Simboli logici fondamentali